

uc3m

Universidad
Carlos III
de Madrid

Máster Universitario en Ingeniería Informática

Trabajo de Fin de Máster

«Diseño e Implementación de un
Sistema de Gestión del
Conocimiento Multilingüe Basado
en LLMs como Alternativa Abierta a
otros sistemas existentes»

Autor

Diego Picazo García

Tutora

Anabel Fraga Vázquez

Leganés, Madrid

Septiembre 2026



© 2026 Diego Picazo García. Todos los derechos reservados.

Esta obra se encuentra sujeta a la licencia Creative Commons.

Reconocimiento – No Comercial – Sin Derivadas 4.0 International

If you don't make mistakes, you're not working on hard enough problems. And that's a big mistake.

— **Frank Wilczek**

RESUMEN

El presente documento propone el desarrollo de un sistema de gestión del conocimiento accesible y multilingüe, basado en [Modelos de Lenguaje a Gran Escala \(LLM\)](#), con el objetivo de abstraer la complejidad inherente a herramientas tradicionales como *Protégé*, ampliamente utilizadas en el ámbito de la ingeniería del conocimiento pero caracterizadas por una elevada curva de aprendizaje. El sistema integra capacidades de análisis de texto, habilitando casos de uso como la interoperabilidad semántica entre plataformas, la **generación de conocimiento** a partir de datos no estructurados y la **automatización** de procesos de razonamiento, todo ello con el propósito de democratizar el acceso a la creación y gestión del conocimiento. Para ello, se contempla el despliegue local de soluciones de código abierto [1], **garantizando** un estricto alineamiento con el **marco regulatorio europeo vigente** [2] (p. ej., Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016 (RGPD) [3] y Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024 (AI Act) [4]), haciendo uso de validadores como *EU AI Act Compliance Checker* [5].

Cabe destacar que la gestión del ciclo de vida del software, el plan de proyecto y la monitorización de métricas se han regido por la aplicación de una **metodología en cascada** [6]. Este enfoque se ha sustentado en el aprendizaje continuo, asegurando así una preparación completa y el cumplimiento de los hitos del proyecto.

A nivel arquitectónico, la solución se estructura mediante un motor de procesamiento lógico, *backend*, encargado de **interpretar** la **entrada** en lenguaje **natural**. Posteriormente, una interfaz gráfica, *frontend*, facilita la representación **visual** e interactiva del **grafo de conocimiento generado**. Este flujo de trabajo permite al usuario validar la ontología antes de proceder a su exportación en formatos estándar [World Wide Web Consortium \(W3C\)](#), tales como [Marco de Descripción de Recursos \(RDF\)](#) o *Turtle*, preservando así la compatibilidad retrospectiva con otros competidores semánticos del sector.

Finalmente, desde una perspectiva de infraestructura tecnológica, el entorno de inferencia ha sido rigurosamente optimizado para expresar el rendimiento de **hardware acelerado local**, mediante unidades de procesamiento gráfico de alto nivel. No obstante, la arquitectura del sistema **soporta** una migración a un paradigma escalable en la **nube** pública o privada (véase [Capítulo 8](#)) donde la capacidad de procesamiento queda acotada principalmente por el coste de aprovisionamiento de recursos.

Palabras clave: Inteligencia Artificial • Ingeniería del Conocimiento • Modelo de Lenguaje de Gran Tamaño (LLM) • Ontología • Arquitectura Orientada a Eventos

ABSTRACT

This document proposes the development of an accessible and multilingual knowledge management system, based on [LLM](#), with the aim of abstracting the inherent complexity of traditional tools such as *Protégé*, widely used in the field of knowledge engineering but characterised by a steep learning curve. The system integrates text analysis capabilities, enabling use cases such as semantic interoperability between platforms, the **generation of knowledge** from unstructured data and the **automation** of reasoning processes, all with the purpose of democratizing access to the creation and management of knowledge. To this end, the local deployment of open-source solutions [\[1\]](#) is considered, **guaranteeing** strict alignment with the **current European regulatory framework** [\[2\]](#) (e.g., [GDPR](#) [\[3\]](#) and the [AI Act](#) [\[4\]](#)), making use of validators such as the EU AI Act Compliance Checker [\[5\]](#).

It should be noted that the software development life cycle, the project plan, and the monitoring of metrics have been governed by the application of a **waterfall methodology** [\[6\]](#). This approach has been supported by the continuous learning, thereby ensuring complete preparation and the fulfilment of the project milestones.

At the architectural level, the solution is structured through a logical processing engine, the backend, responsible for **interpreting** the **natural** language **input**. Subsequently, a graphical user interface, the frontend, facilitates the **visual** and interactive representation of the **generated knowledge graph**. This workflow allows the user to validate the ontology prior to its export into standard formats [W3C](#), such as [RDF](#) or Turtle, thus preserving backward compatibility with other semantic competitors in the sector.

Finally, from a technological infrastructure perspective, the inference environment has been rigorously optimised to maximise the performance of **local accelerated hardware**, by means of high-end graphical processing units. Nevertheless, the system architecture **supports** a migration to a scalable, private or public, **cloud paradigm** (see [Chapter 8](#)), where the processing capacity is bounded primarily by the cost of resource provisioning.

Keywords: Artificial Intelligence • Knowledge Engineering • Large Language Model (LLM) • Ontology • Event Driven Architecture

AGRADECIMIENTOS

Es una tradición, bien conocida, dar las gracias a aquellos seres queridos que me acompañan, inseparables como mi propia sombra, en cada paso del camino. Gracias, papá; gracias, mamá, por confiar una vez más en mi proyección, en mi ambición y ofrecerme toda clase de oportunidades. Conozco vuestros sacrificios, vuestros días aburridos y austeros, que me dan energía cada día para evolucionar, progresar y correr a casa, no con un diploma más, sino con la mochila cargada de nuevos conocimientos y experiencias que me permitan, en el futuro, daros la mano en la cima y mostraros el horizonte que se vislumbra. Tengo muy presentes a mis amigos cercanos —Olmo, Alejandro y Jorge—, a mi hermano —Mario— y a mi querida alma gemela —Emily—, que siempre han apoyado mis locuras, mi ansia de aprendizaje y han escuchado atentamente mis problemas entre videojuegos, viajes y partidos de pádel.

Es imposible olvidar a mis maestros, que, durante mi paso breve y enriquecedor por sus aulas, han transmitido su conocimiento y enseñanza a la perfección. Agradezco a Anabel Fraga por su amabilidad, preciosas explicaciones y tutela en este proyecto. A Carlos Galán, el gran músico, por sus lecciones de la Constitución Española y a Sergio Pastrana por acercar la comunidad científica de cibercrímenes a las aulas. Sin olvidar a mis compañeros de clase, en particular a Francisco, Luis y Lucas. Han hecho del aprendizaje y de los proyectos colaborativos un pasaje interesante y ameno.

Hago extensivo mi agradecimiento a Verisure —Securitas Direct—, empresa que apuesta cada día por mi desarrollo y aprendizaje, facilitando económicamente mi presencia en el plan de formación impartido por la UC3M. Me enorgullece anunciar que mi travesía retorna con nuevo conocimiento listo para ser trasladado al mundo laboral.

Finalmente, quiero dar las gracias a la comunidad de código abierto. Es magnífico observar que personas de distintas culturas, religiones y ámbitos sociales unen fuerzas para una misma causa. Espero que en el futuro tengan la reputación que merecen.

TABLA DE CONTENIDOS

Capítulo 1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
Capítulo 2. Estado del Arte	3
2.1. De la Web Sintáctica a la Semántica	3
2.2. Inferencia de Conocimiento del Presente	4
2.3. De la Alucinación al Determinismo	5
2.4. Orquestación de Agentes Inteligentes	6
2.5. Arquitectura Orientada a Eventos	7
2.6. OntoNova	8
Capítulo 3. Plan de Proyecto	9
3.1. Planificación	9
3.2. Presupuesto	12
3.3. Marco Regulatorio	15
3.4. Entorno Socioeconómico	18
Capítulo 4. Análisis	19
4.1. Gestión de Requisitos	19
4.2. Exposición de Requisitos	20
Capítulo 5. Diseño	27
5.1. Gestión del Diseño	27
5.2. Arquitectura del Sistema	29
5.3. Diseño de Detalle	34
5.4. Diseño de Seguridad	38
5.5. Diseño de Despliegue	39
5.6. Trazabilidad de Requisitos	40
Capítulo 6. Implementación	43
6.1. Entorno y Organización del Código	43
6.2. Hitos de Implementación	44
6.3. Dificultades y Decisiones Técnicas	46
Capítulo 7. Verificación	49
7.1. Estrategia y Balance de Pruebas Automáticas	49

7.2. Estándar IEEE para la Verificación de Requisitos	51
7.3. Verificación del Cumplimiento Normativo	58
Capítulo 8. Propuesta de Exposición Pública	59
8.1. Impacto en la Arquitectura del Sistema	59
8.2. Impacto en el Diseño de Seguridad	61
8.3. Impacto en el Diseño del Despliegue	62
8.4. Impacto en la Legislación Aplicable	63
Capítulo 9. Conclusiones	65
9.1. Conclusiones del Proyecto	65
9.2. Conclusiones Personales	67
9.3. Trabajos Futuros	67
Bibliografía	69
Glosario	73
Apéndice A. Cuestionario de Usabilidad	
Apéndice B. Resultados del Cumplimiento de la Ley de IA de la UE	
Apéndice C. Declaración de uso de IA generativa	

ÍNDICE DE FIGURAS

Figura 1.1	Objetivos de desarrollo sostenible.	2
Figura 2.1	Modelo arquitectónico de un transformer	4
Figura 2.2	Funcionamiento del modo JSON	5
Figura 2.3	Funcionamiento de las salidas estructuradas	6
Figura 3.1	Metodología en cascada aplicada	10
Figura 3.2	Diagrama de estimación de tiempo.	11
Figura 4.1	Diagrama de casos de uso en lenguaje UML.	26
Figura 5.1	Diagrama C4 del contexto del sistema.	29
Figura 5.2	Diagrama C4 de los contenedores del sistema.	30
Figura 5.3	Diagrama C4 de los componentes de la aplicación web.	32
Figura 5.4	Diagrama C4 de los componentes del servicio de orquestación.	33
Figura 5.5	Diagrama de actividad del proceso multi-agente de generación.	35
Figura 5.6	Diagrama C4 de despliegue del sistema.	39
Figura 6.1	Correspondencia entre el código y los componentes C4.	44
Figura 6.2	Del contrato a la gramática de decodificación guiada.	45
Figura 6.3	Generación de una ontología en curso con progreso por etapas.	45
Figura 6.4	Ontología del dominio de la investigación sobre el lienzo interactivo.	46
Figura 6.5	Entrada guiada de un nodo agente en fase de reintento.	47
Figura 6.6	Ontología del dominio de la investigación con conectividad degradada.	48
Figura 7.1	Pirámide de pruebas automáticas de OntoNova.	50
Figura 7.2	Mensaje de error por exceso de caracteres.	52
Figura 7.3	Mensaje de reintento de corrección.	53
Figura 7.4	Validación sintáctica de exportaciones RDF y Turtle.	53
Figura 7.5	Mensaje de error por exceso de tamaño de archivo.	54
Figura 7.6	Informe de <i>Google Lighthouse</i> sobre OntoNova.	55
Figura 8.1	Diagrama C4 del contexto del sistema bajo exposición pública.	60
Figura 8.2	Diagrama operacional del sistema bajo exposición pública.	60
Figura 8.3	Puertas de seguridad que atraviesa cada petición en la exposición pública. .	61
Figura 8.4	Valores del <i>chart</i> por escenario, con el empaquetado común.	62
Figura 8.5	Ciclo de integración continua y despliegue continuo.	63

ÍNDICE DE TABLAS

Tabla 2.1	Comparativa de herramientas de orquestación	6
Tabla 2.2	Comparativa de paradigmas de comunicación cliente-servidor	7
Tabla 2.3	Comparativa de herramientas para la gestión y modelado ontológico	8
Tabla 3.1	Desglose estimado de horas por fase del proyecto	10
Tabla 3.2	Información del proyecto	12
Tabla 3.3	Costes personales	13
Tabla 3.4	Costes materiales	13
Tabla 3.5	Costes indirectos	14
Tabla 3.6	Balance total	14
Tabla 3.7	Propuesta de venta	15
Tabla 3.8	Licencias de complementos	17
Tabla 4.1	Contenido de un requisito	20
Tabla 4.2	Requisitos funcionales de usuario	21
Tabla 4.3	Requisitos no funcionales de usuario	22
Tabla 4.4	Requisitos funcionales del sistema	23
Tabla 4.5	Requisitos no funcionales del sistema	23
Tabla 5.1	Comparativa de alternativas del cliente	30
Tabla 5.2	Comparativa de alternativas del motor de inferencia	31
Tabla 5.3	Entidades del contrato universal de datos	36
Tabla 5.4	Catálogo de eventos del flujo de generación	37
Tabla 5.5	Matriz de trazabilidad de requisitos de usuario a elementos de diseño	40
Tabla 5.6	Matriz de trazabilidad de requisitos del sistema a elementos de diseño	41
Tabla 6.1	Conjunto tecnológico de la implementación	43
Tabla 7.1	Pruebas de aceptación y variabilidad de la cadena de inferencia	50
Tabla 7.2	Resultado de la verificación de los requisitos incluidos	51
Tabla 7.3	Latencia de eventos de progreso	52
Tabla 7.4	Latencia de interacción del lienzo en cien rondas por operación	54
Tabla 7.5	Interacciones necesarias por operación de edición del lienzo	56
Tabla 7.6	Resultados del cuestionario SUS por participante	57
Tabla B.1	Respuestas registradas en el cuestionario de conformidad	79

CAPÍTULO 1

INTRODUCCIÓN

Este capítulo presenta la motivación del trabajo, [Sección 1.1](#), exponiendo qué caso de uso se persigue y qué objetivos principales y secundarios, [Sección 1.2](#), contempla.

1.1. Motivación

El ser humano siente desde sus orígenes la **necesidad intrínseca de estructurar y nombrar el mundo** que le rodea para poder comprenderlo. Con la llegada de la era digital y la expansión desorbitada de internet, esa necesidad se ha trasladado a un océano de datos casi inabarcable. Hoy en día, se interactúa con sistemas de información complejos con una naturalidad pasmosa, dando por sentada la *magia tecnológica* que permite a las máquinas procesar y conectar el conocimiento de forma transparente. En este contexto, **el reto** no consiste únicamente en almacenar grandes volúmenes de información, sino en **dotarlos de un significado profundo, universal y estructurado**.

Históricamente, la traducción del pensamiento humano a la lógica de un computador ha requerido de intérpretes altamente especializados. La Web 3.0 (véase [Sección 2.1](#)) nace con la promesa de crear un lenguaje universal, utilizando estándares matemáticamente precisos para modelar la realidad. Sin embargo, este proceso ha dependido tradicionalmente de herramientas abstractas y lenguajes crípticos, un camino tumultuoso que ha alejado a los verdaderos expertos del dominio —médicos, abogados, educadores— de la creación directa de sus propios modelos de conocimiento.

Con la irrupción contemporánea de la **Inteligencia Artificial (IA) generativa**, surge la tentación lógica de intentar atajar el problema. **Solicitar** directamente a un modelo de lenguaje **que escuche** al usuario y **escriba** la semántica que necesita parece una **solución atractiva**. **Pero** aquí radica una incompatibilidad fundamental. El lenguaje **humano** es fluido, **creativo** y orgánico; por el **contrario**, los lenguajes formales de las **máquinas** son **inflexibles**. Una sola asunción ambigua por parte de la **IA** puede destruir por completo el

significado del conjunto, provocando fallos irreversibles que la máquina rechaza categóricamente.

En consecuencia, **nace** la necesidad de un **enfoque híbrido**, que actúa como guardarraíl para la **IA**, e interconecta la creatividad humana y la precisión formal de los sistemas. En apartados posteriores **se exponen** los requisitos, el diseño e implementación de un **sistema**, de código libre [1], de **gestión del conocimiento** accesible, multilingüe y basado en **LLM**, abordando el enfoque mencionado y enfrentando las debilidades de competidores semánticos de gran escala, como Protégé, que han demostrado poseer una curva de aprendizaje elevada. **OntoNova** integra capacidades de análisis de lenguaje natural, que habilita casos de uso como la **interoperabilidad** de sistemas de información, la **generación de conocimiento** a partir de datos no estructurados y la **automatización** de procesos de razonamiento, todo ello con el propósito de democratizar el acceso a la creación y gestión del conocimiento. Adicionalmente, resuelve el problema del lienzo en blanco para los ingenieros de conocimiento, pues los verdaderos expertos les proporcionan de partida la semántica que necesitan en un formato **W3C**.

1.2. Objetivos

El propósito principal consiste en diseñar y desarrollar un sistema del conocimiento multilingüe basado en **LLM**, que actúe como alternativa accesible y de código abierto a herramientas tradicionales, estableciendo un alcance apropiado al marco de este trabajo.

A continuación, se listan los objetivos secundarios que complementan el objetivo principal:

- **O1:** Obtener nuevos conocimientos en **IA**, indagando en **LLM**, arquitectura, capacidades y limitaciones, hasta adquirir, al menos, el nivel tres en la taxonomía de Bloom [7].
- **O2:** Disminuir la barrera de entrada a la elaboración de ontologías, ofreciendo una alternativa más accesible y cercana al usuario.
- **O3:** Proporcionar un sistema multilingüe que soporte, como mínimo, español e inglés.
- **O4:** Cumplir con el marco regulatorio europeo vigente [2] (p. ej., **RGPD** [3] y **AI Act** [4]), haciendo uso de validadores como *EU AI Act Compliance Checker* [5].
- **O5:** Diseñar una arquitectura siguiendo los estándares de seguridad, escalabilidad y patrones de diseño de *software* [8].
- **O6:** Alinear el proyecto con los Objetivos de Desarrollo Sostenible [9], representados en la **Figura 1.1**.

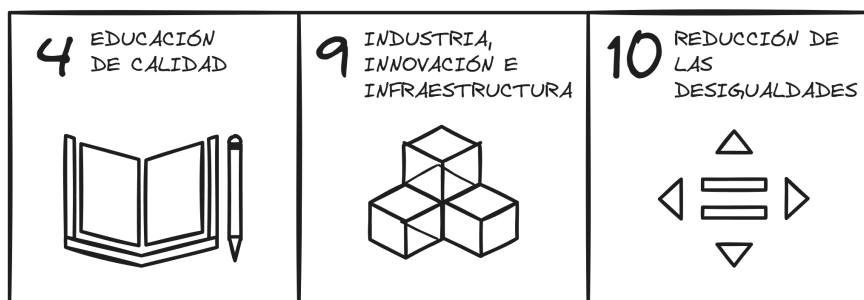


Figura 1.1. Objetivos de desarrollo sostenible.

CAPÍTULO 2

ESTADO DEL ARTE

Este capítulo detalla la evolución a la web semántica, [Sección 2.1](#), la inferencia del conocimiento, [Sección 2.2](#), y la alucinación del LLM, [Sección 2.3](#), concluyendo con la orquestación de agentes inteligentes, [Sección 2.4](#), en arquitecturas de eventos, [Sección 2.5](#).

2.1. De la Web Sintáctica a la Semántica

A comienzos del siglo XXI, surge una crítica esencial [10] a la [World Wide Web \(WWW\)](#), señalando que la mayor parte de su contenido está diseñado para el consumo humano y no para ser procesado significativamente por programas informáticos. Aunque los ordenadores pueden analizar con suma eficacia el diseño o forma de una página web, carecen de métodos fiables para interpretar su semántica de forma aislada. En consecuencia, nace la Web Semántica o Web 3.0, como una extensión progresiva de la web existente. En este nuevo paradigma, la información toma estructura y un significado bien definido, lo que facilita y mejora sustancialmente la cooperación entre personas y máquinas.

Para que esta visión se materialice y adquiera una estandarización global, los computadores necesitan acceder a colecciones estructuradas de datos y a conjuntos de reglas de inferencia que les permitan elaborar un razonamiento automatizado. La Web Semántica asume que la versatilidad requiere tolerar y aceptar que existen preguntas sin respuesta. Por lo tanto, el desafío radica en la creación de un lenguaje capaz de expresar simultáneamente tanto los datos como las reglas necesarias para razonar sobre ellos. Por un lado, [Lenguaje de Marcado Extensible \(XML\)](#) ofrece las características previas, permitiendo adicionalmente la creación de estructuras arbitrarias con etiquetas ocultas. Por otro lado, [RDF](#) articula el significado previo mediante tripletas que funcionan como afirmaciones compuestas por sujeto, verbo y objeto. Para evitar la ambigüedad del lenguaje natural, cada elemento lógico se identifica con un [Identificador de Recursos Uniforme \(URI\)](#), evitando confusiones semánticas.

No obstante, para que distintas bases de datos puedan interoperar correctamente, es preciso que reconozcan cuándo están utilizando identificadores diferentes para referirse a un mismo concepto. Esta necesidad queda resuelta con el concepto de **ontología**, que queda definido como el campo que define las relaciones entre distintos términos. Las ontologías proporcionan **taxonomías** para clasificar objetos y reglas de inferencia que dotan a los sistemas de una potente capacidad deductiva. Además, actúan como mecanismo para establecer relaciones de equivalencia que resuelven discrepancias de vocabulario entre distintos dominios de información. El **Lenguaje Web de Ontologías (OWL)** [11] se apoya en el marco y lenguaje mencionados, pues proporciona una lógica de descripción matemáticamente precisa, permitiendo modelar clases, propiedades e individuos bajo un formalismo estricto.

Históricamente, la confección de las ontologías se produce en herramientas como Protégé. Representa el estándar *de facto* en la industria y en la academia, pues nace para combatir el cuello de botella de adquisición del conocimiento [12]. Su interfaz rígida basada en jerarquías textuales y asunciones lógicas genera una barrera de entrada elevada que requiere la intervención de un ingeniero del conocimiento cualificado. Este hecho evidencia la necesidad de alternativas que presenten una capa de abstracción y acerquen el modelado semántico al verdadero experto del dominio, presentando un lienzo de partida al ingeniero.

2.2. Inferencia de Conocimiento del Presente

Mitigando el cuello de botella mencionado, la investigación en el **Procesamiento de Lenguaje Natural (NLP)** evoluciona hacia la extracción automatizada de relaciones a partir de texto. Con la consolidación de los *transformers* [13], ilustrada en la **Figura 2.1**, y la llegada del **LLM**, la inferencia de estructuras semánticas con información no estructurada crece exponencialmente.

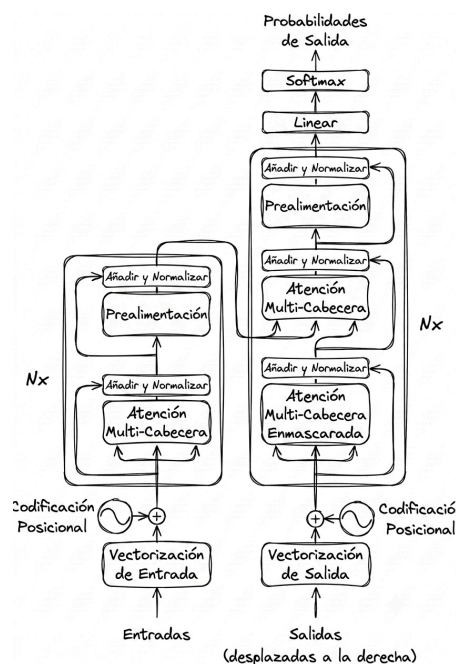


Figura 2.1. Modelo arquitectónico de un transformer [13].

Sin embargo, un **LLM** es intrínsecamente estocástico y probabilístico. Queda advertido el riesgo de solicitar a un modelo generativo la escritura directa de sintaxis formal [14], p. ej., una ontología en código **OWL**. Una simple omisión de un carácter invalida por completo el documento resultante frente a los razonadores lógicos. En consecuencia, delegar la generación directa de una ontología a la **IA** resulta inviable en entornos productivos sin la existencia de mecanismos de contención, véase [Sección 2.3](#), que garanticen el determinismo de las respuestas.

2.3. De la Alucinación al Determinismo

Al comienzo, con la integración del **LLM** entre sistemas, para resolver la dicotomía entre la creatividad fluida de la **IA** y la rigidez formal de la web semántica (véase [Sección 2.1](#)), la obtención de respuestas estructuradas ha dependido de la ingeniería de instrucciones y de técnicas heurísticas de posprocesamiento. Peticiones imperativas en lenguaje natural — como «Responde únicamente en formato **Notación de Objetos de JavaScript (JSON)**» — se convirtieron en el estándar. Sin embargo, el **LLM** introduce, frecuentemente, preámbulos o epílogos que provocan el fallo de algoritmos clásicos de procesamiento de texto.

Para mitigar esta falta de determinismo sintáctico, la industria presenta el denominado **modo JSON** (véase [Figura 2.2](#)). Al fijar el `text.format` a `{"type": "json_object"}`, el modelo es forzado a generar una cadena de caracteres que constituya un objeto **JSON** válido, suprimiendo texto auxiliar [15]. No obstante, aunque este modo garantiza la validez sintáctica del formato, no asegura el cumplimiento de una estructura de datos predefinida. El modelo sigue omitiendo claves obligatorias, inventando nuevas propiedades [14] o alterando los tipos de datos esperados, lo que impide establecer un contrato de datos fiable al integrarse con otros sistemas, p. ej., motores de inferencia lógica o aplicaciones web.

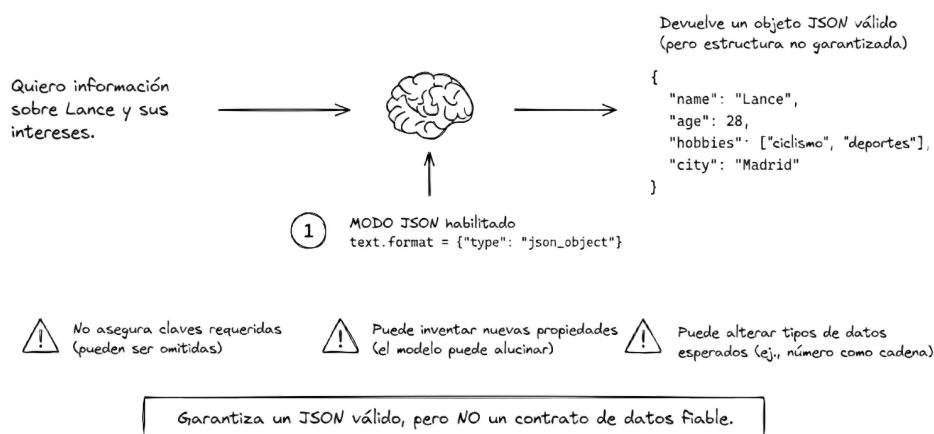


Figura 2.2. Funcionamiento del modo JSON [15].

En definitiva, la generación de salidas deterministas alcanza su clímax con las **salidas estructuradas** [15] respaldadas por esquemas **JSON** de validación, observable en la [Figura 2.3](#). Se garantiza que la información extraída por la **IA** cumpla con los tipos de datos, cardinalidades y formatos requeridos antes de ser procesada por otros sistemas.

Esta evolución interviene en el proceso de decodificación del modelo, *grammar-based sampling* [15], donde el motor de inferencia traduce el esquema JSON a una gramática formal, enmascarando las probabilidades de caracteres, *logit filtering* [15]. En consecuencia, los *tokens* resultantes cumplen estrictamente el contrato de datos, consolidando como fuente de verdad un esquema JSON común, presentado en el Capítulo 5, implementado en el Capítulo 6.

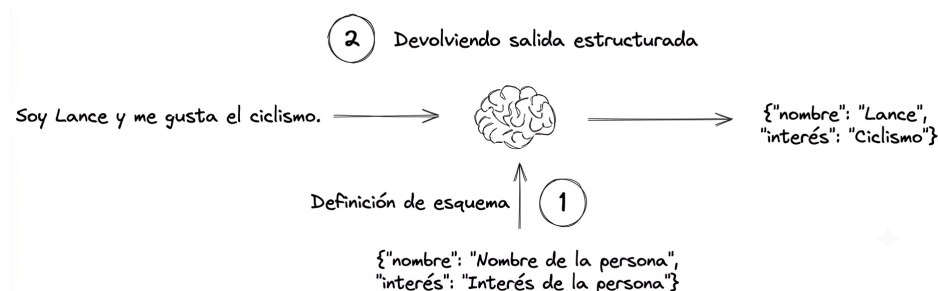


Figura 2.3. Funcionamiento de las salidas estructuradas [15].

2.4. Orquestación de Agentes Inteligentes

La complejidad inmanente al diseño y generación de una ontología completa hace ineficiente delegar toda la carga cognitiva a una única instrucción o modelo. La degradación del contexto en un LLM provoca una pérdida de información cuando se procesan dominios extensos [14]. Adicionalmente, el diseño propuesto genera un punto de fallo crítico en el sistema, eliminando el principio de responsabilidad única [16, Cap. 7].

Como solución, el modelo de negocio evoluciona hacia la orquestación de agentes. Esta estrategia permite dividir un problema complejo en tareas atómicas gestionadas por «un programa diseñado para percibir su entorno, razonar, tomar decisiones y ejecutar acciones de manera autónoma» [17]. En la Tabla 2.1 se expone una evaluación de las principales alternativas del presente estado del arte, en función de los requisitos para la validación semántica.

TABLA 2.1

COMPARATIVA DE HERRAMIENTAS DE ORQUESTACIÓN

Herramienta	Paradigma	Control flujo	Transparencia	Memoria	Multi-agente
LangChain	Secuencial	Medio	Media	Básica	Parcial
AutoGPT	Autónomo	Bajo	Baja	Limitada	Parcial
CrewAI	Colaborativo	Medio	Media	Interna	Nativo
LangGraph	Grafo	Muy alto	Alta	Persistente	Nativo

Ante este análisis, se escoge la herramienta de código abierto [1] de orquestación basada en grafos, *LangGraph* [18], como la solución óptima. A pesar de la base que presentan sus competidores en ciclos de razonamiento y acción [17], contienen limitaciones para sistemas productivos debido a la falta de control explícito del estado y del flujo. *LangGraph*

[18] representa una evolución donde agentes, herramientas y decisiones pueden modelarse como estados y transiciones verificables.

Sin embargo, el despliegue de modelos y orquestadores de IA introduce un desafío crítico, especialmente en la comunicación, pues si bien una petición tradicional se resuelve en milisegundos, la generación de texto de un LLM requiere operaciones síncronas intensivas de cómputo. La Sección 2.5 presenta una alternativa al patrón de diseño mencionado [8], ofreciendo retroalimentación al usuario del sistema de gestión del conocimiento.

2.5. Arquitectura Orientada a Eventos

La literatura en ingeniería de *software* establece que las arquitecturas síncronas tradicionales, fundamentadas en arquitecturas cliente-servidor, presentan problemas al hacer frente a procesos intensos de cómputo [19, Cap. 4]. Cuando se integra un LLM en el lado del servidor, el tiempo de inferencia del procesamiento semántico puede extenderse en gran medida, generando bloqueos en el hilo de ejecución del cliente y provocando un riesgo inminente de desconexión por tiempo de espera o *timeout*. Ante esta problemática, se propone la adopción de una arquitectura orientada a eventos, respetando los patrones de diseño [8], donde los componentes se desacoplan y reaccionan de manera asíncrona a la disponibilidad paulatina de la información.

En la Tabla 2.2 se exponen los diversos paradigmas de comunicación aplicables a la entrega de datos diferidos, comparando enfoques tradicionales con tecnologías nativas de eventos.

TABLA 2.2
COMPARATIVA DE PARADIGMAS DE COMUNICACIÓN CLIENTE-SERVIDOR

Paradigma	Comunicación	Estado	Latencia	Complejidad	Streaming
REST	Cliente → Servidor	Sin estado	Alta	Baja	Limitado
Polling	Cliente ↔ Servidor	Sin estado	Media	Media	Limitado
WebSockets	Cliente ↔ Servidor	Persistente	Baja	Alta	Continuo
SSE	Servidor → Cliente	Sin estado	Baja	Media	Incremental

Por un lado, exigir a la aplicación web que consulte repetidamente al servidor, *polling*, satura la red, mientras que esperar a la conclusión total del modelo, **Transferencia de Estado Representacional (REST)**, degrada drásticamente la interactividad. Por otro lado, aunque protocolos como *Web Sockets* habilitan la comunicación fluida en tiempo real, su naturaleza persistente y bidireccional introduce una sobrecarga innecesaria y un consumo de recursos excesivo para un flujo donde el cliente actúa como receptor una vez emitida la petición inicial.

En consecuencia, la tecnología **Eventos Enviados por el Servidor (SSE)** emerge como el estándar técnico idóneo para la transmisión asíncrona en plataformas semánticas [19, Cap. 11]. Operando de forma nativa sobre el **Protocolo de Transferencia de Hipertexto (HTTP)**, SSE permite al servidor unidireccionalmente empujar, *push*, los eventos y estados de *LangGraph* hacia el cliente a medida que se generan.

2.6. OntoNova

Resulta imperativo evaluar el panorama de herramientas destinadas a la construcción y gestión de ontologías, con el fin de justificar OntoNova, el sistema de gestión del conocimiento multilingüe. El ecosistema de extracción y modelado de conocimiento describe distintas alternativas que, si bien resuelven problemas específicos, manifiestan carencias significativas en entornos productivos modernos.

Los editores semánticos tradicionales, tales como Protégé o TopBraid Composer, anuncian el cumplimiento estricto de los estándares semánticos, anticipados en la [Sección 2.1](#). No obstante, su diseño asume que el operador es un ingeniero del conocimiento altamente cualificado, ralentizando la digitalización de los procesos empresariales [12]. En adición, el ámbito del aprendizaje ontológico solventa el modelado de conocimiento con herramientas clásicas como Text2Onto [20]. Sin embargo, la generación de estructuras taxonómicas superficiales utilizando análisis estadístico y NLP no alcanza el nivel de expresividad y consistencia requerido por el estándar OWL soberanamente, requiriendo una participación humana.

Seguido de la democratización de la IA, el uso de interfaces conversacionales genéricas impulsadas por un LLM, como ChatGPT, emerge como una alternativa accesible para la lluvia de ideas y la conceptualización de dominios. No obstante, estas herramientas ofrecen una caja negra de propósito general, anticipada en la [Sección 2.2](#). En consecuencia, al carecer de integraciones arquitectónicas que aseguren el determinismo sintáctico, [Sección 2.3](#), la construcción de una ontología sufre invariablemente de alucinaciones, omisiones estructurales o sintaxis inválida, requiriendo de una participación humana. En el ámbito de la ingeniería bioinformática nace OntoGPT [21], que analiza texto y extrae conceptos para poblar una ontología preexistente con la ayuda de la IA generativa. Entre sus características destacan su enfoque iterativo, orientación a línea de comandos y su arraigo al sector de la biociencia.

Como alternativa abierta a estos sistemas existentes se crea OntoNova, reuniendo métricas objetivo clave, recogidas en la comparativa de la [Tabla 2.3](#), para impulsar al sector del conocimiento de vuelta al protagonismo.

TABLA 2.3

COMPARATIVA DE HERRAMIENTAS PARA LA GESTIÓN Y MODELADO ONTOLÓGICO

Herramienta	Determinismo	Gestión errores	Multilingüe	Usuario final
Protégé	Estricto	Manual	No	Ingeniero
ChatGPT	Estocástico	Manual	Sí	Universal
OntoGPT	Estricto	Manual	No	Ingeniero
OntoNova	Estricto	Autónoma	Sí	Experto

CAPÍTULO 3

PLAN DE PROYECTO

Este capítulo presenta tanto la planificación, [Sección 3.1](#), como el presupuesto, [Sección 3.2](#), del trabajo realizado, conociendo el marco regulatorio y su impacto en el desarrollo sostenible, [Sección 3.3](#), en el entorno socioeconómico, [Sección 3.4](#), en el que surge.

3.1. Planificación

Esta sección incluye detalles acerca de la planificación del proyecto. Se describen, a continuación, las metodologías aplicadas, [Sección 3.1.1](#), detallando la duración de cada parte y normas asociadas, [Sección 3.1.2](#).

3.1.1. Metodología

El proceso de desarrollo de la propuesta se lleva a cabo siguiendo, principalmente, una metodología en cascada, representada en la [Figura 3.1](#). Este modelo permite una estructuración clara y lógica del trabajo, siguiendo los siguientes pasos:

1. **Análisis de requisitos.** Se realizan reuniones y consultas con el experto para identificar y definir los requisitos necesarios para el correcto desarrollo del sistema. Clave para establecer unos objetivos de negocio y de aprendizaje.
2. **Diseño de la arquitectura.** Reunidos los requisitos y los objetivos, se desencadena una lluvia de ideas y múltiples bocetos con el objetivo de cumplir los principios de desarrollo de *software* [\[8\]](#), [\[19\]](#), detallados en el [Capítulo 5](#).
3. **Proceso de aprendizaje.** Se investiga y selecciona la herramienta adecuada de orquestación de agentes, [Sección 2.4](#), en conjunto con la arquitectura idónea para el proyecto en la [Sección 2.5](#). Este proceso implica un tiempo considerable dedicado a familiarizarse con el *software* elegido y comprender su funcionamiento [\[6, Cap. 6\]](#), enlazando con el objetivo O1 descrito en la [Sección 1.2](#).

4. **Implementación y pruebas.** Se adopta un enfoque de prototipado, desarrollando los componentes de manera aislada y realizando iteraciones sobre ellos. Este planteamiento permite validar cada componente de forma independiente, así como realizar pruebas en un entorno controlado, tanto a nivel de componente como en el flujo completo de negocio. Este movimiento garantiza que se aborden los problemas de manera continua y se optimice el desarrollo según los resultados de las pruebas empíricas realizadas.

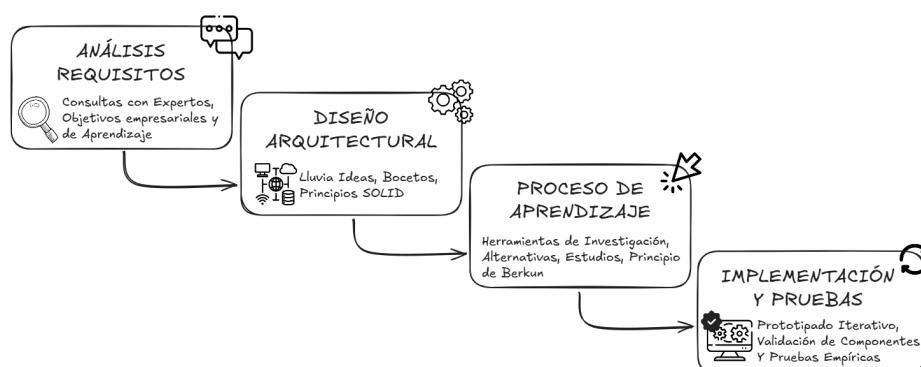


Figura 3.1. Metodología en cascada aplicada [6].

3.1.2. Estimación de Tiempo

La estimación de tiempo queda diseñada en la Figura 3.2, representando un diagrama de estimación de tiempo o Gantt [6, Cap. 9]. El esquema muestra las tareas realizadas en cada fase de la metodología en cascada (véase Figura 3.1), incluyendo una última etapa de informe, donde están incluidas las horas invertidas en la confección de este informe.

El proyecto fija una duración total de 11 meses, aproximadamente 32 horas por mes, cerca de 8 horas a la semana, sin contar fines de semana, vacaciones, días libres, baja por enfermedad y otras casuísticas. Por consiguiente, el tiempo total esperado del trabajo, excluyendo el trabajo del tutor, son 352 horas. En la Tabla 3.1 se recoge la distribución del esfuerzo de cada una de las etapas a lo largo del proyecto, en función de la metodología aplicada (véase Sección 3.1.1), y que sirve como base cuantitativa para complementar la Figura 3.2 presentada posteriormente.

TABLA 3.1

DESGLOSE ESTIMADO DE HORAS POR FASE DEL PROYECTO

Fase	Horas estimadas	% del total
Análisis de Requisitos	66	18,75
Diseño de la Arquitectura	35	9,94
Proceso de Aprendizaje	67	19,04
Implementación y Pruebas	96	27,27
Informe	88	25,00
Total	352	100,00

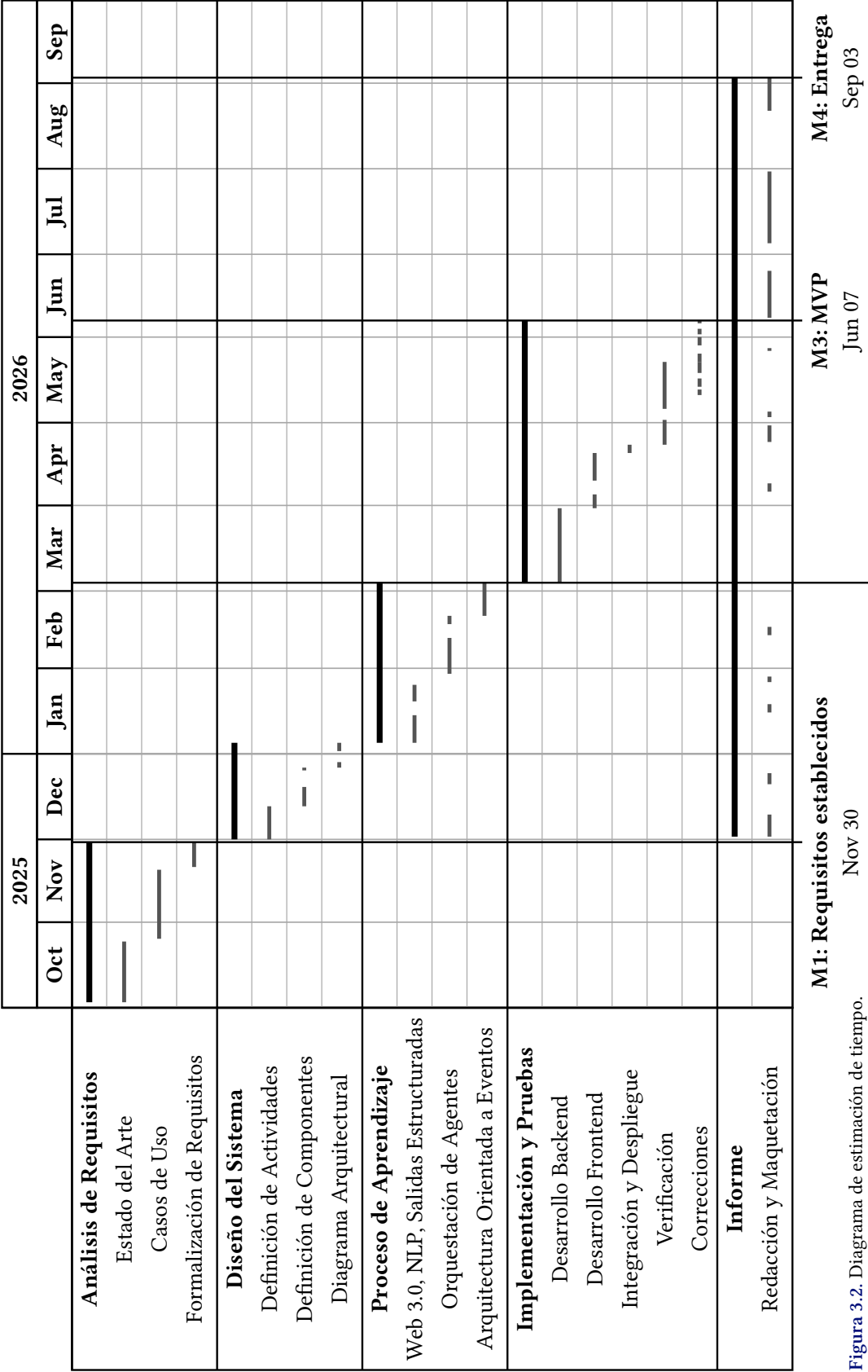


Figura 3.2. Diagrama de estimación de tiempo.

3.2. Presupuesto

Se introduce el presupuesto basado en la estimación de tiempo y metodología descritos en la [Sección 3.1](#). La siguiente [Tabla 3.2](#) resume la información del proyecto, incluyendo su presupuesto total.

TABLA 3.2
INFORMACIÓN DEL PROYECTO

Campo	Descripción
Título	Diseño e Implementación de un Sistema de Gestión del Conocimiento Multilingüe Basado en LLMs como Alternativa Abierta a otros sistemas existentes
Autor	Diego Picazo García
Departamento	Informática
Inicio	03/10/2025
Fin	03/09/2026
Duración	11 meses
Presupuesto	13.000,00€

El balance disponible para el proyecto se ha repartido en las distintas áreas siguiendo **la regla del 70/25**, destinando un 70% de los gastos a recursos humanos y un 25% a elementos materiales. Se ha reservado un 5% del total destinado a gastos imprevistos, concretamente 650,00€ [[6](#), [Cap. 12](#)].

3.2.1. Costes Directos

Aquellos directamente relacionados con el desarrollo del proyecto [[6](#), [Cap. 12](#)]. Se distinguen en dos grupos:

- **Costes personales.** Gastos relacionados con la experiencia, desarrollo personal y transporte de los trabajadores, entre otros. El coste es variable en función del rol del trabajador y su impacto en el proyecto, visible en la [Tabla 3.3](#):
 - **Gerente de Proyecto.** Ofrece apoyo y gestión al proyecto, intermediando entre el nivel de dirección y el nivel de entrega dentro de la organización del proyecto.
 - **Aseguramiento de Calidad (QA).** Diseña y realiza pruebas de las características implementadas.
 - **Desarrollador.** Implementa las funcionalidades de negocio.
 - **Analista.** Evalúa los requisitos y diseña la arquitectura.

Es importante señalar que el rol de gerente de proyecto fue asumido por la figura de la tutora, mientras que los demás roles fueron desempeñados por el estudiante.

TABLA 3.3
COSTES PERSONALES

Rol	Horas	Tarifa	Total
Gerente de Proyecto	80	41,20€/h	3.296,00€
QA	72	25,21€/h	1.815,12€
Desarrollador	150	23,57€/h	3.535,50€
Analista	50	35,20€/h	1.760,00€
Total	352		10.406,62€

- **Costes materiales.** Principalmente *hardware*, como servidores, portátiles, teclados, ratones... De cara al *software*, todo lo utilizado es [Software Libre y de Código Abierto \(FOSS\)](#). El balance queda recogido en la [Tabla 3.4](#), teniendo en cuenta el coste unitario y aplicando un porcentaje de depreciación debido al uso intensivo durante la duración de once meses del proyecto.

TABLA 3.4
COSTES MATERIALES

Elemento	Uso	Coste de compra	Depreciación	Coste final
Servidor	Almacenamiento y procesamiento	900,00€	20%	720,00€
Teclado	Herramienta de desarrollo	50,00€	10%	45,00€
Ratón ergonómico	Herramienta de desarrollo	35,00€	10%	31,50€
Pantalla	Monitor para trabajar	200,00€	15%	170,00€
Software	Desarrollo de Software	0,00€	0%	0,00€
NAS	Copia de seguridad	194,97€	20%	155,97€
UPS	Sistema de alimentación ininterrumpida	150,00€	10%	135,00€
Silla ergonómica	Mejora el confort durante el trabajo	180,00€	15%	153,00€
Cables y accesorios	Soporte técnico	40,00€	10%	36,00€
Auriculares	Comunicación y reuniones virtuales	45,00€	10%	40,50€
Total		2.094,97€		1.486,97€

3.2.2. Costes Indirectos

Aquellos que no se pueden categorizar o incluir bajo ningún grupo elegible por la dirección del proyecto [6, Cap. 12].

Para el coste energético, se ha asumido que todos los equipos informáticos son idénticos y no presentaron fallos durante su utilización durante 11 meses. De media cada hora, se ha hecho uso de 500W, por lo tanto, $500W \times 352h = 176kWh$, lo que supone un gasto total de 44,00€¹. El plan de internet se reduce a 26,00€ al mes, gracias a una oferta por tiempo limitado de una gran empresa de telefonía. En cuanto al transporte, el estudiante hace uso del bono transporte ofrecido por la Comunidad de Madrid, por lo que 10,00€ al mes eran más que suficientes para sufragar los gastos de movilidad.

A continuación, se muestra el balance, en la [Tabla 3.5](#), de internet, luz y transporte, cubierto por el 5% de reserva mencionado en la [Sección 3.2](#).

TABLA 3.5
COSTES INDIRECTOS

Recurso	Coste unitario	Cantidad	Total
Electricidad	0,25€/kWh ¹	176kWh	44,00€
Internet	26,00€/mes	11 meses	286,00€
Transporte	10,00€/mes	11 meses	110,00€
Total			440,00€

3.2.3. Balance Total

La [Tabla 3.6](#) incluye un sumario exhaustivo de todos los costes asociados a la ejecución del presente proyecto. Resulta pertinente destacar que el coste definitivo se ha mantenido por debajo de la asignación inicial prevista, lo que arroja como resultado un superávit favorable. Este margen positivo no solo refleja la viabilidad de la planificación, sino que también evidencia un control financiero riguroso y una gestión eficiente de los recursos económicos dispuestos para este fin.

TABLA 3.6
BALANCE TOTAL

Descripción	Total
Costes directos	12.501,59€
Costes indirectos	440,00€
Costes totales	12.941,59€
Presupuesto inicial	13.000,00€
Superávit	+ 268,41€²

¹Se ha asumido un coste de luz constante de 0,25€/kWh.

²Incluye el remanente de la reserva de gastos imprevistos.

3.2.4. Propuesta de Venta

La [Tabla 3.7](#) ofrece una propuesta de venta del proyecto a una empresa de terceros, por un total de 25.624,33€. Se han contemplado impuestos, beneficios y riesgos esperados, el coste total del trabajo (véase [Tabla 3.6](#)) y regalías. De manera adicional, se otorga la posibilidad de contratar personal mantenedor de la aplicación.

TABLA 3.7
PROPUESTA DE VENTA

Concepto	Afectación	Coste parcial	Coste agregado
Coste del proyecto		12.941,59€	12.941,59€
Riesgo	19%	2.458,90€	15.400,49€
Beneficio esperado	8%	1.035,32€	16.435,81€
Impuestos	21%	2.717,73€	19.153,54€
Regalías	5%/año ³	647,07€	19.153,54€ + 647,07€ × λ ⁴
Total a 10 años			25.624,33€

3.3. Marco Regulatorio

A continuación, se discute la legislación que afecta al desarrollo, implementación y día a día del sistema, en la [Sección 3.3.1](#), incluyendo los estándares, la [Sección 3.3.2](#), adjuntando las licencias FOSS implicadas, [Sección 3.3.3](#), y objetivos de desarrollo sostenible a los que contribuye, disponibles en la [Sección 3.3.4](#).

3.3.1. Legislación Aplicable

Durante el [Capítulo 6](#) de implementación, el *software* y su desarrollo se lanzan en múltiples ocasiones en un entorno FOSS controlado [1]. Posteriormente, en el transcurso del estudio de alternativas de orquestadores de agentes, [Sección 2.4](#), y la elección de arquitectura en la [Sección 2.5](#), tampoco ha recibido ni el sistema ni el modelo una comunicación con el exterior.

Cabe destacar que, si la propuesta de venta sale adelante, [Sección 3.2.4](#), y la aplicación acaba siendo expuesta al público, será necesario incluir una declaración de política de privacidad, así como solicitar permiso al usuario para el tratamiento de sus datos en el proceso de registro y del tratamiento de sus *cookies* durante su navegación, en concordancia con el objetivo O4. Regulación presente en el [Reglamento \(UE\) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016 \(RGPD\)](#) [3] y [Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales \(LOPDGDD\)](#) [22]. Asimismo, el programa se alinea con el [Reglamento \(UE\) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024 \(AI Act\)](#) [4] garantizando la transparencia, la mitigación de sesgos, la supervisión humana y el despliegue de modelos de lenguaje bajo un enfoque basado en el riesgo.

³Las regalías se dividen en 3% tutor y 2% estudiante.
⁴λ = número de años.

3.3.2. Estándares y Marcos Técnicos

El *software* desarrollado persigue los siguientes estándares y marcos regulatorios:

- **Definición de Código Abierto, 2007.** Marco técnico y filosófico establecido por la Iniciativa para el Código Abierto, que determina los criterios para que un *software* sea considerado de código abierto. El proyecto se rige por estos principios para garantizar la libre distribución, el acceso al código fuente y la interoperabilidad, apoyándose en licencias permisivas estandarizadas, presentes en la [Sección 3.3.3](#), que fomentan la soberanía tecnológica frente a soluciones propietarias [23].
- **ISO/IEC 21778, 2017.** Formato **JSON**, utilizado en la transmisión de información [24].
- **ISO/IEC 21838, 2021.** Establece los requisitos para las ontologías de nivel superior [25, [Parte 1](#)] y define la **Ontología Formal Básica (BFO)** con dichos criterios [25, [Parte 2](#)].
- **ISO/IEC 27001, 2022.** Establece directrices sobre cómo implementar y gestionar controles de seguridad para proteger los datos relativos a la [Sección 3.3.1](#) [26].
- **ISO/IEC 42001, 2023.** Norma internacional pionera que especifica los requisitos para establecer, implementar, mantener y mejorar continuamente un sistema de gestión de inteligencia artificial [27].
- **ISO/IEC 25002, 2024.** Define las características que debe cumplir un sistema para garantizar la calidad. Evalúa factores como usabilidad, eficiencia, seguridad, mantenibilidad y portabilidad, contemplados durante todo el desarrollo del *software* [28].
- **OWASP Top 10 GenIA/LLM, 2025.** Extiende las directrices de seguridad tradicionales para abordar de manera específica las vulnerabilidades emergentes del uso de **IA** generativa. Se contempla en el [Capítulo 5](#) de diseño para mitigar riesgos críticos como inyecciones de *prompts* [29, [LLM01](#)], divulgación de información sensible [29, [LLM02](#)] y la desinformación o alucinaciones del modelo [29, [LLM09](#)]. Especialmente relevante es la mitigación del manejo inadecuado de salidas [29, [LLM05](#)], abordado mediante el uso de salidas estructuradas, disponible su definición en la [Sección 2.3](#), que validan y sanean la información antes de su integración en el sistema.

3.3.3. Licencias

El desarrollo y distribución del presente sistema se fundamenta en los estándares mencionados en la [Sección 3.3.2](#), garantizando la transferencia de conocimiento, la transparencia y la soberanía tecnológica del usuario. Para la liberación del código fuente, se adopta la **Licencia Pública de la Unión Europea (EUPL-1.2)** [30], que permite el uso, copia, modificación o distribución del código fuente siempre que se mantenga abierto y adjunto a un aviso de autoría. El código completo se encuentra disponible en un repositorio de GitHub [31]. Adicionalmente, se adopta un marco de trabajo **FOSS** en lo que a librerías se refiere, visible en la [Tabla 3.8](#), decisión que garantiza que la plataforma no dependa de intermediarios propietarios, alineada con el objetivo principal mencionado en la [Sección 1.2](#).

En consonancia, se emplea la licencia CC BY 4.0⁵ para la documentación, que permite compartir, copiar y redistribuir el material en cualquier medio o formato, así como adaptarlo para cualquier propósito, incluso comercial, siempre que se otorgue el crédito correspondiente al autor original.

⁵Creative Commons Attribution 4.0 International.

TABLA 3.8
LICENCIAS DE COMPLEMENTOS

Licencia	Complemento
MIT	FastAPI • LangGraph • Pydantic • React • Vite • Zustand
BSD 3-Clause	Uvicorn
Apache License 2.0	vLLM

3.3.4. Objetivos de Desarrollo Sostenible

Se concibe un firme compromiso con la Agenda 2030 de las Naciones Unidas [9]. Para dar cumplimiento al objetivo O6, descrito en la Sección 1.2, se ha establecido un Modelo de Trabajo (WoW) que trasciende la mera implementación técnica, orientando la arquitectura y el propósito de la herramienta hacia la generación de un impacto social y tecnológico medible. La **educación de calidad** [9, ODS 4] abarca un aprendizaje continuo y la adquisición de competencias técnicas avanzadas por parte de los estudiantes y profesionales del sector. Tradicionalmente, la ingeniería del conocimiento queda restringida a perfiles con una alta especialización técnica, debido a la complejidad sintáctica de herramientas tradicionales como *Protégé*. OntoNova democratiza este acceso al abstraer dicha complejidad con el uso de interfaces visuales y NLP. De este modo, se permite que expertos de diversos sectores —médicos, juristas, humanistas— estructuren y asimilen dominios de conocimiento complejos, impulsando la alfabetización digital y facilitando la transferencia de conocimiento interdisciplinar sin requerir una curva de aprendizaje elevada.

Persiguiendo la necesidad actual de construir **infraestructuras** tecnológicas resilientes, promover la **industrialización** inclusiva y fomentar la **innovación** [9, ODS 9], se apuesta por un ecosistema fundamentado íntegramente en *software* de código abierto, FOSS. Queda optimizada la ejecución de LLM en *hardware* local, exponiendo una innovación disruptiva en el procesamiento de datos no estructurados. Esta aproximación reduce drásticamente la dependencia de infraestructuras propietarias en la nube y de servicios prestados por terceros, dotando a las organizaciones e instituciones de una infraestructura de datos soberana, independiente y altamente interoperable bajo los estándares de la Web Semántica del W3C, disponibles en la Sección 2.1.

La brecha digital contemporánea no solo se manifiesta en la falta de acceso a la red, sino en la incapacidad de participar activamente en la creación de las tecnologías subyacentes. El propósito central es romper las barreras técnicas e idiomáticas que segregan a los beneficiarios de la Web 3.0, alineado con el objetivo O2, generando una **reducción de las desigualdades** [9, ODS 10]. Al ofrecer un entorno accesible y multilingüe, marcado en el objetivo O3, se garantiza que las limitaciones de idioma o la falta de destrezas en programación no impidan a los expertos del dominio diseñar y controlar los modelos de conocimiento, contribuyendo activamente a una distribución más equitativa del control sobre la información en el ecosistema digital.

3.4. Entorno Socioeconómico

El **proyecto OntoNova** surge en un contexto socioeconómico caracterizado por la adopción de la **IA** generativa como herramienta productiva, tanto en España como en el resto de países miembros de la Unión Europea. Tras una fase inicial de revelación masiva, impulsada por la sobreabundancia de información no estructurada y la experimentación a través de **LLM** de uso comercial, en el estudiantado y el ámbito empresarial se genera una nueva necesidad. Estructurar, verificar y dotar de sentido lógico a grandes volúmenes de texto, combatiendo la desinformación y el ruido digital bajo el estricto marco regulatorio europeo [2] (p. ej., **RGPD** [3] y **AI Act** [4]).

Este nuevo paradigma trae consigo una gran sobrecarga cognitiva y una crisis de confianza en la información. Profesionales de todos los sectores se enfrentan diariamente a la tarea de extraer conocimiento útil de repositorios documentales masivos. La proliferación de modelos de lenguaje estocásticos, propensos a las alucinaciones [14] y a la generación de contenido no verificable, ha evidenciado que la simple generación de texto no es suficiente. La sociedad actual requiere sistemas que garanticen la veracidad, la trazabilidad y la estructuración del conocimiento, elementos indispensables para la toma de decisiones críticas en ámbitos como la salud, la justicia o la investigación científica.

Desde una perspectiva económica, la estructura del mercado español, compuesto mayoritariamente por **Pequeñas y Medianas Empresas (PYMES)**, instituciones académicas y entidades públicas, se enfrenta a una fuerte dependencia frente a grandes proveedores tecnológicos multinacionales. El uso continuado de servicios de **IA** basados en la nube a través de recursos de pago por uso supone costes recurrentes y escalables que resultan inasumibles para entidades de menor envergadura. Cabe destacar que externalizar la extracción y estructuración del conocimiento corporativo interno hacia servidores de terceros suscita riesgos significativos de fuga de capital intelectual corporativo.

La creciente preocupación por la privacidad y la entrada en vigor del **Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024** impactan en cómo las organizaciones gestionan sus datos. Con el fin de los periodos de adaptación regulatoria, existe una presión institucional por adoptar tecnologías que aseguren la soberanía del dato y eviten la fuga de capital intelectual hacia servidores externos. A esto se suma el intenso debate en torno a la sostenibilidad de los grandes centros de datos, que impulsa la búsqueda de alternativas de menor impacto ambiental. En este escenario, la comunidad **FOSS** [23] proporciona soluciones, permitiendo que la ejecución local de modelos avanzados y eficientes no sea una utopía técnica.

La consolidación de un conocimiento interno de calidad, en combinación con la optimización de recursos y la evolución de entornos ágiles, se convierte en un deber con la evolución drástica de componentes inteligentes. En este sentido, una aplicación web de código abierto que permita procesar, extraer y estructurar el conocimiento de forma asíncrona mediante **IA** local, exportable a formatos estándar y consumible por ingenieros de conocimiento, representa una solución socioeconómica clave para que profesionales de cualquier sector puedan integrarse de forma segura en el nuevo paradigma de la **IA**.

CAPÍTULO 4

ANÁLISIS

Esta sección incluye detalles acerca del análisis de requisitos. En páginas posteriores, se describe la gestión de requisitos con su origen y estándar a utilizar, [Sección 4.1](#), así como la exposición de los requisitos de usuario, del sistema [\[32\]](#) y los casos de uso derivados de los mismos, en la [Sección 4.2](#).

4.1. Gestión de Requisitos

En este apartado se menciona y destaca el origen de los requisitos, [Sección 4.1.1](#), y qué estándares reconocidos se han utilizado de cara a la declaración de requisitos, en la [Sección 4.1.2](#).

4.1.1. Origen de los Requisitos

El proyecto está determinado mayoritariamente por requisitos y restricciones en el *software* impuestos por la gerente de proyecto, actuando como tutora⁶, con el objetivo de aportar innovación, investigación y modernidad a la implementación del sistema de gestión del conocimiento, en concordancia con las metas descritas en la [Sección 1.2](#) y objetivos de desarrollo sostenible, delimitados en la [Sección 3.3.4](#).

Por otro lado, se consideran requisitos de usuario —médicos, juristas, humanistas— y del sistema derivados de la investigación y revisión documental llevada a cabo en el [Capítulo 2](#), sin olvidar las necesidades de los afectados mostradas en la [Sección 3.4](#). En este sentido, se ha tenido en cuenta la experiencia de los usuarios finales y limitaciones arquitecturales, con el fin de garantizar que el producto final cumpla con las expectativas y requerimientos del proyecto, que influyen en el diseño e implementación de la aplicación [\[33\]](#).

⁶La figura de la tutora asume funcionalmente el rol de *stakeholder* y *product owner*.

4.1.2. Estándar IEEE para la Declaración de Requisitos

La gestión y calidad de requisitos toma como referencia las características definidas por el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE), n.º 830-1998 [34], recomendado para el manejo, especificación de necesidades de usuario y buenas prácticas. Estas son:

- **Correcto.** Las especificaciones de *software* deben ir acorde a las necesidades del usuario [34, Cap. 4.3.1].
- **Sin ambigüedad.** Un requisito no debe dar lugar a más de una interpretación [34, Cap. 4.3.2].
- **Completo.** El documento contenedor debe tener todos los requisitos de *software* significativos [34, Cap. 4.3.3].
- **Consistente.** Los requisitos no deben generar conflictos entre sí [34, Cap. 4.3.4].
- **Orden de prioridad.** Es necesario establecer una prioridad y necesidad para cada uno de los requisitos [34, Cap. 4.3.5].
- **Verificable.** Debe existir un proceso que permita comprobar el éxito de la especificación [34, Cap. 4.3.6].
- **Modificable.** La estructura y estilo del documento de especificación deben permitir que los cambios se realicen de manera fácil, completa y consistente [34, Cap. 4.3.7].
- **Trazable.** Cada requisito funcional debe tener un origen bien definido [34, Cap. 4.3.8].

Posteriormente, la norma ISO/IEC/IEEE 29148:2018 [35, Cap. 5.2] sustituye a IEEE n.º 830-1998 [34], que, en combinación con ISO/IEC/IEEE 15288:2023 [36], mantiene los principios fundamentales destacados, incorporando criterios de aceptación cuantificables conforme a las características y reglas de la guía de la Organización Internacional de Ingeniería en Sistemas (INCOSE) [37].

4.2. Exposición de Requisitos

La Tabla 4.1 enmarca el contenido de un requisito de usuario o de *software* del desarrollo de la aplicación. La nomenclatura utilizada es REQ-XX-YY-ZZ, siendo XX el ámbito del requisito, YY el origen al que pertenece, US para usuario y SW para *software*, y ZZ el tipo de requisito, con los valores FC o NF indicando, funcional o no funcional, respectivamente.

TABLA 4.1
CONTENIDO DE UN REQUISITO

Campo	Descripción
ID	REQ-XX-YY-ZZ.
Descripción	Descripción detallada del requisito con una métrica cuantificable .
Carácter	Necesidad del usuario o del diseño (Obligatorio, Conveniente u Opcional).
Prioridad	Importancia del requisito (Alta, Media o Baja).
Estado	Situación durante el desarrollo (Incluido, Viable o Requiere nuevo diseño). ⁷
Verificabilidad	Indica cuánta es la posibilidad de probar dicho requisito (Alta, Media o Baja).
Origen	Indica el origen del requisito (Usuario, Gerente de Proyecto o REQ-XX-YY-ZZ).

⁷La clave entre viabilidad y complejidad reside en el grado de refactorización de la aplicación.

4.2.1. Requisitos de Usuario

La Tabla 4.2 representa los requisitos de usuario funcionales. Los requisitos no funcionales están organizados en la Tabla 4.3.

TABLA 4.2
REQUISITOS FUNCIONALES DE USUARIO

ID	Atributos	Descripción
REQ-US-FC-01	Carácter: Obligatorio. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Usuario.	El sistema permite al usuario introducir texto de hasta 15.000 caracteres, en al menos español e inglés, a través de la interfaz web para iniciar el proceso de elaboración de una ontología.
REQ-US-FC-02	Carácter: Obligatorio. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Gerente de Proyecto.	La interfaz web muestra el progreso del proceso de elaboración de una ontología empleando eventos, con una latencia máxima de 1 segundo.
REQ-US-FC-03	Carácter: Obligatorio. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Gerente de Proyecto.	El <i>backend</i> debe construir una ontología que supere la validación sintáctica, disponiendo de un motor de degradación elegante que active un ciclo de corrección ante cada fallo, sin interrumpir el proceso.
REQ-US-FC-04	Carácter: Obligatorio. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Usuario.	La interfaz web proporciona un lienzo interactivo que permite al usuario visualizar, modificar, añadir o eliminar las clases o nodos y aristas o relaciones del grafo, ejecutando cada operación en un máximo de 3 interacciones.
REQ-US-FC-05	Carácter: Obligatorio. Prioridad: Media. Estado: Incluido. Verificabilidad: Alta. Origen: REQ-US-FC-04.	El usuario puede exportar la ontología final validada desde la interfaz web a un formato estándar W3C, generando un archivo que supera un validador sintáctico sin errores.
REQ-US-FC-06	Carácter: Opcional. Prioridad: Media. Estado: Requiere nuevo diseño. Verificabilidad: Media. Origen: REQ-US-FC-04.	El sistema permite el modelado y edición simultánea del grafo de conocimiento por al menos 2 usuarios concurrentes, con convergencia del estado compartido en menos de 2 segundos.

ID	Atributos	Descripción
REQ-US-FC-07	Carácter: Conveniente. Prioridad: Media. Estado: Viable. Verificabilidad: Alta. Origen: REQ-US-FC-04.	El sistema incluye un histórico de versiones que registra el historial completo de modificaciones del grafo, permitiendo al usuario revertir y auditar cambios.
REQ-US-FC-08	Carácter: Conveniente. Prioridad: Baja. Estado: Requiere nuevo diseño. Verificabilidad: Alta. Origen: REQ-US-FC-05.	El sistema permite la importación de ontologías existentes en formatos estándar W3C de hasta 5MB a través de un recorrido web, sin pérdida de tripletas.
REQ-US-FC-09	Carácter: Conveniente. Prioridad: Baja. Estado: Viable. Verificabilidad: Alta. Origen: REQ-US-FC-07.	Los usuarios deben poder registrarse en la plataforma proporcionando un correo con formato válido y una contraseña de al menos 8 caracteres.
REQ-US-FC-10	Carácter: Conveniente. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Gerente de Proyecto.	El sistema permite utilizar como entrada, desde la interfaz web, archivos planos o PDF de hasta 5MB, en español e inglés, rechazando los que incumplan el requisito REQ-US-FC-01.

TABLA 4.3

REQUISITOS NO FUNCIONALES DE USUARIO

ID	Atributos	Descripción
REQ-US-NF-01	Carácter: Conveniente. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Usuario.	La interfaz de usuario debe ser intuitiva y fácil de usar, alcanzando una puntuación mínima de 70 en el cuestionario de Escala de Usabilidad del Sistema (SUS) [38] con 5 usuarios de prueba.
REQ-US-NF-02	Carácter: Conveniente. Prioridad: Media. Estado: Incluido. Verificabilidad: Alta. Origen: Usuario.	La aplicación debe cargar en un tiempo máximo de 3 segundos y responder a las interacciones del usuario en menos de 200 milisegundos.
REQ-US-NF-03	Carácter: Conveniente. Prioridad: Alta. Estado: Viable. Verificabilidad: Alta. Origen: Usuario.	Debe existir un sistema de soporte para ayudar a los usuarios con problemas técnicos en un máximo de 24 horas, junto con un plan documentado de mantenimiento mensual.

4.2.2. Requisitos del Sistema

Contemplando el contenido de requisitos de la [Tabla 4.1](#), a continuación se describen los requisitos de *software* derivados de los requisitos establecidos por el usuario [32, Cap. 5], ya sea un médico, jurista o humanista. La [Tabla 4.4](#) recoge los requisitos funcionales y la [Tabla 4.5](#) los no funcionales.

TABLA 4.4
REQUISITOS FUNCIONALES DEL SISTEMA

ID	Atributos	Descripción
REQ-SW-FC-01	Carácter: Conveniente. Prioridad: Media. Estado: Viable. Verificabilidad: Media. Origen: REQ-US-NF-01 y REQ-US-NF-03 .	Incluir trazabilidad y métricas de usuario, de forma que cada acción principal genere un evento registrado que contribuya a la mejora continua.
REQ-SW-FC-02	Carácter: Obligatorio. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Gerente de Proyecto.	El sistema debe comunicarse con el motor de inferencia local utilizando un protocolo estándar, de forma que la sustitución del modelo subyacente requiera solo cambios de configuración.

TABLA 4.5
REQUISITOS NO FUNCIONALES DEL SISTEMA

ID	Atributos	Descripción
REQ-SW-NF-01	Carácter: Obligatorio. Prioridad: Baja. Estado: Viable. Verificabilidad: Alta. Origen: REQ-US-NF-01 y REQ-US-NF-03 .	Asegurar que la aplicación sea accesible para personas con discapacidad, cumpliendo el nivel AA de las Directrices de Accesibilidad para el Contenido Web (WCAG) [39] sin errores críticos.
REQ-SW-NF-02	Carácter: Obligatorio. Prioridad: Alta. Estado: Incluido. Verificabilidad: Media. Origen: Gerente de Proyecto.	El <i>software</i> debe desarrollarse sin vulnerabilidades de severidad alta en las dependencias, comprobadas con herramientas de auditoría, en concordancia con el objetivo O5 de la Sección 1.2 .
REQ-SW-NF-03	Carácter: Conveniente. Prioridad: Media. Estado: Incluido. Verificabilidad: Alta. Origen: REQ-US-FC-01 y REQ-US-FC-03 .	La aplicación debe ser resiliente, devolviendo ante cada error capturado una respuesta controlada al usuario, sin pérdida del estado del lienzo.

ID	Atributos	Descripción
REQ-SW-NF-04	Carácter: Conveniente. Prioridad: Media. Estado: Viable. Verificabilidad: Alta. Origen: Gerente de Proyecto.	Incluir un marco de pruebas automatizadas con una cobertura mínima del 90% sobre la lógica de negocio del sistema.
REQ-SW-NF-05	Carácter: Conveniente. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Gerente de Proyecto.	La aplicación debe estar acompañada de una documentación técnica —instalación, uso y mantenimiento— que permita a un tercero completar un despliegue local limpio por sí mismo.
REQ-SW-NF-06	Carácter: Conveniente. Prioridad: Media. Estado: Incluido. Verificabilidad: Media. Origen: Gerente de Proyecto.	La arquitectura del sistema debe permitir la adición de nuevas funcionalidades, manteniendo el principio de abierto/cerrado [16, Cap. 8] y la gestión de un aumento en el número de usuarios sin necesidad de rediseño completo [16, Cap. 16].
REQ-SW-NF-07	Carácter: Obligatorio. Prioridad: Alta. Estado: Viable. Verificabilidad: Alta. Origen: REQ-US-FC-09.	Implementar controles de acceso con autenticación y autorización, almacenando las credenciales de los usuarios con <i>hash</i> y <i>salt</i> para prevenir accesos no autorizados.
REQ-SW-NF-08	Carácter: Conveniente. Prioridad: Alta. Estado: Incluido. Verificabilidad: Alta. Origen: Gerente de Proyecto.	El sistema debe estar contenerizado para asegurar un despliegue ágil en menos de 5 minutos, predecible y agnóstico respecto a la infraestructura de <i>hardware</i> local.

4.2.3. Casos de Uso

En esta sección, los principales casos de uso están ligados a los requisitos incluidos de los usuarios. Se ofrece una descripción del caso, actores principales, flujos principales y alternativos, según aplique, además de ligarlos a su requisito de usuario correspondiente [40, Cap. 6]. La información quedará expuesta en un diagrama de **Lenguaje Unificado de Modelado (UML)**.

Creación de una ontología

- **Descripción.** El usuario introduce un texto explicativo en un lenguaje de preferencia a través de la interfaz web para que el sistema procese, valide y genere de forma asíncrona el primer borrador de la ontología.
- **Actores involucrados.** Usuario y sistema.

- **Flujo principal.** El usuario accede al formulario, introduce el texto descriptivo del dominio y solicita la creación de la ontología. El sistema provee de la semántica, en una conexión asíncrona, validada y lista para ser renderizada sobre el lienzo interactivo.
- **Flujo secundario.** En caso de error o de generación de una estructura inválida, el sistema activa el motor de degradación elegante y ejecuta un ciclo de corrección alertando al usuario.
- **Requisitos de usuario asociados.** [REQ-US-FC-01](#), [REQ-US-FC-02](#), [REQ-US-FC-03](#), [REQ-US-FC-10](#).

Edición de una ontología

- **Descripción.** El usuario manipula de forma directa los elementos visuales del grafo semántico generado, permitiéndole refinar el conocimiento añadiendo, modificando o eliminando clases y relaciones.
- **Actores involucrados.** Usuario y sistema.
- **Flujo principal.** El usuario interactúa con el lienzo visual, selecciona un componente, ejecuta una acción que es actualizada y validada de forma reactiva y en caliente por el sistema.
- **Flujo secundario.** En caso de error o de generación de una estructura inválida, el sistema activa el motor de degradación elegante y ejecuta un ciclo de corrección alertando al usuario.
- **Requisitos de usuario asociados.** [REQ-US-FC-04](#).

Eliminación de una ontología

- **Descripción.** El usuario purga el lienzo de trabajo actual para descartar el modelo en pantalla y liberar los recursos de la interfaz de cara a un nuevo análisis.
- **Actores involucrados.** Usuario y sistema.
- **Flujo principal.** El usuario, situado en la pantalla del lienzo interactivo, solicita la eliminación de la ontología actual mediante el icono de borrado o reinicio. El sistema solicita confirmación debido a la criticidad de la acción y, en caso afirmativo, limpia el lienzo devolviendo el visual a su estado inicial.
- **Flujo secundario.** Si el usuario cancela la operación en el paso de confirmación, el sistema cierra la ventana de confirmación y el lienzo se mantiene intacto.
- **Requisitos de usuario asociados.** [REQ-US-FC-04](#).

Exportación de una ontología

- **Descripción.** El usuario descarga la ontología validada sobre el lienzo hacia un archivo local en un formato estándar [W3C](#) compatible con otras soluciones semánticas del sector.
- **Actores involucrados.** Usuario y sistema.
- **Flujo principal.** El usuario solicita la exportación a través del icono de descarga seleccionando el formato de destino de preferencia. El sistema compila el estado actual y lo empaqueta en el formato seleccionado, generando un flujo de descarga hacia el almacenamiento local del usuario.
- **Flujo secundario.** No aplica al tratarse de una operación de descarga local.
- **Requisitos de usuario asociados.** [REQ-US-FC-05](#).

Diagrama de Casos de Uso

Se ilustra la Figura 4.1, la cual contiene el diagrama UML de los casos de uso mencionados en la Sección 4.2.3.

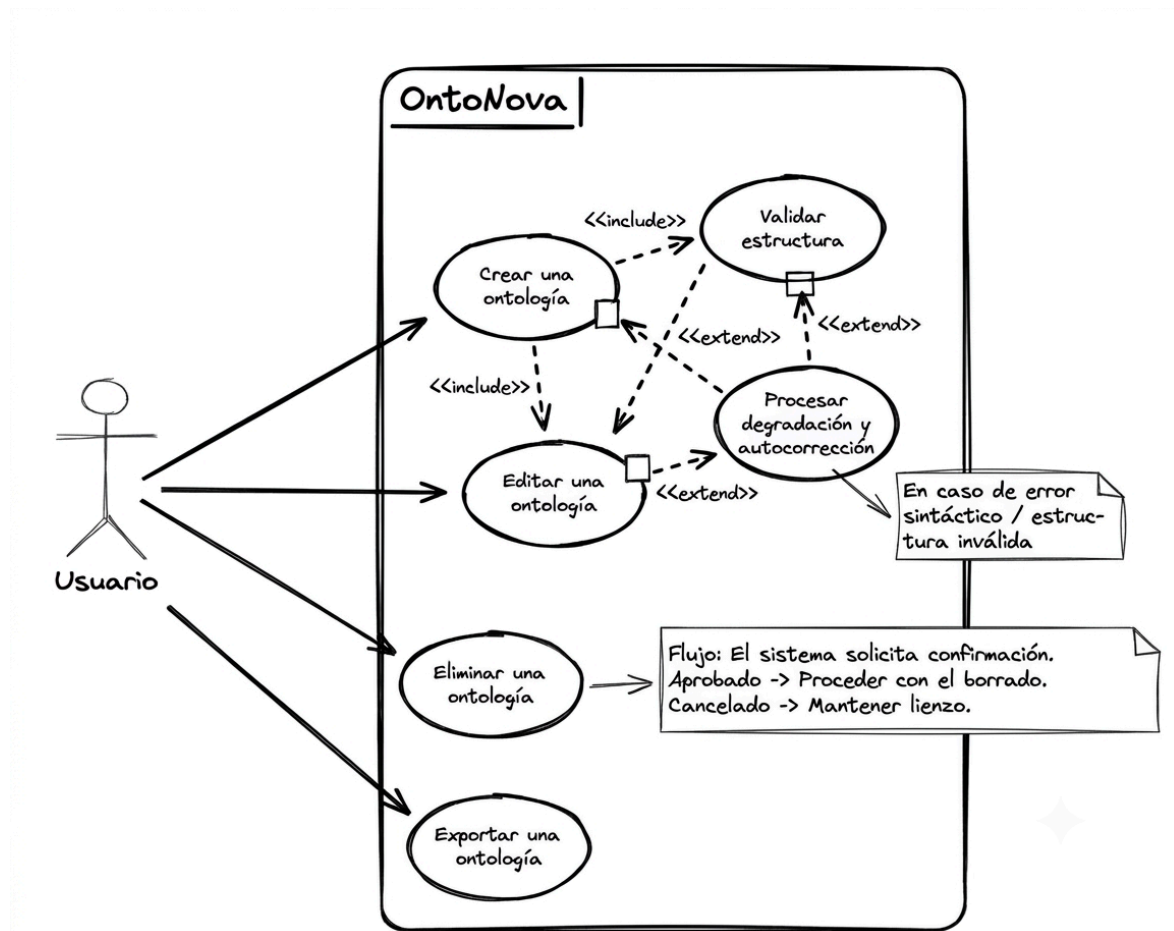


Figura 4.1. Diagrama de casos de uso en lenguaje UML.

CAPÍTULO 5

DISEÑO

Este capítulo expone el diseño del sistema como respuesta a los requisitos recogidos en la [Sección 4.2](#). En primer lugar, se presentan la notación arquitectónica y los principios de diseño que gobiernan cada decisión, [Sección 5.1](#). A continuación, se desarrolla la arquitectura estática del sistema en sus niveles de contexto, contenedores y componentes, [Sección 5.2](#), completada por el diseño de detalle, [Sección 5.3](#), que captura su comportamiento y sus acuerdos: el proceso multi-agente, [Sección 5.3.1](#), el contrato universal de datos, [Sección 5.3.2](#), la comunicación asíncrona, [Sección 5.3.3](#), y las interfaces de programación y exportación a estándares, [Sección 5.3.4](#). Posteriormente, se describen las decisiones de seguridad, [Sección 5.4](#), cubriendo los riesgos de OWASP [29] identificados en la [Sección 3.3.2](#). El capítulo concluye con el diseño del despliegue, [Sección 5.5](#), y la matriz de trazabilidad que vincula cada requisito con los elementos de diseño que lo satisfacen, [Sección 5.6](#).

5.1. Gestión del Diseño

Esta sección establece el marco empleado para documentar la arquitectura, [Sección 5.1.1](#), y los principios de diseño que actúan como criterio de decisión transversal, [Sección 5.1.2](#), en cumplimiento del objetivo O5 descrito en la [Sección 1.2](#).

5.1.1. Modelo C4 para la Notación Arquitectónica

La arquitectura se ilustra, [Sección 5.2](#), siguiendo el Modelo C4 [41], un estándar que organiza la descripción de un sistema en cuatro niveles de abstracción progresivos. Cada nivel actúa a distinta escala, comenzando por el **contexto** que sitúa el sistema frente a sus usuarios y sistemas externos, seguido de los **contenedores** que identifican las unidades ejecutables y desplegables de forma independiente. Los **componentes** revelan la estructura interna de cada contenedor, delegando en el **código** el detalle de la implementación, [Sección 6.1](#). Esta aproximación incremental permite que audiencias con distinto grado de conocimiento técnico compartan un mismo enfoque del sistema.

En el presente capítulo se desarrollan los tres primeros niveles, introducidos en la [Sección 5.2](#). El cuarto nivel, correspondiente al detalle del código, se omite deliberadamente siguiendo la recomendación del propio modelo [\[41, Cap. 6\]](#), debido a su volatilidad y coste de mantenimiento. Cabe destacar que su contenido queda representado por el contrato de datos, expuesto en la [Sección 5.3.2](#), por el repositorio de GitHub [\[31\]](#) y por el [Capítulo 6](#).

5.1.2. Principios Rectores

Toda decisión de diseño, recogida en esta parte, se evalúa contra un conjunto reducido de principios, seleccionados durante la fase de elaboración de bocetos descrita en la [Sección 3.1.1](#):

- **Principio de responsabilidad única.** Cada elemento del sistema posee un único motivo de cambio [\[16, Cap. 7\]](#). Este principio gobierna la descomposición del proceso de generación en agentes especializados —taxonomista, relacional y poblador— donde cada agente recibe únicamente la porción del contrato de datos que le corresponde, imposibilitando por diseño que invada la responsabilidad de otro, observable en la [Sección 5.3.1](#).
- **Principio de abierto/cerrado.** El sistema queda abierto a la extensión y cerrado a la modificación [\[16, Cap. 8\]](#), en respuesta directa al requisito [REQ-SW-NF-06](#). La sustitución del modelo de lenguaje o del motor de inferencia que se utilizan requiere exclusivamente un cambio de configuración, satisfaciendo el [REQ-SW-FC-02](#). Adicionalmente, los formatos de exportación se registran de forma declarativa y el contrato de datos reserva un campo de metadatos extensible para evoluciones futuras.
- **Arquitectura orientada a eventos.** La naturaleza asíncrona de la inferencia del LLM exige desacoplar la petición del usuario de la producción del resultado [\[19, Cap. 11\]](#). La selección de [SSE](#) como mecanismo de transporte, justificada en la [Sección 2.5](#), condiciona el diseño de la comunicación, desarrollado en la [Sección 5.3.3](#), y cumple con el requisito [REQ-US-FC-02](#).
- **Contrato primero.** Un esquema de datos común, presentado en la [Sección 5.3.2](#) y anticipado en la [Sección 2.3](#), actúa como fuente de verdad para la generación guiada del modelo de lenguaje, la validación y la compilación a formatos del [W3C](#). Cualquier evolución del sistema parte de la modificación del contrato, y nunca al contrario, eliminando por construcción la deriva entre productores y consumidores del dato.
- **Separación entre lo estocástico y lo determinista.** El componente probabilístico, el LLM, queda confinado a la generación de contenido, mientras que toda decisión de aceptación, reparación o rechazo recae en componentes deterministas y verificables. Este principio, consecuencia directa de la alucinación del LLM analizada en la [Sección 2.2](#), fundamenta el ciclo de validación y autocorrección de la [Sección 5.3.1](#) y las mitigaciones de seguridad de la [Sección 5.4](#).

5.2. Arquitectura del Sistema

Esta sección recorre la estructura estática de OntoNova aplicando los niveles del Modelo C4 [41], notación establecida en la Sección 5.1.1. El contexto delimita la frontera del sistema frente al usuario y los sistemas externos, Sección 5.2.1, los contenedores constituyen sus unidades ejecutables y desplegadas, Sección 5.2.2, y los componentes detallan el interior de cada uno de ellos, Sección 5.2.3.

5.2.1. Nivel de Contexto

En el primer nivel del Modelo C4 [41, Cap. 3], OntoNova se presenta como una caja negra cuyo interior no impacta a esta escala. El verdadero significado recae en la frontera que separa el sistema de las personas y los sistemas que lo rodean. La Figura 5.1 recoge esta perspectiva, en la que el sistema se relaciona con un actor humano y una única dependencia externa, quedando ambas interacciones contenidas por la frontera de confianza del despliegue local.

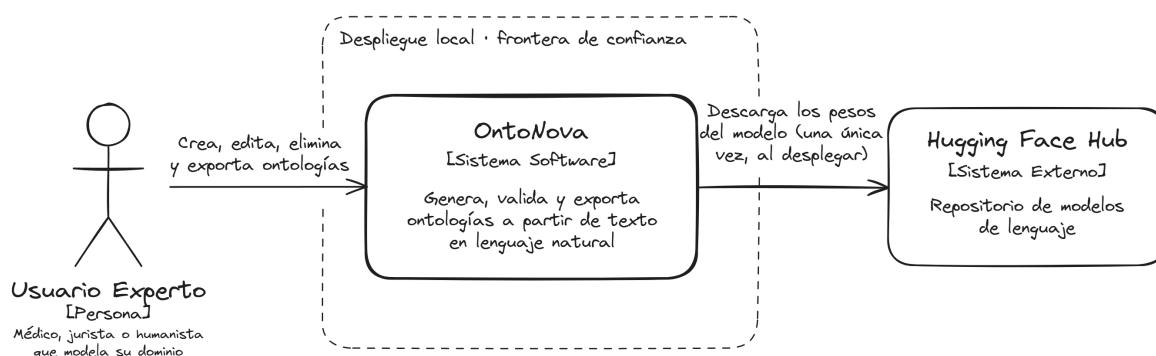


Figura 5.1. Diagrama C4 del contexto del sistema.

El actor del sistema es el usuario experto —médico, jurista, humanista— que interactúa con OntoNova haciendo uso de la interfaz web para crear, editar, eliminar y exportar ontologías, materializando los casos de uso expuestos en la Sección 4.2.3. Cabe destacar que la interacción se produce en el idioma del propio usuario, bien describiendo el dominio o bien adjuntando un documento, en cumplimiento de los requisitos REQ-US-FC-01 y REQ-US-FC-10.

La única dependencia externa del sistema es el repositorio de modelos Hugging Face Hub⁸, consultado una sola vez en la fase del despliegue para la descarga de los pesos del modelo de lenguaje. Superada esa descarga inicial, OntoNova opera localmente, realizando todo el procesamiento, inferencia incluida, dentro de la frontera de confianza representada en la Figura 5.1, sin que los datos del usuario abandonen el entorno local. Esta propiedad, que a nivel de contexto se enuncia como decisión de alcance, fundamenta las garantías de privacidad desarrolladas en la Sección 5.4, en cumplimiento de la legislación vigente, Sección 3.3.1. Quedan fuera de alcance la edición concurrente y la persistencia entre sesiones, requisitos listados bajo el estado «Requiere nuevo diseño» en la Sección 4.2.1.

⁸Disponible en <https://huggingface.co>.

5.2.2. Nivel de Contenedores

Descendiendo un nivel de abstracción [41, Cap. 4], la Figura 5.2 descompone el sistema en unidades ejecutables y desplegadas de forma independiente denominadas contenedores⁹. El sistema se compone de tres contenedores —la aplicación web, el servicio de orquestación y el motor de inferencia— conectados por dos protocolos que posibilitan los requisitos REQ-SW-FC-02 y REQ-US-FC-02. Cabe resaltar la ausencia de un contenedor de persistencia, cuya incorporación queda delegada al trabajo futuro, Sección 9.3, unida a los requisitos REQ-US-FC-06 y REQ-US-FC-07 que la motivan.

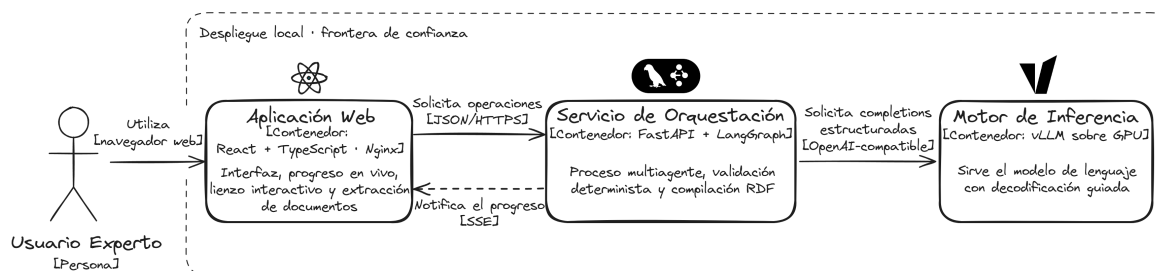


Figura 5.2. Diagrama C4 de los contenedores del sistema.

La aplicación web es una **Aplicación de Página Única (SPA)** desarrollada en *React* y *TypeScript*, servida por *Nginx*, que concentra toda la interacción con el usuario. Entre sus responsabilidades destaca la entrada del texto del dominio, mecanografiada o extraída de ficheros en el propio navegador, REQ-US-FC-10, el seguimiento del progreso de generación, REQ-US-FC-02, y el lienzo interactivo de edición que responde directamente al REQ-US-FC-04. La elección de React como marco técnico en este ámbito, rivalizada en la Tabla 5.1, queda determinada por la madurez de su ecosistema de visualización de grafos, *React Flow*, sin equivalente en otras alternativas, decisiva para el lienzo interactivo.

TABLA 5.1

COMPARATIVA DE ALTERNATIVAS DEL CLIENTE

Alternativa	Curva de aprendizaje	Ecosistema	Soporte a grafos interactivos
VanillaJS	Alta	Mínimo	Deficiente
Angular	Alta	Rígido	Limitado
React	Media	Amplio	Nativo

El *backend* o servicio de orquestación es una **Interfaz de Programación de Aplicaciones (API)** desarrollada con el marco de trabajo *FastAPI*, seleccionado por su escasa curva de aprendizaje, documentación enriquecida y retroalimentación rápida. Por otro lado, *LangGraph* [18], elección debatida en la Sección 2.4, ejecuta el proceso multi-agente de generación, la validación determinista y la compilación a formatos W3C, REQ-US-FC-05, concentrando toda la lógica de negocio del sistema. La naturaleza asíncrona de ambos

⁹El término «contenedor» se extrae del Modelo C4, sin relación con la «contenerización» de la Sección 5.5, aunque en OntoNova los términos tienen correspondencia uno a uno.

marcos resulta idónea para un servicio cuyo trabajo consiste, esencialmente, en esperar a un modelo de lenguaje sin bloquear al resto de usuarios.

En tercer lugar, el motor de inferencia está compuesto por una instancia de *vLLM*. Como es observable en la comparativa de la [Tabla 5.2](#), la selección resulta crítica para sobrevivir a la latencia, tráfico, errores y saturación del sistema. *vLLM* soporta una concurrencia elevada por su arquitectura *paged attention* orientada a maximizar la tasa de procesamiento de la [Unidad de Procesamiento Gráfico \(GPU\)](#) bajo demanda; sin embargo, *Ollama* y *llama.cpp* se centran en inferencia local optimizada para un individuo: bajo cargas concurrentes, *vLLM* alcanza una tasa de procesamiento entre 20 y 29 veces superior y una latencia P95 entre 8 y 19 veces menor que *Ollama*, completando la totalidad de las peticiones [\[42\]](#). En la decodificación guiada por gramática sobre la [GPU](#) local, la técnica que garantiza salidas estructuralmente válidas, analizada en la [Sección 2.3](#), *vLLM* supera a sus alternativas debido a librerías optimizadas en creación de máquinas de estado, *XGrammar*, en contraparte con la [Gramática Backus-Naur para GGML \(GBNF\)](#) flexible pero de alto coste computacional de *Ollama* y *llama.cpp*. Cabe resaltar que todas las alternativas de código abierto mencionadas disponen de compatibilidad con el protocolo *OpenAI*. En consecuencia, el contenedor queda colocado detrás de una [API](#), lo que permite que el *backend* desconozca qué modelo sirve, respetando la separación de responsabilidades [\[16, Cap. 7\]](#) y cumpliendo con el requisito [REQ-SW-FC-02](#), además de obedecer al principio de inversión de dependencias [\[16, Cap. 11\]](#) y seguir el principio de abierto/cerrado establecido en la [Sección 5.1.2](#).

TABLA 5.2

COMPARATIVA DE ALTERNATIVAS DEL MOTOR DE INFERENCIA

Alternativa	Orientación	Procesamiento	Decodificación guiada	Protocolo OpenAI
Ollama	Monousuario	Bajo	Costosa (GBNF)	Compatible
llama.cpp	Biblioteca	Bajo	Costosa (GBNF)	Compatible
vLLM	Multiusuario	Alto	Optimizada	Compatible

En cuanto a los conectores, la aplicación web se comunica con el servicio de orquestación mediante peticiones [JSON](#) sobre [HTTP](#), y recibe el progreso de la generación como un flujo de eventos [SSE](#), acatando el requisito [REQ-US-FC-02](#) y consumando la arquitectura orientada a eventos justificada en la [Sección 2.5](#). Dicho diseño no contempla un contenedor de persistencia. *OntoNova* opera sin estado, pues el conocimiento generado vive exclusivamente en la sesión del navegador del usuario hasta que este decide exportarlo o eliminarlo, sin dejar rastro en el servidor. Esta decisión borra por construcción toda una categoría de riesgos sobre los datos del usuario, facilita el cumplimiento de la legislación aplicable, [Sección 3.3.1](#), cuyas garantías se presentan en la [Sección 5.4](#), y simplifica el despliegue descrito en la [Sección 5.5](#).

5.2.3. Nivel de Componentes

A una mayor granularidad, concretamente hacia el interior de los contenedores, se revelan sus componentes [41, Cap. 5], que quedan asignados a abstracciones reales en función de su código. Es decir, se agrupan funcionalidades relacionadas tras una interfaz bien definida.

La aplicación web, cuya estructura interna recoge la [Figura 5.3](#), ofrece al usuario dos puntos de entrada que se corresponden con los casos de uso de la [Sección 4.2.3](#). Estos son el panel de creación, donde describe el dominio y sigue el progreso de la generación por etapas, [REQ-US-FC-02](#), y el lienzo interactivo, donde refina y exporta el grafo resultante. El inspector completa la tríada de vistas, habilitando la edición de los atributos e individuos de la clase seleccionada. Los tres componentes de interfaz se articulan en torno al almacén de estado, fuente de verdad de la ontología en edición; leen y mutan dicho almacén sin comunicarse jamás entre sí, un patrón unidireccional que elimina por diseño las inconsistencias entre vistas. Cabe destacar que el almacén replica la estructura del contrato de datos, [Sección 5.3.2](#), de modo que el estado del lienzo resulta exportable en cualquier instante, [REQ-US-FC-05](#). Por otro lado, el panel de creación delega en el extractor de documentos la obtención del texto en el propio navegador, [REQ-US-FC-10](#), reforzando la frontera de privacidad establecida en la [Sección 5.2.1](#). Por último, la capa de API concentra toda la comunicación con el servicio de orquestación a través de la generación asíncrona y la revalidación de cada mutación del grafo, que mantiene el cumplimiento del [REQ-US-FC-04](#) en caliente.

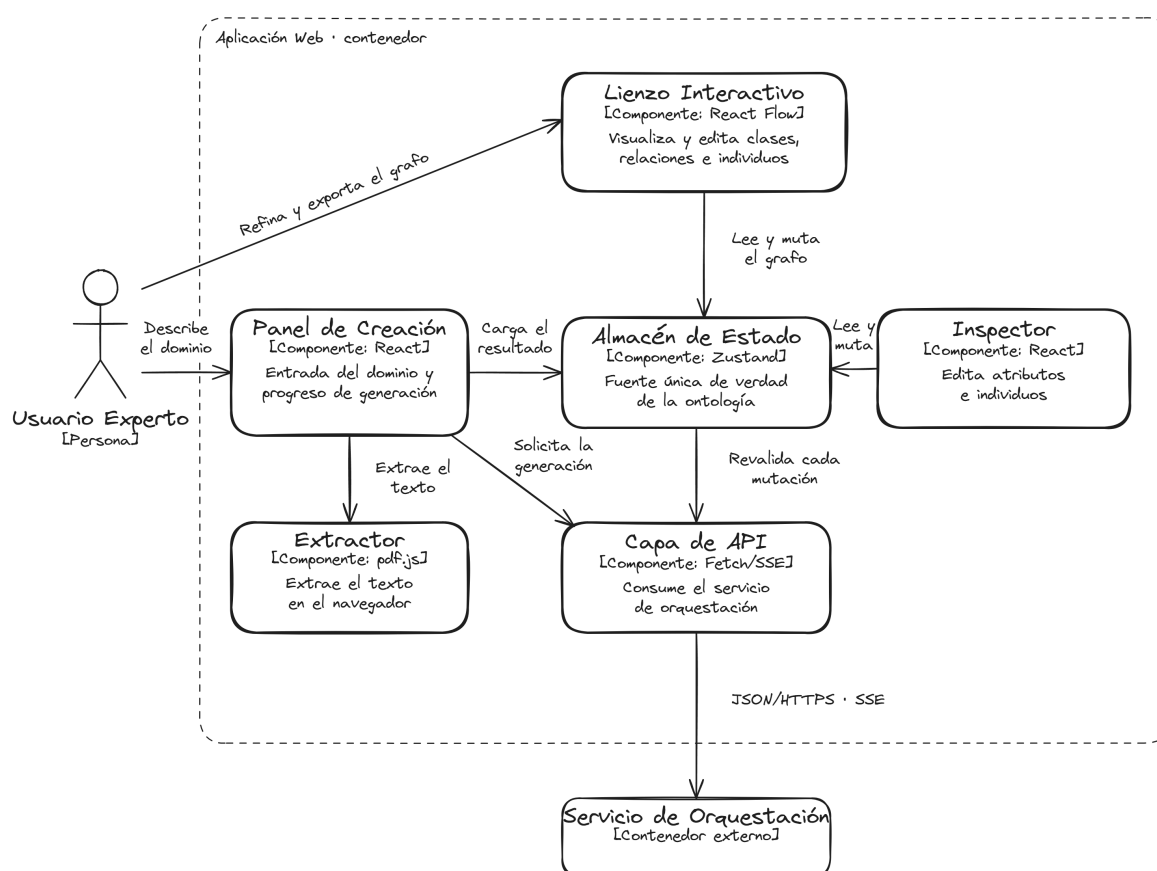


Figura 5.3. Diagrama C4 de los componentes de la aplicación web.

Por su parte, el servicio de orquestación o *backend* se organiza en tres capas¹⁰, visibles en la [Figura 5.4](#). Se establece una separación clara de responsabilidades, posibilitando su mantenimiento, escalado e independencia entre ellas [[16, Cap. 22](#)].

En la capa de transporte, el *router* de ontologías expone los tres recursos del sistema — generación, validación y exportación— detallados en la [Sección 5.3.4](#). La capa de dominio concentra el conocimiento de negocio en el contrato de datos, la fuente de verdad que gobierna el sistema, [Sección 5.3.2](#). La inteligencia queda contenida en el grafo multi-agente, que orquesta a los agentes especializados junto a su ciclo de ejecución y autocorrección, [Sección 5.3.1](#), en tanto que el validador determinista ejecuta las comprobaciones de integridad estructural y referencial que exige el [REQ-US-FC-05](#), cumpliendo con la separación entre lo estocástico y lo determinista comprometida en la [Sección 5.1.2](#). Finalmente, los adaptadores sirven de capa de salida hacia el exterior, disponiendo de un cliente de inferencia que encapsula el protocolo *OpenAI* y un compilador *RDF* que serializa el conocimiento a formatos estándar del *W3C*.

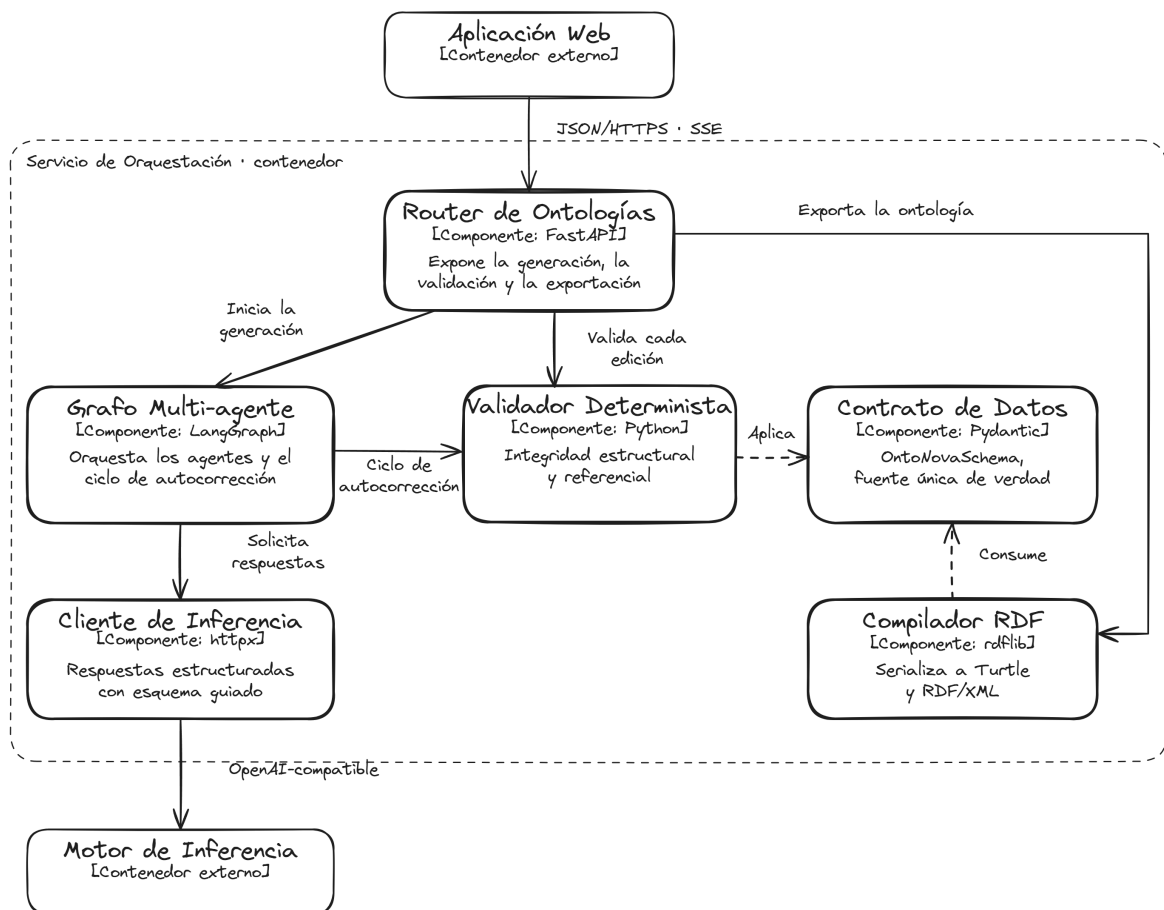


Figura 5.4. Diagrama C4 de los componentes del servicio de orquestación.

¹⁰El motor de inferencia no se desgana al tratarse de un producto de terceros.

5.3. Diseño de Detalle

Definida la estructura, esta sección descende al comportamiento y a los acuerdos que la gobiernan. Se abre con la vista dinámica del sistema —el diagrama suplementario que el Modelo C4 [41] reserva para la colaboración en tiempo de ejecución— aplicada al proceso multi-agente de generación, [Sección 5.3.1](#). En divisiones posteriores, el contrato universal de datos se erige como fuente de verdad del sistema, [Sección 5.3.2](#), seguido del diseño de la comunicación asíncrona, [Sección 5.3.3](#), y las interfaces de programación junto a la exportación a formatos estándar del W3C, [Sección 5.3.4](#).

5.3.1. Proceso Multi-agente de Generación

El Modelo C4 reserva un espacio para la vista dinámica del sistema [41, Cap. 7]. Esta sección aprovecha dicho espacio para ilustrar, en el diagrama de actividad de la [Figura 5.5](#) [40, Cap. 11], el recorrido de una petición de generación desde el texto del dominio hasta su evento terminal, acatando el requisito [REQ-US-FC-03](#), que exige una ontología que supere la validación con degradación elegante ante el fallo.

Como guardarraíl previo, toda petición atraviesa una comprobación determinista de longitud derivada de los límites de los requisitos [REQ-US-FC-01](#) y [REQ-US-FC-10](#), rechazando con un mensaje accionable las entradas que excedan la ventana de contexto del modelo en funcionamiento. Superada esta fase, la cadena de generación encadena tres agentes contruidos con *LangGraph* [18]: el taxonomista extrae las clases y su jerarquía, el relacional declara las propiedades de objeto y de datos sobre las clases ya existentes, y el poblador identifica los individuos y sus aserciones. Cada agente opera bajo un esquema de salida acotado a la porción del contrato que le corresponde, tal y como anticipó la [Sección 5.1.2](#), de modo que la decodificación guiada de la [Sección 2.3](#) imposibilita estructuralmente que un agente invada el terreno de otro.

El validador determinista cierra el ciclo evaluando la estructura, las referencias cruzadas y la conformidad de cada aserción con el dominio y rango declarados. Ante un resultado inválido, el sistema aplica un procedimiento de tres pasos que reserva el coste de la inferencia para los errores genuinamente semánticos. En primer lugar, una reparación sin pérdidas corrige de forma determinista los errores triviales como identificadores duplicados, referencias con grafía inconsistente y aserciones de dirección invertida, sin consumir reintento alguno. Si el error persiste, el validador identifica la etapa responsable a partir del propio mensaje de error y relanza la cadena desde ella, adjuntando al agente su salida previa junto a la instrucción de corregirla sin regenerarla, evitando que el modelo descarte contenido válido [14] para esquivar el error.

Agotado el presupuesto de cuatro reintentos, el último paso poda las aserciones inválidas y entrega un grafo válido aunque empobrecido, reservando el fallo total a los defectos que ninguna poda puede resolver. Toda ruta, incluida la de fallo, concluye con un evento terminal que es la invariante sobre la que descansa la comunicación asíncrona disponible en la [Sección 5.3.3](#).

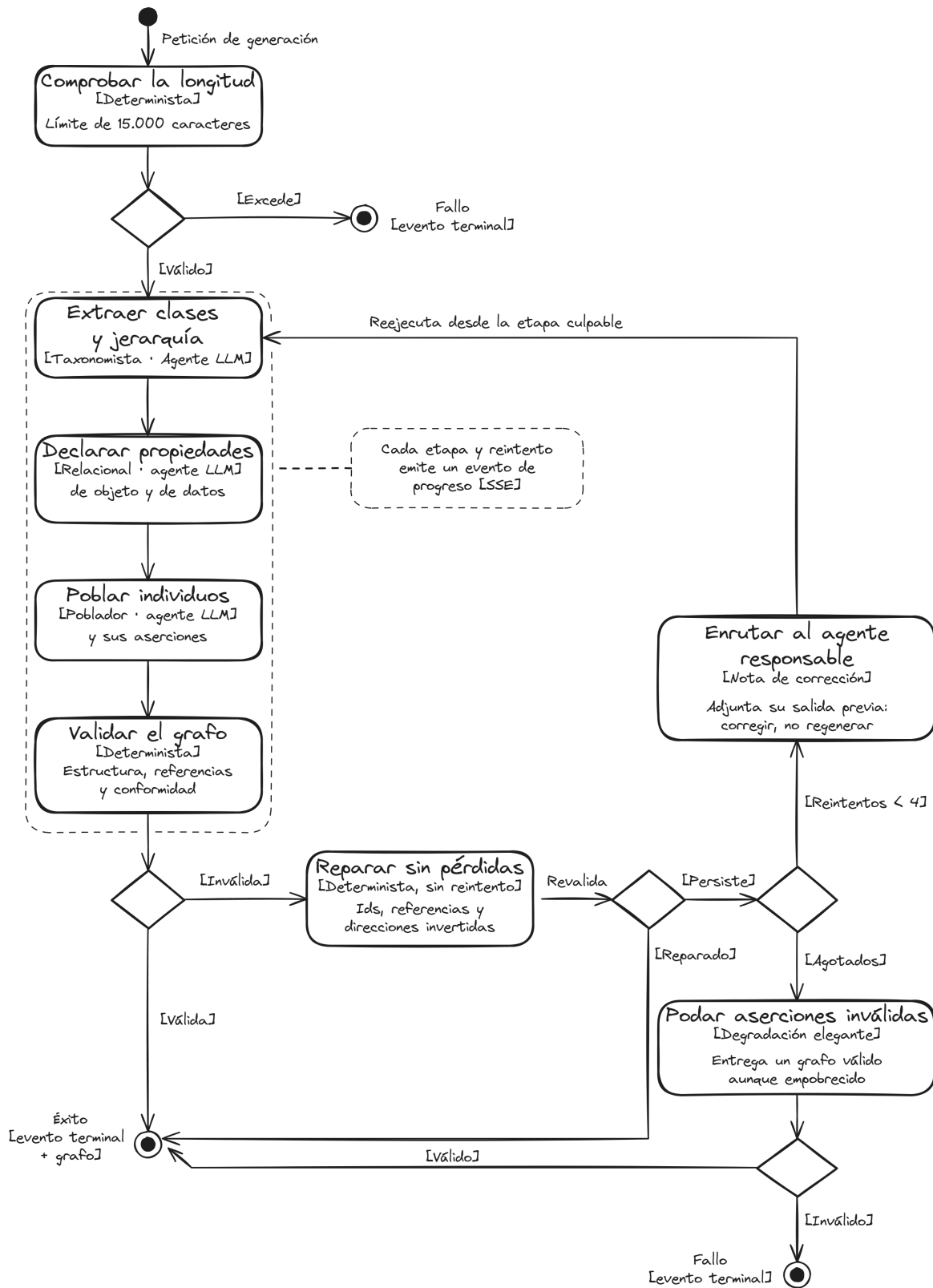


Figura 5.5. Diagrama de actividad del proceso multi-agente de generación¹¹.

¹¹La figura asume que el agente culpable es el taxonomista; el destino real es la etapa que el error señala.

5.3.2. Contrato Universal de Datos

Todo el sistema pivota sobre un esquema de datos, *OntoNovaSchema*, que implanta el principio de contrato primero establecido en la [Sección 5.1.2](#). El contrato actúa como gramática de la decodificación guiada, propiciando que cada agente reciba la porción del esquema que le corresponde, [Sección 5.3.1](#). A su vez, fundamenta las fases de validación determinista y de compilación a los formatos del W3C, [Sección 5.3.4](#). Cabe resaltar que cualquier evolución del sistema parte del contrato extensible [31], nunca al contrario, evitando incoherencias entre productores y consumidores. La [Tabla 5.3](#) descompone su estructura en las cuatro entidades que lo forman, alineadas con los pilares de la representación del conocimiento expuestos en la [Sección 2.1](#).

TABLA 5.3
ENTIDADES DEL CONTRATO UNIVERSAL DE DATOS

Entidad	Campos principales	Papel semántico
Clase	id, name, subClassOf	Concepto del dominio y su jerarquía taxonómica
Propiedad de objeto	id, name, domain, range, characteristics	Relación binaria entre clases
Propiedad de datos	id, name, domain, range	Atributo literal con tipo primitivo XML
Individuo	id, name, typeClass, aserciones	Instancia concreta y sus hechos

Los identificadores obedecen un patrón alfanumérico porque se utilizan directamente como nombres de recurso [RDF](#) en el caso de uso de exportar una ontología, restricción que neutraliza por construcción la inyección de contenido malicioso en la frontera del esquema, mitigación expandida en la [Sección 5.4](#). Otra característica del contrato es la prohibición de todo campo no declarado, de modo que alucinaciones del [LLM](#) [14] provocan un fallo de validación, con su consiguiente autocorrección, en lugar de propagarse silenciosamente. Por último, cada entidad reserva un campo de metadatos extensible, la vía de evolución prevista por el principio de abierto/cerrado de la [Sección 5.1.2](#).

En términos de expresividad, el contrato define un perfil reducido de [OWL](#) compuesto por jerarquías mediante `subClassOf`, propiedades con dominio y rango, las siete características de propiedad del estándar —funcional, inversa funcional, transitiva, simétrica, asimétrica, reflexiva e irreflexiva—, cinco tipos primitivos¹² y las aserciones de individuos. Quedan fuera las restricciones de cardinalidad, las clases anónimas y los axiomas de equivalencia; elementos que herramientas exhaustivas como *Protégé* ofrecen a cambio, precisamente, de la complejidad que este trabajo pretende abstraer, tal y como recoge la [Sección 2.6](#). Su incorporación gradual se anota en la [Sección 9.3](#), sin coste estructural gracias al campo de metadatos.

¹²Los tipos primitivos soportados son: string, integer, float, boolean, dateTime.

5.3.3. Comunicación Asíncrona de Eventos

Justificada la elección de SSE como mecanismo de transporte en la Sección 2.5, esta sección diseña el vocabulario del canal de comunicación, catalogado en la Tabla 5.4, declarando los eventos que el servicio de orquestación emite hacia la aplicación web durante la generación. Cada transición del proceso descrito en la Sección 5.3.1 produce un evento, permitiendo que la interfaz refleje el estado real de la generación con la latencia máxima de un segundo que impone el requisito REQ-US-FC-02.

TABLA 5.4
CATÁLOGO DE EVENTOS DEL FLUJO DE GENERACIÓN

Estado	Carga útil	Etapas
Completado		Taxonomista • Relacional • Poblador • Validador
En reintento	Motivo del error	Validador
Fallo	Motivo y, si procede, código con parámetros	Validador • Generación • Fin
Éxito	Grafo completo	Fin

Toda petición, con independencia de su desenlace —éxito o fallo—, concluye con un evento de cierre que transporta el grafo, con o sin poda, o el motivo del error. La aplicación web libera la interfaz al recibirlo y trata como error cualquier flujo que se cierre sin él, de esta manera ningún fallo del servicio, del LLM o de la red puede dejar al usuario esperando indefinidamente, logrando la resiliencia comprometida en el requisito REQ-SW-NF-03. Adicionalmente, los eventos de fallo transportan, junto a un texto descriptivo, un código de error acompañado de sus parámetros, permitiendo a la aplicación web presentar el error localizado en el idioma del usuario, en coherencia con el carácter multilingüe del sistema, REQ-US-FC-01, sin privar de contexto a los consumidores directos de la API, cuyos recursos se detallan en la Sección 5.3.4.

5.3.4. Interfaces de Programación y Exportación

El servicio de orquestación publica tres recursos de API consumidos por la aplicación web. Primeramente, la **generación** se modela en *streaming*, alejado del clásico esquema petición-respuesta. El cliente formula una solicitud y recibe el canal de eventos de la Sección 5.3.3, asumiendo la asimetría entre lo inmediato de preguntar y lo costoso de inferir. En segundo lugar, en la **validación**, a diferencia de la convención habitual de una API [19, Cap. 4], un grafo inválido no constituye un error en la petición, sino una respuesta aceptada que transporta los defectos de forma estructurada. El motivo es cuestión de diseño, porque a lo largo de la edición sobre el lienzo, el estado intermedio inválido es lo esperado porque, por ejemplo, el usuario aún no ha conectado dos clases o está renombrando un atributo. Tratarlo como excepción convertiría el ciclo de revalidación del requisito REQ-US-FC-04 en un flujo permanente de errores. Por último, la **exportación** se alinea con REQ-US-FC-05 y solo compila a un formato del W3C aquello que supera la validación determinista, evitando la fuga de archivos inválidos, bien por incumplimiento del estándar o bien por incompatibilidad con el contrato de la Sección 5.3.2.

5.4. Diseño de Seguridad

La seguridad de OntoNova no constituye un módulo adicional, sino una evaluación de las decisiones tomadas bajo la lente del riesgo, pues cada mitigación que se expone a continuación es una decisión de diseño presentada en el transcurso del capítulo, ahora justificada frente a los cuatro riesgos del OWASP Top 10 GenIA/LLM identificados en la [Sección 3.3.2](#). Se establece una defensa en profundidad, descendiendo por los niveles que estructuran la [Sección 5.2](#), estrechando la protección a medida que el dato se acerca al modelo de lenguaje.

La protección comienza en el nivel de contexto, donde la frontera de confianza enunciada en la [Sección 5.2.1](#) se convierte en la garantía de privacidad allí prometida. Al operar el sistema en local y sin estado, [Sección 5.2.2](#), el conocimiento del dominio, potencialmente confidencial en manos de un médico o un jurista, ni atraviesa la frontera ni persiste tras la sesión, y el único contacto con el exterior, la descarga inicial de los pesos del modelo, no transporta dato alguno del usuario. Esta doble ausencia neutraliza por construcción la divulgación de información sensible [\[29, LLM02\]](#) y facilita el cumplimiento de la legislación aplicable, [Sección 3.3.1](#), pues no puede filtrarse lo que nunca sale ni almacenarse lo que nunca se guarda.

Un nivel por debajo, los contenedores acotan las vías de entrada y salida. El motor de inferencia permanece aislado tras el protocolo estándar de la [Sección 5.2.2](#), la aplicación web solamente acepta orígenes cruzados autorizados y el suministro de dependencias se produce sin vulnerabilidades de severidad alta, verificado en el [Capítulo 7](#) en conformidad con el requisito [REQ-SW-NF-02](#). La contención más fina ocurre, sin embargo, en el nivel de componentes, donde gobierna la separación entre lo estocástico y lo determinista comprometida en la [Sección 5.1.2](#). La decodificación guiada acota la salida del modelo a la gramática del contrato, por lo que una inyección de *prompts* [\[29, LLM01\]](#) puede sesgar el contenido generado, pero nunca escapar de la estructura ni desencadenar acción alguna debido a que el modelo está capacitado exclusivamente para rellenar un esquema, careciendo de herramientas invocables o privilegios del sistema. Cuanto rellena tampoco se integra a ciegas, porque el contrato de la [Sección 5.3.2](#) prohíbe los campos desconocidos y restringe los identificadores de recurso [RDF](#), saneando las salidas del [LLM](#) antes de que alcancen el lienzo interactivo o la exportación [\[29, LLM05\]](#).

Existe, no obstante, un riesgo que escapa a la gramática, producido por una alucinación estructuralmente válida que supone una desinformación [\[29, LLM09\]](#) indetectable para el esquema. Su contención recae en el validador determinista y el ciclo de autocorrección de la [Sección 5.3.1](#), que eliminan las inconsistencias referenciales, y su arbitraje final, en el experto que revisa y refina el grafo sobre el lienzo, en consonancia con la supervisión humana que demanda el marco regulatorio, [Sección 3.3.1](#).

Otros riesgos ligados a requisitos no incluidos como la autenticación y autorización, [REQ-SW-NF-07](#), o el registro de usuarios reclamado por el requisito [REQ-US-FC-09](#), retrasan su examen al escenario de exposición pública contemplado en el [Capítulo 8](#), puesto que un despliegue local y monousuario no las requiere.

5.5. Diseño de Despliegue

El despliegue se documenta mediante el diagrama suplementario con el que el Modelo C4 completa la vista estática [41, Cap. 8], respondiendo a la cuestión de dónde vive cada contenedor. La Figura 5.6 sitúa el sistema completo en un nodo, el equipo del experto, donde la correspondencia anticipada en la Sección 5.2.2 se hace realidad, debido a que cada contenedor arquitectónico se materializa en un contenedor de *Docker* orquestado por *docker compose*. La aplicación web es la pieza que escapa parcialmente a dicha correspondencia, ya que se distribuye desde el servidor estático *nginx* pero se ejecuta en el navegador del propio computador, consumiendo el servicio de orquestación bajo los orígenes autorizados, Sección 5.4.

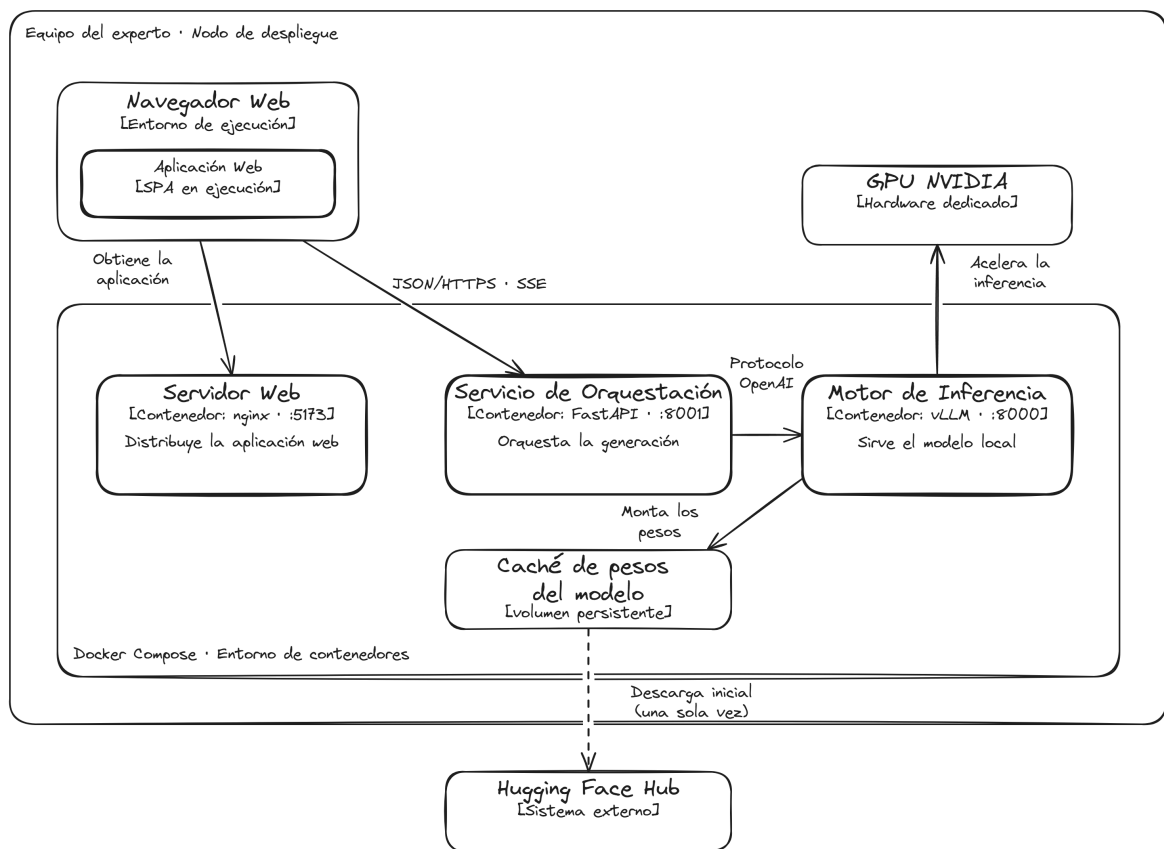


Figura 5.6. Diagrama C4 de despliegue del sistema.

La composición de contenedores adopta un comando de lanzamiento, configurable con variables de entorno. A su vez, dado que los contenedores dependen entre sí, los *health checks* gobiernan el orden de arranque, por ejemplo, el servicio de orquestación no acepta trabajo hasta que el motor de inferencia supera su comprobación de despliegue. Se emplea un volumen persistente a modo de caché que conserva los pesos del modelo, logrando, una vez poblado el volumen, un arranque por debajo de cinco minutos, REQ-SW-NF-08, verificable en el Capítulo 7. Asimismo, se reduce la descarga desde Hugging Face Hub¹³ a un solo evento en la vida del despliegue, tal y como enuncia la Sección 5.2.1.

¹³Disponible en <https://huggingface.co>.

El nodo de trabajo necesita un anfitrión con *Docker*, una GPU compatible con la *Arquitectura Unificada de Dispositivos de Cómputo (CUDA)* y sus respectivas librerías, *NVIDIA Container Toolkit*. Aunque carece de dependencia de servicios externos, sí necesita un núcleo Linux —nativo o virtualizado—, la única plataforma soportada por *vLLM*. Aun así, consume el carácter agnóstico del *hardware* que requiere el requisito **REQ-SW-NF-08** y preserva la frontera de confianza en cualquier anfitrión. Esa neutralidad habilita el camino inverso, es decir, la composición que se despliega en el equipo del experto es la que un proveedor de exposición pública puede ejecutar con leves modificaciones, escenario propuesto en el [Capítulo 8](#).

5.6. Trazabilidad de Requisitos

La propiedad de trazabilidad solicitada por los estándares de la [Sección 4.1.2](#) se refleja en la [Tabla 5.5](#), para los requisitos de usuario, y en la [Tabla 5.6](#), para los del sistema, que vinculan cada requisito de la [Sección 4.2](#) con los elementos de diseño que lo satisfacen. Los requisitos diferidos no quedan huérfanos, sino trazados a la decisión de alcance que los pospone; los de naturaleza operativa —soporte, pruebas y documentación— se remiten al proceso que los gobierna porque no adoptan forma de diseño. La comprobación de que cada elemento cumple su métrica no corresponde al diseño, sino al [Capítulo 7](#), cuya matriz de verificación toma la [Tabla 5.5](#) de requisitos de usuario y la [Tabla 5.6](#) de requisitos del sistema como punto de partida.

TABLA 5.5

MATRIZ DE TRAZABILIDAD DE REQUISITOS DE USUARIO A ELEMENTOS DE DISEÑO

Requisito	Elementos de diseño
REQ-US-FC-01	Panel de creación, Sección 5.2.3 , y comprobación de longitud, Sección 5.3.1 .
REQ-US-FC-02	Conector SSE, Sección 5.2.2 , y catálogo de eventos, Sección 5.3.3 .
REQ-US-FC-03	Proceso multi-agente con degradación elegante, Sección 5.3.1 .
REQ-US-FC-04	Lienzo y almacén de estado, Sección 5.2.3 , y validación reactiva, Sección 5.3.4 .
REQ-US-FC-05	Exportación como guardián, Sección 5.3.4 , y compilador RDF, Sección 5.2.3 .
REQ-US-FC-06	Diferido junto a la persistencia, Sección 5.2.2 , a la Sección 9.3 .
REQ-US-FC-07	Diferido junto a la persistencia, Sección 5.2.2 , a la Sección 9.3 .
REQ-US-FC-08	Diferido a la Sección 9.3 .
REQ-US-FC-09	Diferido, Sección 5.4 , al Capítulo 8 .
REQ-US-FC-10	Extracción de documentos, Sección 5.2.3 , y comprobación de longitud, Sección 5.3.1 .
REQ-US-NF-01	Componentes de interfaz y patrón unidireccional, Sección 5.2.3 .
REQ-US-NF-02	SPA servida en local, Sección 5.2.2 y Sección 5.5 .
REQ-US-NF-03	Proceso operativo, sin forma de diseño.

TABLA 5.6

MATRIZ DE TRAZABILIDAD DE REQUISITOS DEL SISTEMA A ELEMENTOS DE DISEÑO

Requisito	Elementos de diseño
REQ-SW-FC-01	Diferido a la Sección 9.3.
REQ-SW-FC-02	Conector <i>OpenAI</i> , Sección 5.2.2, cliente de inferencia, Sección 5.2.3, y variables de entorno, Sección 5.5.
REQ-SW-NF-01	Diferido a la Sección 9.3.
REQ-SW-NF-02	Cadena de suministro auditada, Sección 5.4.
REQ-SW-NF-03	Invariante del evento terminal, Sección 5.3.3, y escalera de degradación, Sección 5.3.1.
REQ-SW-NF-04	Proceso de pruebas, diferido a la Sección 9.3.
REQ-SW-NF-05	Proceso documental del proyecto, sin forma de diseño.
REQ-SW-NF-06	Principio de abierto/cerrado, Sección 5.1.2, y metadatos extensibles, Sección 5.3.2.
REQ-SW-NF-07	Diferido, Sección 5.4, al Capítulo 8.
REQ-SW-NF-08	Composición y arranque de contenedores, Sección 5.5.

CAPÍTULO 6

IMPLEMENTACIÓN

Este capítulo explora la implementación del proyecto, detallando la organización del código fuente y su correspondencia directa con los componentes del Modelo C4 [41, Cap. 6], Sección 6.1. Adicionalmente, se presentan los hitos de implementación alcanzados, Sección 6.2, así como las dificultades encontradas y decisiones técnicas tomadas durante el desarrollo del mismo, Sección 6.3.

6.1. Entorno y Organización del Código

El desarrollo, la experimentación y la validación del sistema, sometida a juicio en el Capítulo 7, se realizaron íntegramente sobre un equipo de consumo, con librerías visibles en la Tabla 6.1, dotado de una GPU NVIDIA RTX 4090 con 24GB de memoria y un núcleo Linux Ubuntu 24.04.1 LTS virtualizado en WSL 2.7.11, en coherencia con el despliegue diseñado en la Sección 5.5. El uso de *hardware* doméstico¹⁴ refuerza la soberanía tecnológica perseguida, y demuestra que la barrera de entrada es alcanzable para cualquier individuo, Sección 3.3.4.

TABLA 6.1
CONJUNTO TECNOLÓGICO DE LA IMPLEMENTACIÓN

Ámbito	Tecnologías
Aplicación web	React 19 • TypeScript 6 • Vite 8 • React Flow 12 • Zustand 5 • pdf.js 6
Servicio de orquestación	Python 3.12 • FastAPI 0.141 • LangGraph 1.0 • Pydantic 2 • rdflib 7
Motor de inferencia	vLLM 0.9.2 • Qwen3-14B con cuantización AWQ de 4 bits
Infraestructura	Docker Compose 2 • Nginx 1.27 • NVIDIA Container Toolkit

¹⁴El Capítulo 8 plantea un marco teórico con *hardware* remoto.

La organización del código honra el compromiso de la [Sección 5.1.1](#), que delega el cuarto nivel del Modelo C4 [41, Cap. 6] en el repositorio de GitHub [31]. La [Figura 6.1](#) salda esa deuda recorriendo el árbol del proyecto y marcando la correspondencia, en sentido descendente, de cada componente de la [Sección 5.2.3](#) con el fichero que lo implementa, generando una navegación inmediata del diagrama al código, sin ambigüedad. Otras partes del repositorio —pruebas, configuración y guías técnicas— se omiten del árbol al no adoptar forma de componente, sin que ello reste protagonismo al papel que desempeñan en el [Capítulo 7](#).

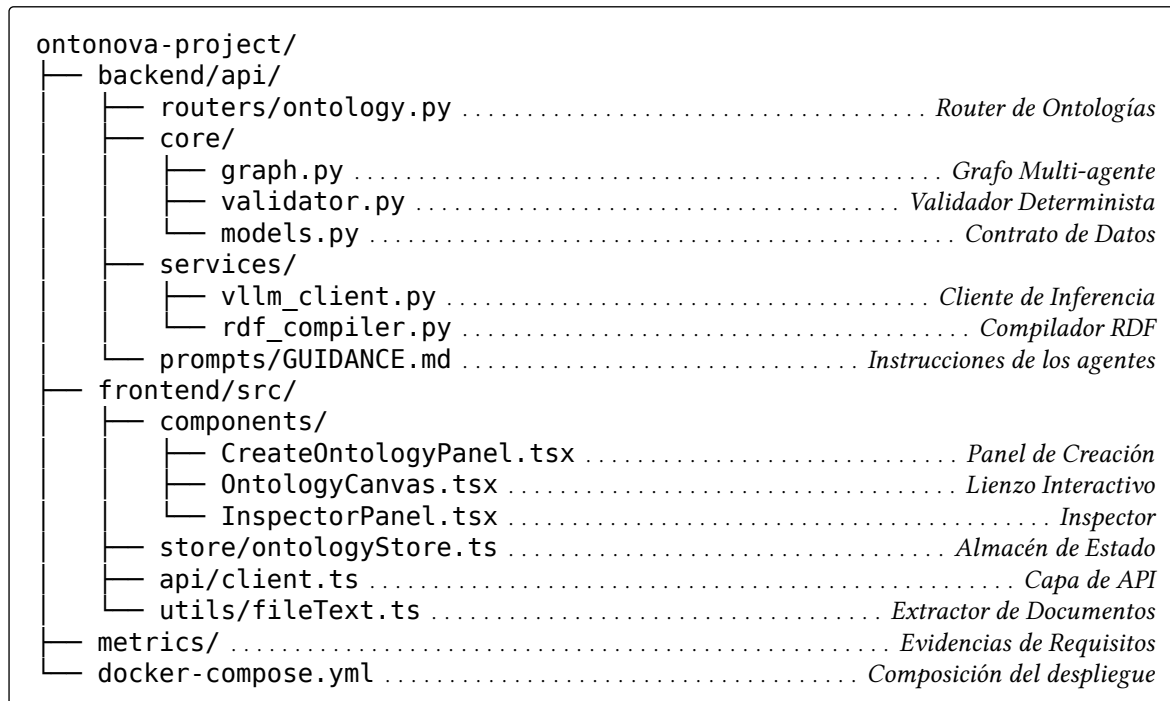


Figura 6.1. Correspondencia entre el código y los componentes C4.

6.2. Hitos de Implementación

El desarrollo iterativo, enunciado en la [Sección 3.1.1](#), produce tres hitos que expresan el valor técnico de la implementación, cada uno ligado a su evidencia, complementados por el desarrollo restante almacenado en el repositorio [31].

El primer logro salda el compromiso contraído en la [Sección 2.3](#) permitiendo convertir el contrato de datos en una gramática formal. Cada agente especializado —taxonomista, relacional y poblador— declara su salida como una porción del contrato de la [Sección 5.3.2](#) utilizando modelos de *Pydantic*, cuya conversión a *JSON schema* viaja en cada petición en el campo `guided_json` del protocolo *OpenAI*, transporte ilustrado en el fragmento de código de la [Figura 6.2](#). De esta manera, cada nodo del grafo trabaja para construir la porción de la fuente de verdad que le corresponde, persiguiendo un estado final del grafo que evite duplicidades en los tramos de información, recogidos entre la definición y el motor de inferencia.

```

1 | # core/graph.py
2 | class TaxonomistOutput(BaseModel):
3 |     classes: List[OntoClass] = Field(default_factory=list)
4 |     async def taxonomist_node(state: OntologyGenerationState) -> Dict[str, Any]:
5 |         result = await generate_structured(
6 |             messages,
7 |             TaxonomistOutput.model_json_schema(),
8 |             base_url=_base_url("taxonomist"),
9 |         )
10 | # services/vllm_client.py
11 | payload = {
12 |     "model": model,
13 |     "messages": messages,
14 |     "temperature": 0.0,
15 |     "guided_json": json_schema
16 | }

```

Figura 6.2. Del contrato a la gramática de decodificación guiada.

En segundo lugar se implementa el flujo reactivo de extremo a extremo. Para eventos SSE, una limitación destacable de la interfaz nativa del navegador es la imposibilidad de enviar peticiones con cuerpo; por ello, la capa API del frontal tiene un consumidor propio de *Fetch API*, que procesa los eventos de la Sección 5.3.3 de manera incremental. La Figura 6.3 evidencia el resultado durante una generación real, con la primera etapa completada.

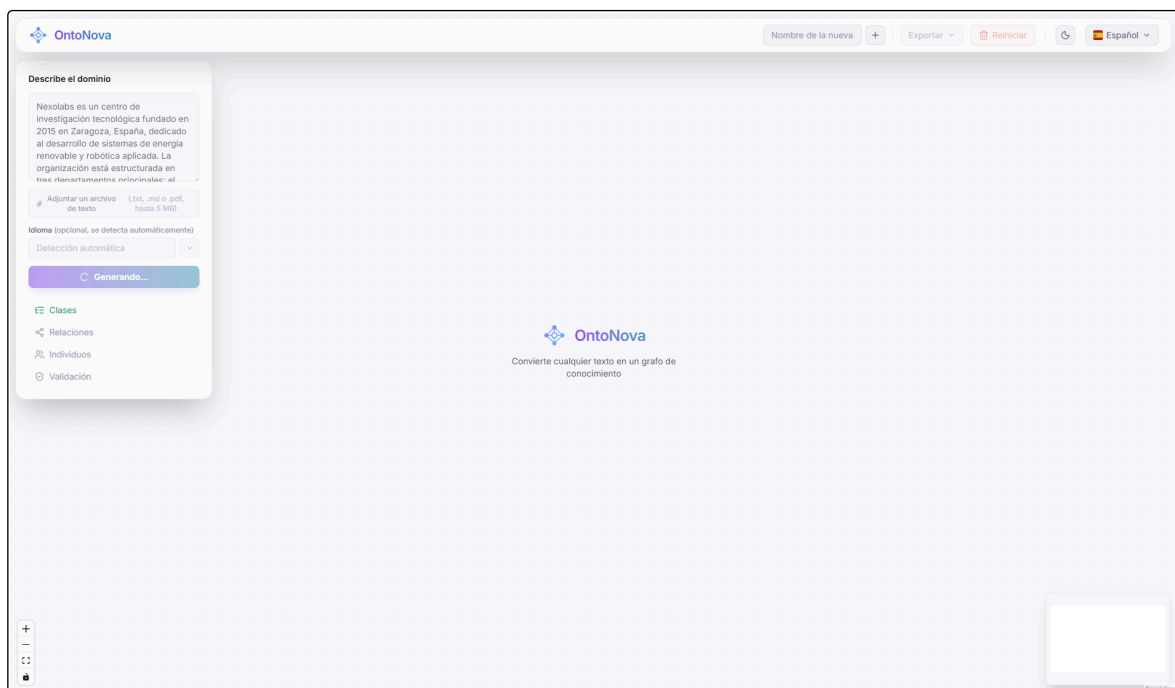


Figura 6.3. Generación de una ontología en curso con progreso por etapas.

El tercer hito reside en el ciclo de refinamiento o validación sobre el lienzo. El grafo generado se compone de nodos y aristas modificables, y cada cambio vuelve a verificar el estado contra el servicio de orquestación, asegurando que una futura exportación no genere inconsistencias o errores. La extracción de documentos se ejecuta en un hilo de trabajo del propio navegador y la exportación se resuelve en el servicio de orquestación local, sin que el fichero ni la ontología abandonen el equipo, en consonancia con la legislación aplicable, Sección 3.3.1. La Figura 6.4 muestra la ontología resultante del mismo dominio de la Figura 6.3, lista para su edición y exportación.

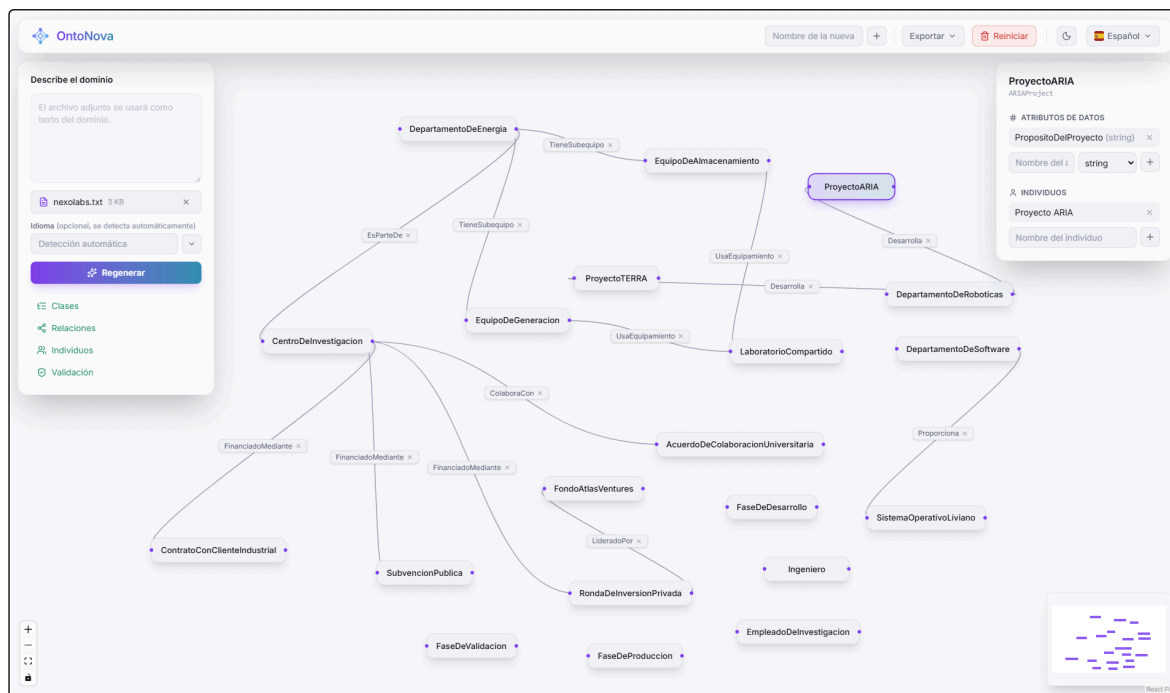


Figura 6.4. Ontología del dominio de la investigación sobre el lienzo interactivo.

6.3. Dificultades y Decisiones Técnicas

Las dificultades encontradas y decisiones técnicas evaluadas en el transcurso de la elaboración de la aplicación OntoNova giran en torno al desafío de acercar un elemento probabilístico y no determinista, como un LLM, al plano determinista, reto presentado en la Sección 2.3 y trabajado en las fases de diseño, Capítulo 5, e implementación. En una etapa temprana, las primeras iteraciones de prueba revelan que, ante un error de validación, reintentar una etapa desde cero empeora el resultado, pues el modelo intenta satisfacer al validador por la vía barata, es decir, omitiendo todo el contenido implicado en el error. Ante esta tesitura, se decide que cada reintento en un nodo agente reciba su salida anterior junto al error concreto, con una instrucción explícita de reparar el problema en lugar de regenerar la construcción desde cero, Figura 6.5, activando la reparación sin pérdidas del validador, Sección 5.3.1, antes de consumir *tokens* adicionales¹⁵ de inferencia.

¹⁵En el ámbito de la IA, la reducción de *tokens* es una optimización de coste habitual.

Esta decisión eleva la calidad del sistema, pasando de un fallo irrecuperable hasta una puntuación casi perfecta, evidenciando, en el [Capítulo 7](#), que la fiabilidad del sistema proviene mayoritariamente de su ingeniería determinista con un complemento leve del refinamiento de instrucciones de sistema, delegado en la [Sección 9.3](#).

```

1 | # core/graph.py
2 | def _correction_note(state: OntologyGenerationState, stage: str) -> str:
3 |     retry_stage = state.get("retry_stage")
4 |     if not retry_stage or not state.get("last_error"):
5 |         return ""
6 |     if STAGE_ORDER.index(stage) < STAGE_ORDER.index(retry_stage):
7 |         return "" # this stage already succeeded and won't re-run this pass
8 |     previous_output = {key: state.get(key, []) for key in _STAGE_STATE_KEYS[stage]}
9 |     return (
10 |         "\n\nA previous attempt failed schema validation with this error:\n"
11 |         f"{state['last_error']}\n\n"
12 |         f"Your previous output for this stage was:\n{previous_output}\n\n"
13 |         "Produce a corrected version of that output: keep every entry that is "
14 |         "not implicated in the error and fix only what the error requires. Do "
15 |         "NOT drop previously produced valid content (classes, properties, "
16 |         "individuals or their assertions) just to make the error disappear. "
17 |         "If the error says an assertion uses an undeclared property or "
18 |         "individual id, the relationship itself is usually correct: declare "
19 |         "the missing item if this stage owns it, or re-express the assertion "
20 |         "with a declared id – removing the relationship is the last resort."
21 |     )

```

Figura 6.5. Entrada guiada de un nodo agente en fase de reintento.

Los textos densos en entidades traen consigo una segunda familia de problemas. En ocasiones, la decodificación guiada degenera en un bucle de espacios en blanco que consume el presupuesto de 8.192¹⁶ *tokens* reservados durante la generación, produciendo un **JSON** truncado y un error de análisis sintáctico¹⁷ notificado tras minutos de espera. Para mitigar la situación, se incorpora al proceso de generación una secuencia de parada junto a una detección explícita del truncamiento, asegurando la manejabilidad del mensaje.

Una verdad instructiva, asumida desde el [Capítulo 2](#), surge al constatar que una misma entrada produce salidas distintas entre ejecuciones, incluso generando resultados con temperatura nula. El procesamiento por lotes continuo del motor de inferencia *vLLM* altera el orden de las reducciones en coma flotante según las peticiones concurrentes y, al no tener propiedad asociativa este tipo de operaciones, *tokens* empatados se resuelven de forma distinta en cada ejecución, divergencia e impacto que el ciclo de reintentos amplifica después. En determinadas ejecuciones, los grafos presentan carencias de conectividad o número de clases, [Figura 6.6](#). Por consiguiente, se documenta un efímero intento de solución

¹⁶Presupuesto calculado en función de la GPU disponible, véase [Sección 6.1](#).

¹⁷Malformed response from <http://vllm:8000/v1>: Expecting “,” delimiter: line 20709 column 1 (char 39087).

con un suelo de calidad determinista, consistente en medir la proporción de clases aisladas y conceder un reintento extra, por encima de los cuatro establecidos en el proceso de reparación de la [Sección 5.3.1](#), dirigido al agente relacional con la lista nominal de clases huérfanas. La verificación sobre un texto organizativo de 3.000 caracteres demuestra que el reintento adicional no reduce el aislamiento, apuntando a la capacidad del modelo desplegado como causa, [Sección 6.1](#), para anclar dominios y rangos, y descartando la hipótesis inicial de una falta de oportunidades de corrección, siendo necesaria una nueva teoría en un reto futuro, [Sección 9.3](#).

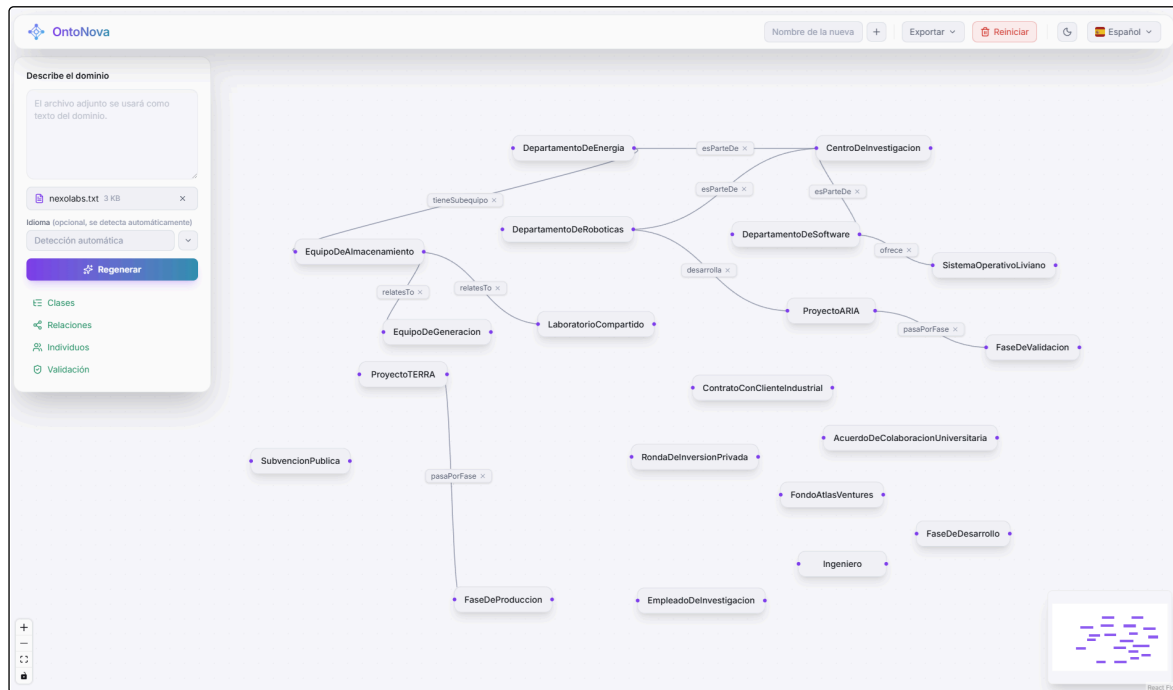


Figura 6.6. Ontología del dominio de la investigación con conectividad degradada.

Por último, el entorno local disponible impone sus propias decisiones. La versión del motor de inferencia queda fijada en la recogida por la [Tabla 6.1](#), al ser la última compatible con la virtualización WSL empleada. Como anécdota adicional, un error en tiempo de ejecución con módulos *ECMAScript* con un tipo *MIME* incorrecto deriva en la imposibilidad de utilizar ficheros como entrada, capacidad exigida por el requisito [REQ-US-FC-10](#). Las pruebas unitarias de la funcionalidad no se percatan de este tipo de problemas, que aflora con una prueba de extremo a extremo sobre un navegador real. Como primera lección, se orienta la estrategia de verificación del [Capítulo 7](#) contra un despliegue completo, sin perder de vista pruebas automáticas reducidas. Como segunda lección, ante la hipótesis de que determinadas acusaciones descritas pueden resolverse en un entorno con un modelo más potente, se delega en el trabajo futuro propuesto en el [Capítulo 8](#) la posibilidad de cambio de entorno, con los posibles efectos colaterales que esto pueda generar.

CAPÍTULO 7

VERIFICACIÓN

Este capítulo detalla la estrategia de pruebas automáticas [43] en conjunto con el balance de las mismas, Sección 7.1. Finalmente, se realiza la verificación de los requisitos establecidos e incluidos [35], Sección 7.2, y la revisión del cumplimiento del objetivo O4, Sección 7.3.

7.1. Estrategia y Balance de Pruebas Automáticas

La táctica de pruebas automáticas plantea como eje principal la pirámide de automatización de pruebas a distintos niveles [43, Cap. 16], que surge con las metodologías ágiles convirtiéndose en un estándar ágil en la industria. El modelo organiza las pruebas automáticas en tres niveles según su granularidad. La base concentra las pruebas unitarias, numerosas, rápidas y aisladas, que evalúan cada pieza por separado y localizan el defecto de código en el instante en que se introduce. El tramo intermedio agrupa las pruebas de servicio, de amplia cobertura aunque en menor número, entendiendo por servicio aquello que la aplicación hace en respuesta a una determinada entrada. Estos casos revisan la lógica de negocio separadamente de la interfaz, evitando el coste asociado a la capa de presentación. La cúspide reserva un espacio reducido a las pruebas de interfaz, fieles a la experiencia del usuario pero también lentas y costosas de mantener, motivo por el que su número es el más reducido en la pirámide. Un planteamiento alternativo consiste en invertir estas proporciones de pruebas, lo que produce un antipatrón, donde la capa dominante de pruebas es la de interfaz, encareciendo cada cambio y demorando la detección de errores. El proyecto no comparte dicha inversión, adoptando la táctica mencionada en contribución al enfoque de pruebas automáticas de desplazamiento a la izquierda [44]. La Figura 7.1 ilustra el modelo con el inventario del proyecto, superando 117 pruebas sin excepción sobre el entorno descrito en la Sección 6.1. La base reúne 72 pruebas unitarias repartidas entre ambos contenedores, con 19 pruebas del servicio de orquestación para el validador determinista, el cliente de inferencia y el compilador RDF, y 53 pruebas para los componentes de interfaz, el almacén de estado y las utilidades de extracción del contenedor de la aplicación web. El nivel de

7.2. Estándar IEEE para la Verificación de Requisitos

La verificación recorre los catorce requisitos con estado incluido de la [Sección 5.6](#), aplicando a cada uno el método que su métrica solicita entre los posibles que define la norma ISO/IEC/IEEE 29148:2018 [35], esto es, prueba, demostración, análisis e inspección. La [Tabla 7.2](#) anticipa el resultado global y las subsecciones siguientes, organizadas por método, presentan el procedimiento y la evidencia de cada requisito.

TABLA 7.2
RESULTADO DE LA VERIFICACIÓN DE LOS REQUISITOS INCLUIDOS

Requisito	Métrica	Método	Resultado
REQ-US-FC-01	15.000 caracteres, al menos 2 idiomas	Prueba	Cumple
REQ-US-FC-02	Progreso por etapas, latencia $\leq 1s$	Prueba	Cumple
REQ-US-FC-03	Validación con ciclo de corrección	Prueba	Cumple
REQ-US-FC-04	Edición inferior o igual a 3 interacciones	Demostración	Cumple
REQ-US-FC-05	Exportación cumple con validador W3C	Prueba	Cumple
REQ-US-FC-10	Ficheros de texto o PDF $\leq 5MB$	Prueba	Cumple
REQ-US-NF-01	SUS ≥ 70 con al menos 5 usuarios	Análisis	Cumple
REQ-US-NF-02	Carga $\leq 3s$, interacción $< 200ms$	Prueba	Cumple
REQ-SW-FC-02	Cambio de modelo por configuración	Demostración	Cumple
REQ-SW-NF-02	Ninguna vulnerabilidad de severidad alta	Análisis	Cumple
REQ-SW-NF-03	Respuesta controlada ante errores	Prueba	Cumple
REQ-SW-NF-05	Despliegue autodidáctico por un tercero	Demostración	Cumple
REQ-SW-NF-06	Extensión sin rediseño	Inspección	Cumple
REQ-SW-NF-08	Despliegue containerizado $< 5min$	Prueba	Cumple

7.2.1. Método de Prueba

REQ-US-FC-01. La métrica compromete la aceptación de descripciones de hasta 15.000 caracteres en al menos español e inglés. El límite se verifica en los dos componentes que lo imponen. En la aplicación web, el área de texto acota la entrada en origen y el elemento que permite adjuntar ficheros rechaza con un mensaje localizado aquellos cuyo texto extraído excede el límite, comportamiento ilustrado en la [Figura 7.2](#) y cubierto por pruebas de interfaz, [Sección 7.1](#). En el servicio de orquestación, una validación previa del proceso multi-agente de la [Sección 5.3.1](#) rechaza las entradas excedidas en menos de un segundo, emitiendo un evento terminal con su código de error localizable, cubierto por una prueba de servicio dedicada. El carácter multilingüe queda verificado por las campañas de la [Tabla 7.1](#), pues la entrada liviana se procesa en inglés y la densa en español con detección automática de idioma, sumando 50 generaciones reales en ambos idiomas. El requisito se da por cumplido.



Figura 7.2. Mensaje de error por exceso de caracteres.

REQ-US-FC-02. La métrica requiere un evento de progreso por cada etapa de la generación, con una latencia máxima de un segundo entre la detección del fin de etapa en el servicio de orquestación y su reflejo visual en la interfaz web. La verificación instrumenta el recorrido completo sobre la aplicación completa en ejecución, cronometrando por un lado la llegada de cada frame SSE al navegador y por otro el instante en que el indicador de progreso pinta, en verde, la etapa como completada. El tramo de transporte resulta despreciable, pues los frames sin cómputo pendiente llegan al cliente con diferencias inapreciables entre sí, y el tramo de presentación queda acotado entre 1,7 y 20,5 milisegundos según la etapa, correspondiendo a la Tabla 7.3. La latencia total queda dos órdenes de magnitud por debajo del segundo comprometido. Completan la cobertura una prueba de servicio que revisa los eventos por etapa en el orden del proceso y una prueba de componentes que verifica el avance del indicador con cada evento recibido. El requisito se da por cumplido.

TABLA 7.3

LATENCIA DE EVENTOS DE PROGRESO

Etapas	Latencia
Taxonomista	2,1ms
Relacional	1,7ms
Poblador	20,5ms
Validador	19,1ms

REQ-US-FC-03. La métrica anuncia que todo grafo creado supera la validación sintáctica del esquema de datos y que, ante un resultado inválido, el sistema lance un ciclo de corrección, establecido en cuatro intentos en la Sección 5.3.1. La verificación recorre los tres niveles de la pirámide [43]. En la base, 13 pruebas unitarias fijan el comportamiento del validador determinista sobre la estructura, las referencias cruzadas y la conformidad de cada aserción con el dominio y rango declarados. En el nivel de servicio, 20 pruebas ejercitan la cadena multi-agente con inferencia simulada, cubriendo el enrutado del rein-tento hacia la etapa culpable, las reparaciones sin pérdidas, la poda como último recurso

y el fallo definitivo cuando la malformación es estructural, comportamientos diseñados en la Sección 5.3.1. En aceptación, las campañas de la Tabla 7.1 ejecutan el ciclo contra la aplicación real con especial elocuencia, pues la entrada liviana necesita un reintento de corrección, visible en la Figura 7.3, en cada una de sus 25 generaciones y aun así mantiene la puntuación perfecta, mientras que la densa agota el presupuesto de reintentos y degrada con poda en la totalidad de sus éxitos, entregando en todos los casos un grafo que supera la validación. El requisito se da por cumplido.

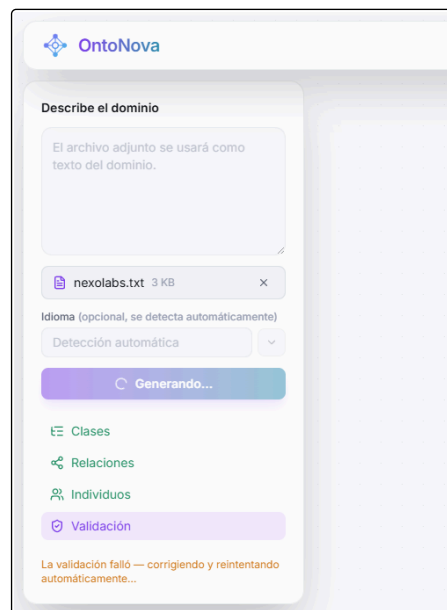


Figura 7.3. Mensaje de reintento de corrección.

REQ-US-FC-05. La métrica necesita que las ontologías exportadas superen un validador RDF sin errores. La verificación exporta dos ontologías reales —dominio liviano y dominio denso— en los dos formatos estándar del W3C que ofrece el sistema, *Turtle* y *RDF/XML* [11], y somete los ficheros resultantes a un analizador RDF del W3C [45] y *Turtle* [46]. Los documentos se analizan sin error alguno, veredicto disponible en la Figura 7.4. Completan la cobertura tres pruebas unitarias del compilador RDF y una prueba de extremo a extremo que verifica la descarga de una exportación *Turtle* desde el navegador. El requisito se da por cumplido.

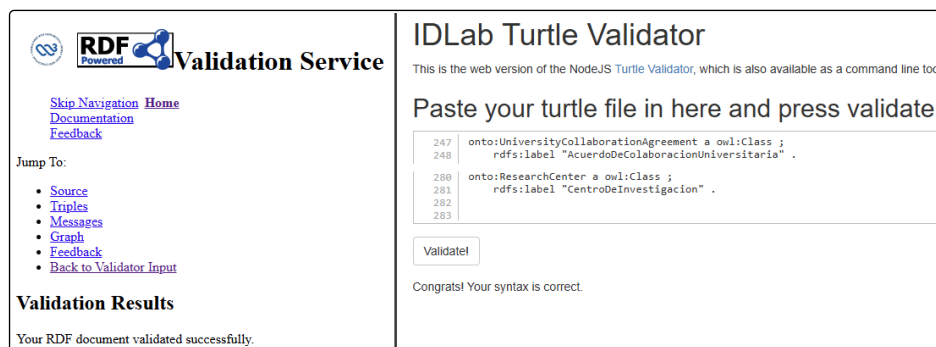


Figura 7.4. Validación sintáctica de exportaciones RDF y Turtle.

REQ-US-FC-10. La métrica impone la entrada estándar de ficheros de texto, *markdown* o PDF de hasta 5MB, rechazando con un mensaje informativo aquellos de mayor tamaño, véase [Figura 7.5](#), o cuyo texto extraído supere el límite de caracteres anunciado en el [REQ-US-FC-01](#). La verificación cubre el requisito en el servicio de orquestación y en la aplicación web, cubriendo los tres tipos de ficheros mencionados con pruebas de componente. Además, pruebas de extremo a extremo exploran dichos límites en un navegador real, y pruebas de aceptación someten a la aplicación completa a un PDF extenso, destapando problemas con el tipo *MIME* encontrados en la [Sección 6.3](#) y que solo un navegador real puede cubrir. El requisito se da por cumplido.

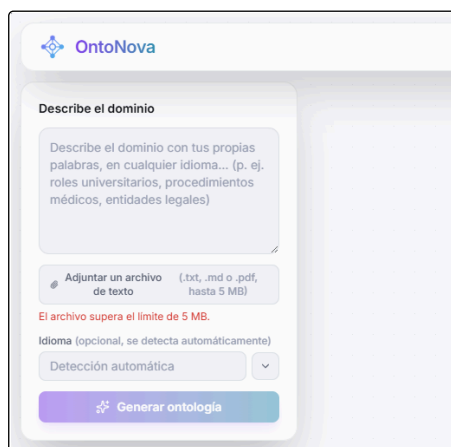


Figura 7.5. Mensaje de error por exceso de tamaño de archivo.

REQ-US-NF-02. La métrica limita la carga inicial de la aplicación web a 3 segundos y la respuesta a la interacción a 200 milisegundos. La carga se verifica cronometrando la aplicación web, servida por *Nginx*, en cien rondas de operaciones, resumidas en la [Tabla 7.4](#), sobre lienzo en el equipo de pruebas, [Sección 6.1](#), con el perfil objetivo de escritorio por su uso y naturaleza de despliegue, [Sección 5.5](#). El primer trazo útil de contenido, así como el instante interactivo, se alcanzan en 0,9 segundos con un rendimiento de 98 sobre 100; con el perfil móvil 4G simulado, que estrangula la red y la CPU, la carga asciende a 5 segundos, dato que señala el peso del extractor de documentos como margen de optimización, ilustrado en el informe de *Google Lighthouse* de la [Figura 7.6](#). El requisito se da por cumplido.

TABLA 7.4

LATENCIA DE INTERACCIÓN DEL LIENZO EN CIEN RONDAS POR OPERACIÓN

Operación	Mínimo	Media	Mediana	p95	p99	Máximo
Crear clase	49ms	51ms	51ms	53ms	64ms	65ms
Seleccionar clase	10ms	25ms	26ms	29ms	30ms	30ms
Eliminar clase	31ms	34ms	34ms	36ms	36ms	48ms

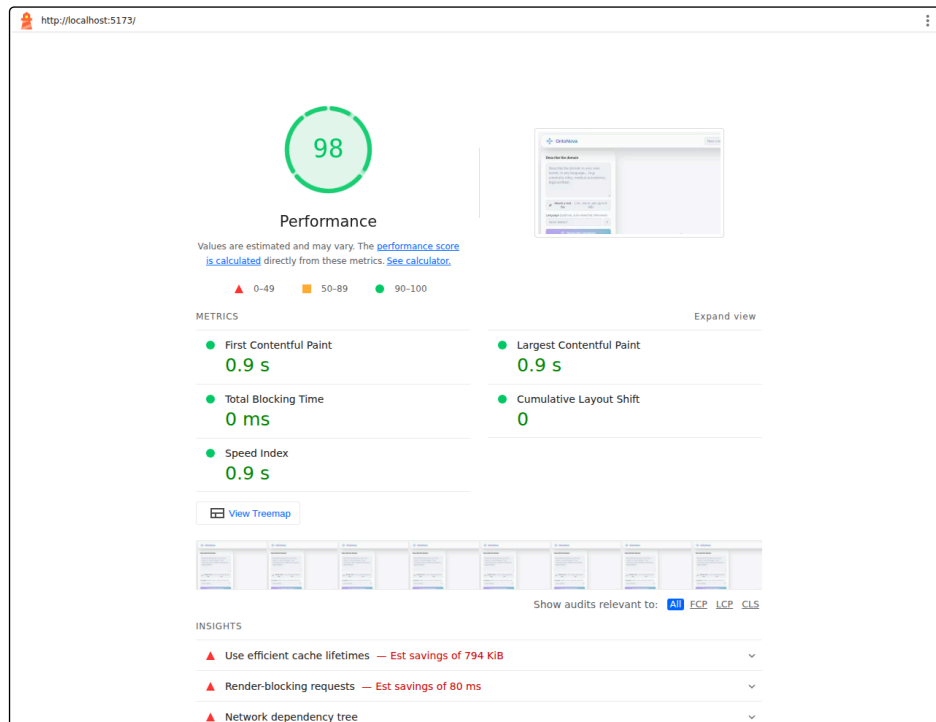


Figura 7.6. Informe de *Google Lighthouse* sobre OntoNova.

REQ-SW-NF-03. La métrica reclama que todo error capturado produzca una respuesta controlada al usuario sin pérdida del estado del lienzo. La verificación descansa sobre el invariante del evento terminal de la [Sección 5.3.3](#), cubierto por cuatro pruebas de servicio que fuerzan los caminos de fallo, y en el grafo en edición, inmune por construcción al residir en el almacén del navegador. Las campañas de la [Sección 7.1](#) ejercitan el control de errores en vivo, pues los tres fallos por truncamiento del dominio denso alcanzan el frontal como respuestas controladas con su código de error propio. El requisito se da por cumplido.

REQ-SW-NF-08. La métrica delimita a cinco minutos el despliegue completo del sistema. La verificación parte de un equipo de pruebas sin despliegue previo —eliminando contenedores y red— que cronometra la orden de arranque `docker compose up -d --wait` hasta que los tres servicios responden y superan sus comprobaciones de salud. La ejecución completa dura 2 minutos y 6 segundos, un 58% por debajo del umbral comprometido, siendo la carga del modelo en la memoria de la GPU por parte del motor de inferencia el proceso rezagado. Como condición de contorno, los pesos del modelo residen ya en el volumen persistente de la [Sección 5.5](#), descartando como evidencia la primera instalación absoluta por su dependencia del ancho de banda del equipo de pruebas. El requisito se da por cumplido.

7.2.2. Método de Demostración

REQ-US-FC-04. La métrica exige que cada operación de edición del lienzo se complete en tres interacciones como máximo. La demostración enumera las operaciones disponibles y cuenta las interacciones de la secuencia más corta que las completa, recogidas en la [Tabla 7.5](#), con el máximo alcanzado por el renombrado de una relación con tres interacciones y resolviendo el resto en una o dos. La demostración es reproducible, pues cada

operación está cubierta por una prueba de extremo a extremo que lanza sobre un navegador la secuencia contada, y el estado de cada cambio resultante es comprobado contra el servicio de orquestación manteniendo el grafo exportable en todo momento, [Sección 5.3.4](#). El requisito se da por cumplido.

TABLA 7.5

INTERACCIONES NECESARIAS POR OPERACIÓN DE EDICIÓN DEL LIENZO

Operación	Interacciones
Crear clase (nombre y botón)	2
Seleccionar e inspeccionar clase	1
Editar atributos desde el inspector	2
Eliminar clase	1
Crear relación (arrastre entre extremos)	1
Renombrar relación (doble clic, texto, intro)	3
Reconectar relación (arrastre del extremo)	1
Recolocar etiqueta de relación	1
Eliminar relación	1
Exportar (botón y formato)	2
Reiniciar lienzo (botón y confirmación)	2

REQ-SW-FC-02. La métrica asegura escalabilidad y mantenibilidad, comprometiendo al sistema a que la sustitución del modelo de lenguaje se realice únicamente con un cambio de configuración, sin modificar código. La demostración inspecciona los puntos de configuración y ejecuta el cambio. El nombre del modelo se declara en una variable de entorno, `LLM_MODEL_NAME`, consumida por el cliente de inferencia, y *vLLM* lo recibe como argumento de arranque en la composición. La demostración amplía horizontes respecto al requisito, pues cada nodo del proceso multi-agente declara su propio recurso dentro del motor de inferencia, ofreciendo la posibilidad futura de adoptar un [LLM](#) distinto por agente, posibilidad heredada del protocolo *OpenAI* adoptado en la [Sección 5.2.2](#) que hace sustituible cualquier motor compatible. En complemento, tres pruebas unitarias fijan el contrato del cliente con el protocolo. El requisito se da por cumplido.

REQ-SW-NF-05. La métrica demanda que un tercero pueda desplegar el sistema siguiendo la documentación del repositorio. La demostración reproduce el camino del tercero literalmente, pues la prueba del requisito [REQ-SW-NF-08](#) parte de un equipo sin despliegue previo y ejecuta el procedimiento documentado en la raíz del repositorio [31], que se reduce a los requisitos declarados en la [Sección 5.5](#) y una orden de arranque `docker compose up`. La aplicación queda operativa sin intervención adicional en el tiempo objetivo del requisito [REQ-SW-NF-08](#), dado que toda la configuración variable ofrece valores por defecto funcionales, [REQ-SW-FC-02](#). Adicionalmente, la prueba de aceptación del formato PDF se ejecuta contra un despliegue levantado exactamente con ese procedimiento, por lo que cada lanzamiento de pruebas demuestra el requisito. El requisito se da por cumplido.

7.2.3. Método de Análisis

REQ-US-NF-01. La métrica requiere una puntuación igual o superior a 70 en la [Escala de Usabilidad del Sistema](#) con 5 usuarios. El análisis administra el cuestionario, disponible en el [Apéndice A](#), a participantes ajenos al desarrollo y de perfiles diversos, recogidos en la [Tabla 7.6](#), con edades comprendidas entre los 18 y los 53 años y ninguno con conocimiento previo de las ontologías, quienes completan sin asistencia las tres tareas guía de la encuesta antes de valorar las diez afirmaciones. La corrección sigue el procedimiento del instrumento [38], una puntuación acotada entre 0 y 100 por participante formalizada en la [Ecuación \(7.1\)](#), donde r_i denota la respuesta a la afirmación i -ésima. Las puntuaciones individuales, declaradas en la [Tabla 7.6](#), generan una media de 76,50 con una desviación típica de 11,90, dispersión coherente con la variabilidad individual característica del instrumento, por encima del umbral comprometido y del promedio de referencia del instrumento, situado en 68. Resulta revelador que las puntuaciones altas, mayoritariamente, provengan de los perfiles sin formación técnica, alcanzando el máximo en la estudiante de artes en contraposición a la exigencia del ingeniero de *software*, en coherencia con el objetivo [O2](#). El requisito se da por cumplido.

$$SUS = 2,5 \cdot \left(\sum_{i \in \{1,3,5,7,9\}} (r_i - 1) + \sum_{i \in \{2,4,6,8,10\}} (5 - r_i) \right) \quad (7.1)$$

TABLA 7.6

RESULTADOS DEL CUESTIONARIO SUS POR PARTICIPANTE

Participante	Perfil	Edad	Puntuación SUS
U1	Ingeniería de software	38	67,50
U2	Jefatura de almacén	53	80,00
U3	Gerencia de grandes cuentas	52	75,00
U4	Estudios de artes	22	95,00
U5	Entrenamiento personal	20	65,00
Media $\pm \lambda$¹⁹			76,50 $\pm$ 11,90

REQ-SW-NF-02. La métrica reclama ausencia de vulnerabilidades de severidad alta en las dependencias del sistema. El análisis emplea herramientas de revisión estática de dependencias sobre ambos mundos, con `pip-audit` para el servicio de orquestación y `npm audit` para la aplicación web, contrastando el árbol de dependencias contra las bases de datos públicas de avisos de seguridad. La ejecución de ambos comandos afirma cero vulnerabilidades a día 23 de agosto de 2026. Cabe destacar que el método demuestra su valor durante la propia verificación, pues ejecuciones periódicas son requeridas ante la constante evolución de las amenazas, siempre lanzando la batería de pruebas automáticas de la [Sección 7.1](#) con el fin de evitar un cambio disruptivo en los procesos de negocio, definiendo dicho análisis como un proceso capaz de detectar y corregir. El requisito se da por cumplido.

¹⁹ λ = desviación típica.

7.2.4. Método de Inspección

REQ-SW-NF-06. La métrica establece que las nuevas funcionalidades o extensiones de las actuales se incorporen sin requerir una nueva fase de diseño de los componentes existentes, en aplicación del principio abierto/cerrado [16, Cap. 8]. La inspección examina los puntos de extensión del diseño y el coste de ejercitarlos. Añadir un formato de exportación se reduce a una entrada en el mapa de formatos del router de ontologías; un nuevo idioma de interfaz, a un fichero de recursos de localización; un nuevo código de error localizado, a una clave de traducción sobre la convención de eventos de la Sección 5.3.3; la sustitución del modelo, e incluso un modelo distinto por agente, a pura configuración, como demuestra REQ-SW-FC-02; y un agente adicional en la cadena, a un nodo con su esquema de salida acotado insertado en el grafo de la Sección 5.3.1, sin alterar los existentes. La inspección se contrasta también con dos requisitos tardíos analizados e incorporados durante el desarrollo, la entrada por fichero —un caso adicional del extractor de documentos, sin impactar al flujo de entrada estándar— y la localización de errores del servicio —una convención de código y parámetros sobre el transporte intacto—, en las que las pruebas automáticas de la Sección 7.1 pasaron sin adaptación alguna, señal de que los componentes cerrados permanecieron cerrados. En cuanto al aumento de usuarios concurrentes [16, Cap. 16], la inspección constata que ningún contenedor acumula estado de sesión, porque la aplicación web es contenido estático, en consecuencia, el estado de la ontología en edición reside en el propio navegador del usuario, el servicio de orquestación resuelve cada petición de forma autocontenida y el motor de inferencia atiende peticiones independientes mediante procesamiento por lotes continuo, Sección 2.3. El escalado se reduce a la replicación de contenedores, sin necesidad de una nueva fase de diseño, camino teorizado en la propuesta de exposición pública del Capítulo 8. El requisito se da por cumplido.

7.3. Verificación del Cumplimiento Normativo

El objetivo O4 enfrenta a la aplicación a la verificación de conformidad [5] del Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024 [4]. Este cumplimiento comienza su definición en la Sección 3.3.1 con la legislación aplicable, se somete a una fase de diseño en la Sección 5.4 y desvela las mitigaciones a seguir en la implementación, Capítulo 6. El veredicto, recogido en la Tabla B.1 del Apéndice B, declara al autor como un proveedor de un sistema de IA, OntoNova, puesto en servicio en la Unión Europea, ajeno a las prácticas prohibidas del Artículo 5 y a las categorías de alto riesgo de los Anexos I y III. Las obligaciones de transparencia del Artículo 50 no aplican porque el sistema no emula interacción humana y el grafo generado constituye un artefacto técnico para el usuario. Adicionalmente, se reconocen dos exclusiones de ámbito, por actividad de investigación y desarrollo y por publicarse bajo licencia de código abierto, Sección 3.3.3, restando como obligación la alfabetización en IA, Artículo 4, satisfecha en el contexto del proyecto por la formación que este mismo trabajo acredita. Cabe subrayar que ambas exclusiones están condicionadas a la ausencia de comercialización, por lo que una eventual exposición pública, Capítulo 8, demanda una nueva verificación bajo ese nuevo supuesto, delegada en un trabajo futuro, Sección 9.3. El objetivo O4 se da por verificado.

CAPÍTULO 8

PROPUESTA DE EXPOSICIÓN PÚBLICA

La propuesta de venta de la [Sección 3.2.4](#) plantea la transferencia a un tercero. Este capítulo teoriza el escenario que explota el sistema como servicio público, de un usuario a una multitud, recorriendo las áreas impactadas desde la [Sección 8.1](#) de arquitectura y la [Sección 8.2](#) de seguridad hasta el planteamiento de un despliegue automático, [Sección 8.3](#), en unión al endurecimiento de la legislación aplicable, [Sección 8.4](#).

8.1. Impacto en la Arquitectura del Sistema

La arquitectura existente recibe una afectación reducida gracias a la adopción directa del diseño propuesto, [Sección 5.2](#). El método de inspección de la [Sección 7.2.4](#) constata que ningún contenedor almacena estado de sesión, por lo que la transición de un usuario a una multitud se resuelve replicando los contenedores actuales tras un balanceador de carga, en concordancia con las modificaciones que anticipa la [Sección 5.5](#). El estado de la ontología en edición permanece en el navegador de cada usuario, lo que resuelve condiciones de carrera y evita la necesidad de sincronización entre réplicas. En cuanto al motor de inferencia, este absorbe la demanda por dos vías complementarias, la escalabilidad horizontal habilitada por el protocolo *OpenAI* y la adopción de un modelo de mayor capacidad, que a su vez aborda el margen de calidad sobre textos densos identificado en la [Sección 6.3](#).

Las novedades arquitectónicas provienen de los requisitos que el despliegue local difiere y que la exposición pública torna indispensables. La [Figura 8.1](#) actualiza el contexto del sistema bajo exposición pública. Por un lado, la identidad en presencia del registro de usuarios, [REQ-US-FC-09](#), y la autenticación y autorización, [REQ-SW-NF-07](#), introducen la primera pieza con estado del sistema, delegada en un proveedor de identidad externo y de código libre, *Keycloak*, que aprovecha el protocolo *OpenID Connect* para ofrecer lo que las métricas de dichos requisitos necesitan. Al introducir una nueva pieza, los contenedores existentes permanecen intactos en un nuevo ejercicio en favor del principio abierto/cerrado [16, Cap. 8] verificado en [REQ-SW-NF-06](#). Por otro lado, la trazabilidad de los casos de uso

que anticipa el requisito [REQ-SW-FC-01](#) en estado viable pero no incluido en la solución local toma fuerza al existir una multitud de ejecuciones que observar, sin embargo, su implantación resulta natural porque los eventos por etapa del diseño de la [Sección 5.3.3](#) ya existen, restando un servicio como *Grafana* que los persista por usuario para alimentar la mejora continua y la auditoría.

Descendiendo un nivel de abstracción, la [Figura 8.2](#) esboza una propuesta operacional de los contenedores en la nube. El balanceador de carga concentra la entrada cifrada de los usuarios y la reparte entre las réplicas de la aplicación web, que junto a las del servicio de orquestación habitan un espacio de nombres de un clúster de *Kubernetes*. Por otra parte, el motor de inferencia se aísla en un espacio de nombres propio con un grupo de nodos con *GPU* y autoescalado, cuya elección y operación desarrolla la [Sección 8.3](#), mientras que la identidad y el registro de trazabilidad se consumen como servicios gestionados, minimizando la superficie que el proveedor debe operar por sí mismo.

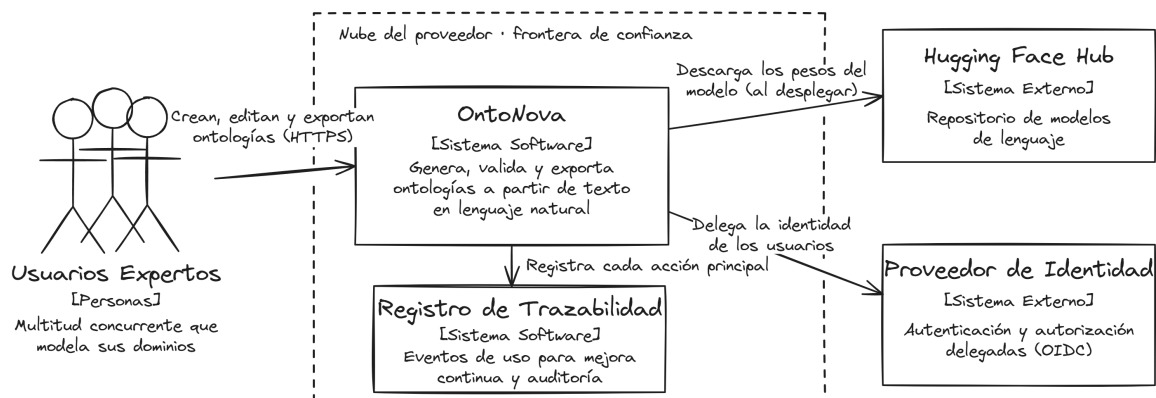


Figura 8.1. Diagrama C4 del contexto del sistema bajo exposición pública.

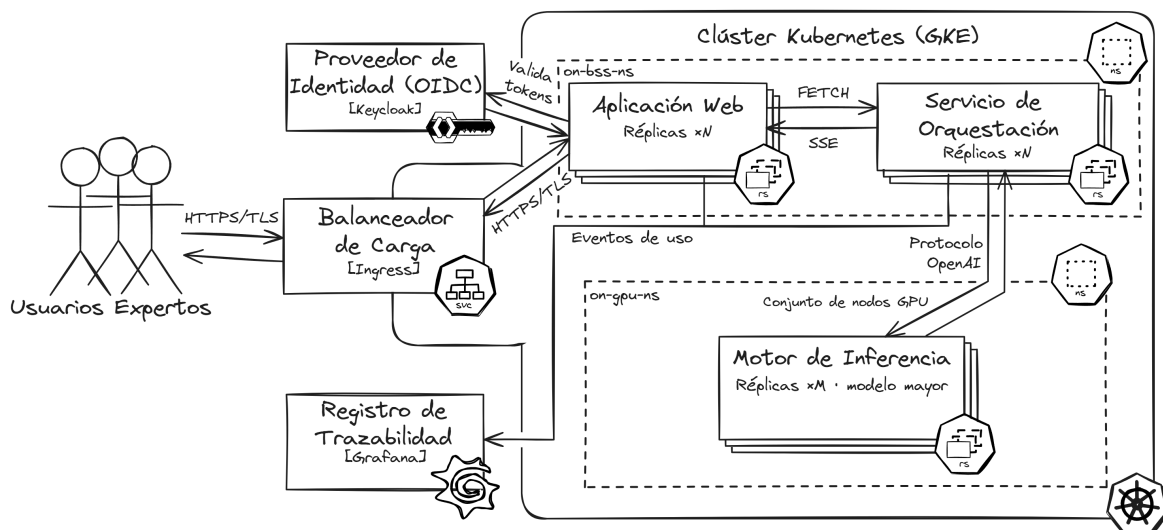


Figura 8.2. Diagrama operacional del sistema bajo exposición pública.

8.2. Impacto en el Diseño de Seguridad

La exposición pública convierte en indispensables los requisitos de seguridad que el despliegue local no implementa. El registro de usuarios, [REQ-US-FC-09](#), y la autenticación y autorización, [REQ-SW-NF-07](#), recaen sobre el proveedor de identidad introducido en la [Sección 8.1](#), de modo que cada petición atraviesa las puertas que ilustra la [Figura 8.3](#) acompañada de un testigo verificable por el proveedor y emitido por *OpenID Connect*, por lo tanto, los contenedores, [Sección 5.2.2](#), conservan su ausencia de estado. El servicio de trazabilidad adquiere una segunda función, ya que la misma bitácora que alimenta la mejora continua, [REQ-SW-FC-01](#), constituye el registro de auditoría que todo proceso operacional requiere para atribuir acciones a identidades.

El análisis de riesgos de la [Sección 5.4](#) se mantiene vigente pero la superficie de impacto se ensancha. Las mitigaciones estructurales contra la inyección de *prompts* [[29](#), [LLM01](#)] y la manipulación de la salida [[29](#), [LLM05](#)] —la decodificación guiada y el contrato validado de forma determinista— escalan sin cambios, dado que no dependen del número de usuarios sino de la salida. Sin embargo, un público anónimo y potencialmente hostil puede multiplicar los intentos, por lo que la limitación de tasa y las cuotas por identidad pasan de un estado conveniente a ser una nueva mitigación indispensable. Esta medida previene un consumo desbocado [[29](#), [LLM10](#)], un riesgo potencial en la nube e inaplicable en local, pues si cada petición consume tiempo de una GPU con un coste asociado y se produce un escenario de ataque, el abuso se traduce tanto en un daño económico al proveedor como en la pérdida de disponibilidad del servicio. La divulgación de información sensible [[29](#), [LLM02](#)] también se reevalúa, porque el motor de inferencia pasa a atender peticiones entrelazadas de usuarios distintos con el procesamiento por lotes continuo, obligando a garantizar el aislamiento entre peticiones y la ausencia de persistencia de los textos en *vLLM*.

Completa el impacto el endurecimiento del perímetro. El cifrado de transporte [HTTP](#) se extiende de extremo a extremo desde el balanceador que la [Sección 8.1](#) expone, seguido de la configuración sensible que abandona los ficheros de entorno `.env` en favor de un gestor de secretos de código abierto como *CyberArk Conjur*, y el punto de entrada incorpora protección perimetral frente al tráfico malicioso. El desplazamiento profundo es no obstante el de la privacidad, pues el texto del dominio abandona el equipo del experto para entrar en la frontera de confianza de la [Sección 8.1](#) del proveedor, giro que desactiva el argumento central de privacidad del diseño original, [Sección 5.4](#), y traslada la protección del dato del plano arquitectónico al plano normativo que examina la [Sección 8.4](#).

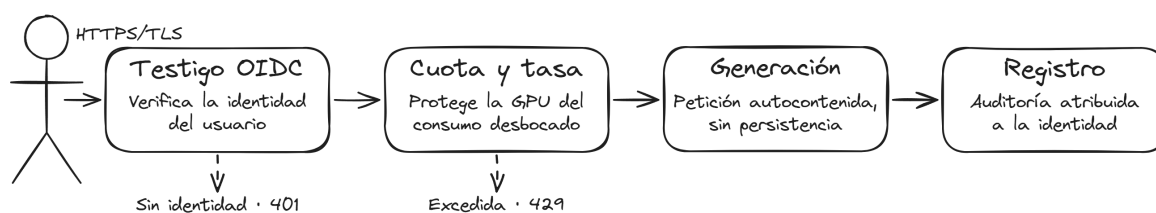


Figura 8.3. Puertas de seguridad que atraviesa cada petición en la exposición pública.

8.3. Impacto en el Diseño del Despliegue

El despliegue es el área donde la propuesta introduce más piezas nuevas, aunque ninguna altera los artefactos existentes. La composición local de la [Sección 5.5](#) se traslada a *Kubernetes* con una correspondencia directa, pues cada contenedor deviene un despliegue replicable con su servicio, el volumen de pesos del modelo se convierte en un volumen persistente del clúster y la vista de despliegue actualizada la anticipa la [Sección 8.1](#), donde los nodos pasan del equipo doméstico a un clúster gestionado. La elección de *Google Kubernetes Engine* responde a tres motivos, la condición de estándar libre *de facto* de *Kubernetes* para orquestar contenedores, el soporte nativo de grupos de nodos con GPU dotados de autoescalado²⁰ y el encaje natural en el marco universitario del proyecto, beneficiario del ecosistema académico de *Google*.

El plantillado con *Helm* reduce la complejidad operativa y fomenta la reutilización comprometida por el requisito [REQ-SW-NF-08](#). Un *chart* agrupa tres *subcharts* —aplicación web, servicio de orquestación y motor de inferencia— cuyos valores por entorno separan lo que cambia entre escenarios de lo que permanece, por lo que el mismo empaquetado sirve al clúster público y a un despliegue local mínimo, preservando la soberanía tecnológica del usuario doméstico que defiende la [Sección 3.3.4](#) como un caso particular del mismo artefacto, [Figura 8.4](#).

<pre> 1 # values-local.yaml 2 web: 3 replicas: 1 4 orquestacion: 5 replicas: 1 6 inferencia: 7 replicas: 1 8 modelo: Qwen3-14B-AWQ 9 gpu: rtx-4090 10 identidad: 11 habilitada: false </pre>	<pre> 1 # values-gke.yaml 2 web: 3 replicas: 3 4 orquestacion: 5 replicas: 3 6 inferencia: 7 replicas: 4 8 modelo: Qwen3-235B-AWQ 9 nodePool: gpu-h100 10 identidad: 11 habilitada: true </pre>
--	---

Figura 8.4. Valores del *chart* por escenario, con el empaquetado común.

La automatización se materializa en un flujo de integración y entrega continuas sobre *GitHub Actions*, ilustrado en la [Figura 8.5](#), cuyas etapas institucionalizan la pirámide de la [Sección 7.1](#), convirtiendo cada nivel en una etapa de calidad. Con cada contribución al código fuente, se ejecutan el análisis estático y las pruebas unitarias y de servicio, se construyen las imágenes con el escaneo de dependencias que verifica [REQ-SW-NF-02](#) extendido a las imágenes, y se validan los casos de uso con las pruebas de navegador sobre una composición efímera. Las pruebas de aceptación exigen una GPU dedicada y minutos por generación, por lo que se ejecutan de forma nocturna y como requisito previo a toda entrega, trasladando al proceso automatizado las campañas estadísticas del [Capítulo 7](#).

²⁰ *Google Kubernetes Engine* permite el escalado a cero réplicas, optimizando el coste de aprovisionamiento que acota la propuesta.

Superadas las fases, la entrega lanza `helm upgrade` sobre el entorno de preproducción y promociona a producción con aprobación manual mediante un despliegue progresivo, viable sin riesgo de incoherencia de estado de la [Sección 7.2.4](#) porque las réplicas son intercambiables. Cierra la operación la observabilidad, dado que los eventos por etapa de la [Sección 5.3.3](#) y su identidad se plasman en el registro de trazabilidad de la [Sección 8.1](#), que sin instrumentación adicional ofrece las métricas de latencia y uso que el autoescalado de *Kubernetes* y la mejora continua consumen.

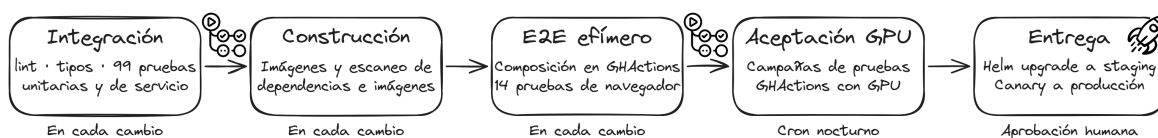


Figura 8.5. Ciclo de integración continua y despliegue continuo.

8.4. Impacto en la Legislación Aplicable

El marco legislativo de la [Sección 3.3.1](#) permanece vigente pero sufre un endurecimiento en dos frentes. El primero es la protección de datos, ya que el argumento que sostiene el cumplimiento del RGPD [3] en la solución local —el texto del dominio nunca abandona el equipo del experto— queda desactivado por el desplazamiento de la frontera de confianza descrito en la [Sección 8.2](#), asumiendo el proveedor de OntoNova la condición de responsable del tratamiento sobre los textos, grafos y datos de cuenta de sus usuarios. Por ende, el proveedor de infraestructura se instrumenta contractualmente como mero encargado en su adenda de tratamiento de datos [47], sin asumir responsabilidad alguna sobre los fines. De esa condición se derivan las obligaciones plenas del responsable, desde la base jurídica del RGPD [3] y la política de retención hasta la selección de regiones europeas de despliegue que evite transferencias internacionales de datos. El segundo impacto queda recogido en el AI Act [4], dado que la verificación del mismo, en cumplimiento del objetivo O4, [Sección 7.3](#), recoge exclusiones como la investigación y desarrollo, y código abierto que se extinguen con la publicación, restaurando las obligaciones ordinarias de proveedor. El cuestionario de conformidad del [Apéndice B](#) plantea una reevaluación bajo el nuevo supuesto, representando el primer entregable de esta propuesta futura, tal y como anticipa la [Sección 7.3](#), con especial atención a las obligaciones de transparencia del Artículo 50.

En términos de licenciamiento, la elección de la [Sección 3.3.3](#) se reexamina ante el cambio de escenario de servicio. La incertidumbre radica en si la licencia del código fuente EUPL-1.2 [30] debe ceder ante una licencia orientada a la nube como la [Licencia Pública General de Affero \(AGPL-3\)](#), cuyo rasgo distintivo es extender el *copyleft* a quien presta el *software* como servicio sin obligar a distribuirlo. La EUPL-1.2, en compatibilidad declarada con AGPL-3, también incorpora ese alcance, definiendo la distribución o comunicación como la puesta a disposición en línea de funcionalidades esenciales a otra persona física o jurídica. Cabe resaltar el compromiso del licenciataria de facilitar al público la información mínima reclamada por la legislación aplicable, siendo indispensable en un trabajo futuro, [Sección 9.3](#), un aviso legal y una declaración de política de privacidad dentro de OntoNova. En consecuencia, se mantiene la [Licencia Pública de la Unión Europea](#).

CAPÍTULO 9

CONCLUSIONES

Este capítulo relata las conclusiones del proyecto que ofrecen los objetivos propuestos, [Sección 9.1](#), así como el punto de vista personal del autor, [Sección 9.2](#), revelando los próximos pasos y delegando requisitos y pinceladas funcionales en trabajos futuros, [Sección 9.3](#).

9.1. Conclusiones del Proyecto

El documento ha recorrido el estado del arte, el plan de proyecto, el análisis, el diseño, la implementación y la verificación de una aplicación web de gestión del conocimiento multilingüe, culminando con una propuesta teórica de exposición pública, todo ello al amparo de estándares y marcos internacionales a destacar como ISO/IEC/IEEE 29148 [\[35\]](#), ISO/IEC/IEEE 15288 [\[36\]](#), el Modelo C4 [\[41\]](#) y el OWASP Top 10 GenIA/LLM [\[29\]](#). La [Sección 1.2](#) anuncia el propósito principal consistente en diseñar y desarrollar un sistema del conocimiento multilingüe basado en LLM que actúe como alternativa accesible y de código abierto a las herramientas tradicionales. OntoNova cumple dicho propósito, con los requisitos incluidos y verificados en la [Sección 7.2](#), pruebas automáticas superadas, [Sección 7.1](#), y una validación con usuarios reales por encima del umbral de usabilidad comprometido, [Sección 7.2.3](#).

Adicionalmente, todos los objetivos secundarios han sido cumplidos:

- **O1:** El aprendizaje alcanzado supera el nivel tres exigido de la taxonomía de Bloom [\[7\]](#), porque el conocimiento adquirido no solo se aplica, la decodificación guiada estudiada en la [Sección 2.3](#) queda convertida en el proceso multi-agente de la [Sección 5.3.1](#), sino que también se analiza y evalúa, como demuestran el diagnóstico de comportamientos del motor de inferencia y de su ausencia de determinismo entre sesiones, [Sección 6.3](#), y las campañas estadísticas de calidad diseñadas a medida para caracterizarlo, [Sección 7.1](#).
- **O2:** La barrera de entrada disminuye de manera medible. La escala de usabilidad arroja una media de 76,5 sobre el umbral comprometido de 70, [Sección 7.2.3](#), con el matiz

revelador de que las puntuaciones elevadas provienen de participantes sin formación técnica ni conocimiento previo de las ontologías. Cabe resaltar que cada operación de edición se completa en tres interacciones o menos, [Sección 7.2.2](#), y la generación parte de texto libre en el idioma del experto, disolviendo el problema del lienzo en blanco para los ingenieros de conocimiento que motiva el proyecto, [Sección 1.1](#).

- **O3:** El carácter multilingüe queda verificado con [REQ-US-FC-01](#) acompañado de generaciones reales repartidas entre inglés y español con detección automática del idioma, [Sección 7.1](#), extendiéndose a la interfaz y a los errores del servicio, traducidos ambos al idioma del usuario, [Sección 5.3.3](#).
- **O4:** El cumplimiento del marco regulatorio europeo vigente [2] (p. ej., [RGPD](#) [3] y [AI Act](#) [4]) dispone de una verificación dedicada, [Sección 7.3](#), con el veredicto favorable del verificador de conformidad reproducido en el [Apéndice B](#), la privacidad garantizada por diseño al no abandonar los datos el equipo del experto, [Sección 5.4](#), y el consiguiente impacto normativo en el escenario público, [Sección 8.4](#).
- **O5:** La arquitectura se diseña y documenta con el Modelo C4 [41] en sus tres niveles, [Sección 5.2](#), incorpora las mitigaciones de seguridad frente a los riesgos de las aplicaciones que hacen uso de [LLM](#), [Sección 5.4](#), y aplica patrones de diseño [8] como el principio de responsabilidad única por agente [16, [Cap. 7](#)] y el principio abierto/cerrado [16, [Cap. 8](#)], verificado este último por inspección junto a la escalabilidad sin rediseño que otorga la ausencia de estado, [Sección 7.2.4](#), proyectada después sobre la nube pública en el [Capítulo 8](#).
- **O6:** La alineación con los Objetivos de Desarrollo Sostenible, [Sección 3.3.4](#), queda revelada con el desarrollo y validación del sistema completo sobre *hardware* doméstico, [Sección 6.1](#), preservando la soberanía tecnológica y demostrando una barrera de entrada alcanzable [9, [ODS 10](#)]. En segundo lugar, la democratización perseguida está evidenciada por los perfiles diversos que lo utilizaron con éxito, [Sección 7.2.3](#). En tercer lugar, el aprendizaje adquirido en el proyecto [9, [ODS 4](#)], director de orquesta de principio a fin, [Sección 3.1.1](#), queda ligado al objetivo [O1](#). En última instancia, el marco de innovación [9, [ODS 9](#)] engloba el proyecto de OntoNova por su unicidad frente a otros competidores semánticos, [Sección 2.6](#), un sistema que espera una patente de dicho ámbito y futuras mejoras en cuanto se ponga a disposición de la comunidad FOSS [23].

El proceso de aprendizaje ha marcado cada ciclo de imprecisión-decisión de diseño, [Sección 5.1.2](#), e implementación, [Sección 6.3](#), con el propósito de ofrecer al autor una retroalimentación y nuevos conocimientos, [Sección 9.2](#).

9.2. Conclusiones Personales

Reforzando la idea introducida en el [Capítulo 1](#) de introducción, el desarrollo de este proyecto me ha permitido tener la oportunidad de exclamar al mundo qué conocimientos he aprendido a lo largo de la carrera, el máster y la trayectoria profesional, y unirlos en un nuevo aprendizaje para formar, en última instancia, una solución que pueda facilitar la construcción multilingüe de conocimiento a los verdaderos expertos del dominio. Especialmente, me entusiasma haber podido llevar a cabo un proyecto que me ha involucrado en todas y cada una de sus fases, desde el análisis inicial hasta la implementación final. Se trata de una práctica que apenas estoy comenzando a ejercitar en el ámbito laboral, y poder ser partícipe de un ciclo de desarrollo completo me ha servido para reforzar los conocimientos y la confianza necesarios para retomar mi carrera profesional con más fuerza que nunca.

Asimismo, no puedo evitar sentirme afortunado de haber cursado la carrera en ausencia de la [Inteligencia Artificial](#). Cuando comencé la presente titulación de máster en 2024 ya se apreciaban los primeros coletazos de la IA generativa y agéntica, la cual ha cambiado el mundo drásticamente —para bien— en 2026. Sin embargo, siento que, si tuviera que repetir las titulaciones hoy en día, me perdería muchas experiencias y aprendizajes que he podido vivir precisamente gracias a su ausencia.

Asignaturas como Calidad de Software y Sistemas, Sistemas de Ciberseguridad, Software para Internet de las cosas y Aplicaciones Avanzadas de la Inteligencia Artificial han fortalecido la base de mi conocimiento actual y han servido de inspiración para seguir mi travesía por el mundo digital con el mismo objetivo de siempre: «Estudiar y ser partícipe de todas las ramas de la informática que el mundo pueda ofrecer».

9.3. Trabajos Futuros

Los capítulos precedentes señalan, allí donde surgen, las líneas que exceden los objetivos comprometidos, [Sección 1.2](#), y los requisitos incluidos, [Sección 4.2](#). La consolidación se presenta en tres planos, el funcional que heredan los requisitos no incluidos, el de la calidad de generación y el normativo-operativo, cerrando con el trabajo propuesto.

En el plano funcional, los requisitos en estado viable o pendientes de una nueva fase de diseño trazan la evolución natural del producto:

- **Persistencia y colaboración.** La incorporación del contenedor de persistencia, anotada en la [Sección 5.2.2](#), habilita la edición simultánea con convergencia del estado compartido, [REQ-US-FC-06](#), y el histórico de versiones con reversión y auditoría de cambios, [REQ-US-FC-07](#).
- **Importación de ontologías.** El camino inverso a la exportación verificada en [REQ-US-FC-05](#), [Sección 7.2.1](#), es decir, cargar ontologías existentes en formatos [W3C](#) sin pérdida de tripletas, [REQ-US-FC-08](#), reclama una fase de diseño que reconcilie grafos externos con el contrato de la [Sección 5.3.2](#).
- **Expresividad del contrato.** La ampliación gradual del perfil de [OWL](#) con las restricciones de cardinalidad, las clases anónimas y los axiomas de equivalencia, anotada en la [Sección 5.3.2](#), sin coste estructural gracias al campo de metadatos.

- **Accesibilidad, soporte y cobertura.** La conformidad de nivel AA con las [Directrices de Accesibilidad para el Contenido Web \[39\]](#), [REQ-SW-NF-01](#), el sistema de soporte con su plan de mantenimiento, [REQ-US-NF-03](#), y la superación del umbral de cobertura de las pruebas automáticas, [REQ-SW-NF-04](#), atributos que un prototipo verificado aún no reclama.

En el plano de la calidad de generación, las campañas de la [Sección 7.1](#) y las dificultades de la [Sección 6.3](#) acotan empíricamente dónde invertir:

- **Refinamiento de instrucciones.** Las categorías débiles del dominio denso —la jerarquía y las aserciones entre individuos— señalan el objetivo del refinamiento delegado por la [Sección 7.1](#), con las métricas clásicas como vara de medir cada iteración.
- **Modelos de mayor capacidad.** La brecha entre la puntuación perfecta del dominio liviano y el 0,41 de F1 del denso apunta a la capacidad del modelo como techo, hipótesis contrastable en el grupo de nodos de inferencia que propone la [Sección 8.3](#) y evaluable bajo las pruebas declaradas en la [Sección 7.1](#).
- **Conectividad y documentos extensos.** El suelo de calidad ensayado y descartado con datos en la [Sección 6.3](#) requiere una teoría alternativa para las clases huérfanas, y el límite de entrada de [REQ-US-FC-01](#) invita a una estrategia de división y consolidación que procese documentos más allá de los 15.000 caracteres.

En el plano normativo-operativo, la exposición pública dispone del [Capítulo 8](#) al completo, donde la identidad y la trazabilidad reciben su encaje arquitectónico, adelantando la fase de diseño de los requisitos [REQ-US-FC-09](#), [REQ-SW-NF-07](#) y [REQ-SW-FC-01](#). El despliegue automatizado se detalla etapa a etapa y el impacto legal se analiza, dejando su implementación a un único paso de distancia del presente trabajo. Asimismo, la exposición pública activa obligaciones que son latentes en la solución local:

- **Reevaluación de conformidad.** La caída de las exclusiones del [Reglamento \(UE\) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024](#) al publicarse, comprometida en la [Sección 7.3](#) y analizada en la [Sección 8.4](#), convierte la repetición del cuestionario del [Apéndice B](#) en el primer entregable de una futura explotación.
- **Información pública obligatoria.** El aviso legal y la declaración de política de privacidad dentro de OntoNova, exigidos por el compromiso de información mínima de la licencia [EUPL-1.2](#) señalado en la [Sección 8.4](#).

BIBLIOGRAFÍA

- [1] J. Manchanda, L. Boettcher, M. Westphalen, y J. Jasser, «The Open Source Advantage in Large Language Models (LLMs)», *arXiv preprint arXiv:2412.12004*, 2024, doi: 10.48550/arXiv.2412.12004.
- [2] I. Barberá, «AI Privacy Risks & Mitigations – Large Language Models (LLMs)», European Data Protection Board (EDPB), Support Pool of Experts, abr. 2025. [En línea]. Disponible en: <https://www.edpb.europa.eu/system/files/2025-04/ai-privacy-risks-and-mitigations-in-llms.pdf>
- [3] «Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo (Reglamento General de Protección de Datos)», *Diario Oficial de la Unión Europea*, vol. L 119, abr. 2016.
- [4] «Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024», *Diario Oficial de la Unión Europea*, jul. 2024.
- [5] Future of Life Institute, «AI Act Compliance Checker Flowchart», Versión 1.0, jul. 2025. Accedido: 29 de agosto de 2026. [En línea]. Disponible en: https://artificialintelligenceact.eu/wp-content/uploads/2025/07/AI-Act-Compliance-Checker-Flowchart-v1.0_compressed.pdf
- [6] S. Berkun, *The Art of Project Management*, 1.^a ed. O'Reilly Media, 2005.
- [7] M. Forehand, «Bloom's Taxonomy Original and Revised», en *Emerging Perspectives on Learning, Teaching, and Technology*, M. Orey, Ed., University of Georgia, 2010.
- [8] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, 1.^a ed. Addison-Wesley Professional, 1994.
- [9] Naciones Unidas, «Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible», 2015. Accedido: 9 de diciembre de 2025. [En línea]. Disponible en: <https://undocs.org/es/A/RES/70/1>
- [10] T. Berners-Lee, J. Hendler, y O. Lassila, «The Semantic Web: A New Form of Web Content That is Meaningful to Computers Will Unleash a Revolution of New Possibilities», *Scientific American*, vol. 284, n.º 5, pp. 34-43, 2001, doi: 10.1038/scientificamerican0501-34.
- [11] W3C, «OWL - Semantic Web Standards». Accedido: 15 de octubre de 2025. [En línea]. Disponible en: <https://www.w3.org/OWL/>
- [12] R. Hoekstra, «The Knowledge Reengineering Bottleneck», *Semantic Web*, vol. 1, n.º 1-2, pp. 21-27, 2010.

- [13] A. Vaswani *et al.*, «Attention Is All You Need», en *Advances in Neural Information Processing Systems* 30, 2017. doi: [10.48550/arXiv.1706.03762](https://doi.org/10.48550/arXiv.1706.03762).
- [14] L. Huang *et al.*, «A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions», *ACM Transactions on Information Systems*, vol. 43, n.º 2, pp. 1-55, 2025.
- [15] M. X. Liu *et al.*, «"We Need Structured Output": Towards User-centered Constraints on Large Language Model Output», en *Proceedings of the CHI Conference on Human Factors in Computing Systems*, New York, NY, USA: Association for Computing Machinery, 2024, pp. 1-17. doi: [10.1145/3613905.3650756](https://doi.org/10.1145/3613905.3650756).
- [16] R. C. Martin, *Arquitectura Limpia: Guía para especialistas en la estructura y el diseño de software*. Anaya Multimedia, 2018.
- [17] S. Yao *et al.*, «ReAct: Synergizing Reasoning and Acting in Language Models», en *International Conference on Learning Representations*, 2023. doi: [10.48550/arXiv.2210.03629](https://doi.org/10.48550/arXiv.2210.03629).
- [18] LangChain, «LangGraph: Build Stateful, Multi-Actor Applications with LLMs». Accedido: 10 de febrero de 2026. [En línea]. Disponible en: <https://langchain-ai.github.io/langgraph/>
- [19] M. Kleppmann, *Designing Data-Intensive Applications: Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.
- [20] P. Cimiano y J. Völker, «Text2Onto», en *Natural Language Processing and Information Systems (NLDB)*, vol. 3513, en LNCS, vol. 3513., 2005, pp. 227-238. doi: [10.1007/11428817_21](https://doi.org/10.1007/11428817_21).
- [21] J. H. Caufield *et al.*, «Structured Prompt Interrogation and Recursive Extraction of Semantics (SPIRES): a method for populating knowledge bases using zero-shot learning», *Bioinformatics*, 2024, doi: [10.1093/bioinformatics/btae104](https://doi.org/10.1093/bioinformatics/btae104).
- [22] «Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales», *Boletín Oficial del Estado*, vol. 294, dic. 2018.
- [23] Iniciativa para el Código Abierto, «La Definición de Código Abierto». Accedido: 22 de diciembre de 2025. [En línea]. Disponible en: <https://opensource.org/osd/>
- [24] «Tecnología de la información — Sintaxis de intercambio de datos JSON», n.º ISO/IEC 21778:2017. Organización Internacional de Normalización, Geneva, Suiza, noviembre de 2017.
- [25] «Tecnología de la información — Ontologías de nivel superior (TLO)», n.º ISO/IEC 21838:2021. Organización Internacional de Normalización, Geneva, Suiza, 2021.
- [26] «Seguridad de la información, ciberseguridad y protección de la privacidad — Sistemas de gestión de la seguridad de la información — Requisitos», n.º ISO/IEC 27001:2022. Organización Internacional de Normalización, Geneva, Suiza, octubre de 2022.

- [27] «Tecnología de la información — Inteligencia artificial — Sistema de gestión», n.º ISO/IEC 42001:2023. Organización Internacional de Normalización, Geneva, Suiza, diciembre de 2023.
- [28] «Ingeniería de sistemas y de software — Requisitos y evaluación de la calidad de sistemas y software (SQuaRE) — Visión general y uso del modelo de calidad», n.º ISO/IEC 25002:2024. Organización Internacional de Normalización, Geneva, Suiza, marzo de 2024.
- [29] «Top 10 Riesgos y Medidas de Mitigación para LLMs y Aplicaciones de IA Generativa». Proyecto abierto de seguridad de aplicaciones a nivel mundial (OWASP), Delaware, Estados Unidos, noviembre de 2024.
- [30] «Decisión de Ejecución (UE) 2017/863 de la Comisión, por la que se actualiza la licencia de software de código abierto EUPL (European Union Public Licence v. 1.2)», *Diario Oficial de la Unión Europea*, vol. L 128, may 2017.
- [31] D. Picazo García, «ontonova-project». Accedido: 2 de septiembre de 2026. [En línea]. Disponible en: <https://github.com/gu4re/ontonova.project>
- [32] K. Wiegers y J. Beatty, *Software Requirements (Developer Best Practices)*. Microsoft Press, 2013.
- [33] J. Patton, *User Story Mapping: Discover the Whole Story, Build the Right Product*. O'Reilly Media, 2014.
- [34] «IEEE Prácticas Recomendadas para la Especificación de Requisitos de Software», n.º IEEE 830–1998. IEEE, Piscataway, Nueva Jersey, Estados Unidos, junio de 1998.
- [35] «Ingeniería de sistemas y de software — Procesos del ciclo de vida — Ingeniería de requisitos», n.º ISO/IEC/IEEE 29148:2018. Organización Internacional de Normalización, Geneva, Suiza, noviembre de 2018.
- [36] «Ingeniería de sistemas y de software — Procesos del ciclo de vida del sistema», n.º ISO/IEC/IEEE 15288:2023. Organización Internacional de Normalización, Geneva, Suiza, mayo de 2023.
- [37] «Guía para la escritura de requisitos V4», *INCOSE*, vol. INCOSE-TP-2010-006-04, jun. 2023.
- [38] J. Brooke, «SUS: A 'Quick and Dirty' Usability Scale», en *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, y I. L. McClelland, Eds., London: Taylor & Francis, 1996, pp. 189-194.
- [39] Iniciativa de Accesibilidad Web, «Directrices de Accesibilidad para el Contenido Web». Accedido: 25 de noviembre de 2025. [En línea]. Disponible en: <https://www.w3.org/WAI/standards-guidelines/wcag/es>
- [40] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley Professional, 2003.
- [41] S. Brown, *The C4 Model for Visualising Software Architecture*. O'Reilly Media, 2026.

- [42] B. Ay y Y. E. Demirdağ, «Benchmarking Ollama and vLLM for Concurrent LLM Serving: A Multi-Scenario Evaluation of Performance and Scalability», *Applied Sciences*, vol. 16, n.º 11, 2026, doi: [10.3390/app16115435](https://doi.org/10.3390/app16115435).
- [43] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, 2009.
- [44] L. Smith, «Shift-left testing», *Dr. Dobbs's J.*, vol. 26, n.º 9, p. 56–ff, sep. 2001.
- [45] W3C, «RDF Validation Service». Accedido: 24 de agosto de 2026. [En línea]. Disponible en: <https://www.w3.org/RDF/Validator/>
- [46] IDLab - Ghent University, «Turtle Validator». Accedido: 24 de agosto de 2026. [En línea]. Disponible en: <https://github.com/MMLab/TurtleValidator>
- [47] Google Cloud, «Cloud Data Processing Addendum». Accedido: 30 de agosto de 2026. [En línea]. Disponible en: <https://cloud.google.com/terms/data-processing-addendum>

GLOSARIO

AGPL-3 – Licencia Pública General de Affero: Licencia *copyleft* derivada de la Licencia Pública General de GNU diseñada específicamente para asegurar la cooperación con la comunidad en el caso de *software* ejecutado en servidores de red.

IA – Inteligencia Artificial: Disciplina científica que se ocupa de crear programas informáticos que ejecutan operaciones comparables a las que realiza la mente humana, como el aprendizaje o el razonamiento lógico.

AI Act – Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024: Reglamento que mejora el funcionamiento del mercado interior mediante el establecimiento de un marco jurídico uniforme, en particular para el desarrollo, la introducción en el mercado, la puesta en servicio y la utilización de sistemas de inteligencia artificial.

API – Interfaz de Programación de Aplicaciones: Conjunto de reglas y protocolos que permiten a diferentes aplicaciones comunicarse entre sí.

BFO – Ontología Formal Básica: Ontología de nivel superior que sirve como un marco de referencia estandarizado para apoyar la integración, recuperación y análisis de datos científicos e información entre diferentes disciplinas.

CUDA – Arquitectura Unificada de Dispositivos de Cómputo: Modelo de programación y conjunto de instrucciones desarrollado por NVIDIA para el procesamiento paralelo en las GPU.

EUPL-1.2 – Licencia Pública de la Unión Europea: Licencia de *software* libre y *copyleft* creada por la Unión Europea concebida para la liberación de programas de las administraciones públicas, las empresas y los ciudadanos.

FOSS – Software Libre y de Código Abierto: Programa informático cuyo código fuente es público y accesible, permitiendo a cualquier usuario ejecutarlo, estudiarlo, modificarlo y distribuirlo libremente.

GBNF – Gramática Backus-Naur para GGML: Formato de gramática utilizado para definir la estructura que debe seguir la respuesta de un modelo de lenguaje en la generación de texto.

RGPD – Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016: Reglamento del derecho de la Unión Europea sobre protección de datos y privacidad, aplicable en la UE y el Espacio Económico Europeo.

GPU – Unidad de Procesamiento Gráfico: Componente *hardware* especializado en la renderización de imágenes, gráficos y en el cómputo paralelo masivo.

HTTP – Protocolo de Transferencia de Hipertexto: Protocolo de comunicación que permite la transferencia de información y archivos en la World Wide Web.

IEEE – Instituto de Ingenieros Eléctricos y Electrónicos: Organización profesional dedicada a promover el desarrollo y aplicación de tecnologías de la información y la comunicación.

INCOSE – Organización Internacional de Ingeniería en Sistemas: Organización profesional dedicada a promover el desarrollo y aplicación de metodologías y prácticas en ingeniería de sistemas.

JSON – Notación de Objetos de JavaScript: Formato de texto sencillo para el intercambio de datos.

LLM – Modelos de Lenguaje a Gran Escala: Modelos de lenguaje entrenados con grandes cantidades de datos y parámetros, capaces de generar texto coherente y realizar tareas de procesamiento del lenguaje natural.

LOPDGDD – Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales: Adapta el ordenamiento jurídico español al Reglamento General de Protección de Datos (RGPD) de la Unión Europea y regula los derechos digitales de los ciudadanos conforme a la Constitución.

NLP – Procesamiento de Lenguaje Natural: Campo de las ciencias de la computación, de la inteligencia artificial y de la lingüística que estudia las interacciones entre los ordenadores y el lenguaje humano, así como los detalles computacionales de las lenguas naturales.

OWL – Lenguaje Web de Ontologías: Lenguaje de marcado para publicar y compartir datos usando ontologías en la WWW. Tiene como objetivo facilitar un modelo de marcado construido sobre RDF y codificado en XML.

PYMES – Pequeñas y Medianas Empresas: Empresas que cuentan con ciertos límites ocupacionales y financieros prefijados por los estados o regiones.

QA – Aseguramiento de Calidad: Disciplina orientada a garantizar el correcto funcionamiento de las soluciones tecnológicas mediante su prueba sistemática.

RDF – Marco de Descripción de Recursos: Estándar del W3C para describir recursos web y sus relaciones mediante tripletas (sujeto, predicado, objeto), utilizado en la web semántica.

REST – Transferencia de Estado Representacional: Estilo de arquitectura de *software* para crear servicios web.

SPA – Aplicación de Página Única: Arquitectura de aplicación web donde toda la lógica y el contenido se cargan en una sola página, mejorando la experiencia del usuario.

SSE – Eventos Enviados por el Servidor: Tecnología web que permite a un servidor enviar actualizaciones automáticas e información en tiempo real a un navegador web a través de una conexión HTTP estándar.

SUS – Escala de Usabilidad del Sistema: Instrumento para medir la usabilidad de un sistema desde la perspectiva del usuario.

UML – Lenguaje Unificado de Modelado: Lenguaje gráfico de modelado para visualizar, especificar, construir y documentar un sistema.

URI – Identificador de Recursos Uniforme: Cadena de caracteres que identifica los recursos –físicos o abstractos– de una red de forma unívoca.

W3C – World Wide Web Consortium: Consorcio internacional que desarrolla estándares web para garantizar la interoperabilidad y el crecimiento de la web.

WCAG – Directrices de Accesibilidad para el Contenido Web: Normas internacionales para garantizar que el contenido web sea accesible para personas con discapacidad.

WoW – Modelo de Trabajo: Conjunto de principios, metodologías y prácticas que un equipo o una organización adopta para llevar a cabo su trabajo de manera eficiente y efectiva.

WWW – World Wide Web: Sistema de distribución de documentos de hipertexto interconectados y accesibles a través de Internet mediante un navegador web.

XML – Lenguaje de Marcado Extensible: Metalenguaje que permite definir lenguajes de marcas, desarrollado por el World Wide Web Consortium y utilizado para almacenar datos en forma legible.

APÉNDICE A

CUESTIONARIO DE USABILIDAD

Evaluación de usabilidad – OntoNova

Instrucciones. Complete las tres tareas siguientes utilizando la aplicación, sin ayuda del evaluador. Al terminar, valore cada afirmación marcando una casilla de 1 (totalmente en desacuerdo) a 5 (totalmente de acuerdo).

Tareas. Genere una ontología a partir de un texto propio de su ámbito. Edite el grafo resultante creando una clase nueva y renombrando una relación. Exporte el resultado al formato que desee.

Afirmación	1	2	3	4	5
1. Creo que me gustaría utilizar este sistema con frecuencia.	☐	☐	☐	☐	☐
2. Encuentro el sistema innecesariamente complejo.	☐	☐	☐	☐	☐
3. Creo que el sistema es fácil de usar.	☐	☐	☐	☐	☐
4. Creo que necesitaría el apoyo de un técnico para poder utilizar este sistema.	☐	☐	☐	☐	☐
5. Encuentro que las diversas funciones del sistema están bien integradas.	☐	☐	☐	☐	☐
6. Creo que hay demasiada inconsistencia en este sistema.	☐	☐	☐	☐	☐
7. Imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente.	☐	☐	☐	☐	☐
8. Encuentro el sistema muy engorroso de utilizar.	☐	☐	☐	☐	☐
9. Me siento muy seguro utilizando el sistema.	☐	☐	☐	☐	☐
10. Necesité aprender muchas cosas antes de poder empezar a utilizar este sistema.	☐	☐	☐	☐	☐

Edad: Profesión: ¿Conocía las ontologías? Sí ☐ No ☐

APÉNDICE B

RESULTADOS DEL CUMPLIMIENTO DE LA LEY DE IA DE LA UE

TABLA B.1

RESPUESTAS REGISTRADAS EN EL CUESTIONARIO DE CONFORMIDAD

Sección del cuestionario	Respuesta marcada
Tipo de entidad	Proveedor
Modificaciones aguas abajo	Ninguna
Alto riesgo (Anexo I, secciones A y B)	Ninguna
Alto riesgo (Anexo III)	Ninguna
Alcance	Puesta en servicio de sistemas IA en la UE
Sistemas excluidos	Actividad de I+D en IA Componentes con licencias libres
Sistemas prohibidos (Artículo 5)	Ninguna
Sistemas transparentes (Artículo 50)	Ninguna

Excluidos: Sistemas de código abierto. Hasta que el sistema se comercialice o se ponga en servicio como parte de un sistema de alto riesgo, prohibido, de uso general o sujeto a obligaciones de transparencia, queda excluido del ámbito de aplicación de la ley (Artículo 2, apartado 12).

Excluidos: Investigación y desarrollo. Las actividades de investigación y desarrollo quedan excluidas hasta la comercialización o puesta en servicio del sistema (Artículo 2, apartados 6 y 8).

Obligaciones en materia de alfabetización en IA. Como proveedor, deben garantizarse conocimientos suficientes sobre IA en quienes operen el sistema, atendiendo a su formación y al contexto de uso (Artículo 4).

APÉNDICE C

DECLARACIÓN DE USO DE IA GENERATIVA

He utilizado IA generativa en este trabajo:

☐ Sí; ☐ No;

1. Reflexión sobre comportamiento legal, ético y responsable

Pregunta	
1. En mi interacción con herramientas de Inteligencia Artificial Generativa he facilitado datos de carácter confidencial contando siempre con la debida autorización de los interesados.	
Sí, he usado estos datos con la autorización correspondiente.	No, no he usado datos de carácter confidencial.
2. En mi interacción con herramientas de Inteligencia Artificial Generativa he facilitado materiales protegidos por derechos de autoría contando con la autorización respectiva.	
Sí, he usado estos materiales con autorización de los titulares de derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley: • Obra en dominio público • Obra licenciada (licencias Creative Commons) • Uso de fragmentos con fines de investigación (derecho de cita)	No, no he usado materiales protegidos por derechos de autoría.

3. En mi interacción con herramientas de Inteligencia Artificial Generativa he facilitado datos de carácter personal con la debida autorización de los interesados.	
Sí, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad .	No, no he usado datos de carácter personal.
4. Mi utilización de la herramienta de Inteligencia Artificial Generativa ha respetado sus términos de uso , así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado.	
Sí	No

2. Declaración de uso técnico

Declaro haber hecho uso del sistema de IAG (Claude Sonnet 5) para:

Documentación y redacción

• Búsqueda de bibliografía:

Ejemplo 1. **Utilizando la instrucción:** «¿Qué libro, artículo o fuente oficial introduce el concepto de la piramide de testing?», **teniendo como interacción** la declaración de la fuente y el año de publicación del libro *Succeeding with Agile* de Mike Cohn, 2009, en unión a una breve explicación de la pirámide y su relación con la metodología ágil.
 Ejemplo 2. **Utilizando la instrucción:** «¿Quién introduce el concepto de Shift Left? Porque la pirámide de testing parece contribuir a ello», **teniendo como interacción** la mención de la fuente y el año de publicación del artículo de revista *Shift Left Testing* de Larry Smith, 2001, en conjunto con una explicación de la relación entre ambos conceptos.

• Resumen de bibliografía consultada:

Ejemplo 1. **Utilizando la instrucción:** «Revisa Arquitectura Limpia de C.Martin y extrae el índice de contenidos pues me gustaría citarlo en un requisito de diseño que menciona el escalado horizontal», **teniendo como interacción** la propuesta del capítulo 16 del libro con un resumen del mismo.

Desarrollar contenido específico

• Asistencia en el desarrollo de líneas de código (programación):

Ejemplo 1. **Utilizando la instrucción:** «Could you prepare a playwright e2e test passing the application these Domain 2 PDF of less than 5MB and compare the results with these Domain 1 metrics so we can test agent chain performance?», **teniendo como interacción** la propuesta de construcción de una prueba cuya ejecución destapa un tipo MIME incorrecto, la desalineación de instrucciones y colisiones de identificadores.

- **Procesos de optimización:**

Ejemplo 1. **Utilizando la instrucción:** «Given nexolabs.txt which its ontology generation is failing with the error “Malformed LLM response from <http://vllm:8000/v1>: Expecting “,” delimiter: line 20709 column 1 (char 39087)”, please analyze these python snippets and provide a STDOUT solution», **teniendo como interacción** el diagnóstico razonado de la causa raíz del error enlazado a propuestas de optimizaciones secundarias como secuencias de parada, la detección de truncamiento y la ampliación de reparaciones del validador determinista.

3. Reflexión sobre utilidad

El uso de la inteligencia artificial generativa en este trabajo es coherente con el sentimiento expuesto en las conclusiones personales de la memoria, donde me declaro afortunado de haberme formado en el sistema educativo en su ausencia. Mantengo un rechazo absoluto a que la inteligencia artificial generativa participe en las fases de innovación e imaginación de un proyecto, pues considero que la concepción de ideas, la motivación y las decisiones creativas son íntegramente humanas. En esta iniciativa, toda propuesta arquitectónica nace de mi criterio a partir de las necesidades expuestas por mi gerente de proyecto —la tutora—, y la herramienta se limita a asistir en su implementación. Su fortaleza diferencial la encuentro, en cambio, en el ámbito de la calidad del *software*. Como asistente de pruebas ha demostrado una capacidad de sobreanalizar casos de uso, casos límite y condiciones de carrera que adelanta la detección de defectos a fases tan tempranas del ciclo de vida que hasta el enfoque de desplazamiento a la izquierda da las gracias por ello. Como debilidad, señalo la necesidad permanente de supervisión, pues la herramienta puede errar con facilidad —bibliografía incluida—, lo que me empuja a verificar las salidas propuestas con una disciplina que, lejos de ser un coste, ha reforzado mi propio aprendizaje, obligándome a comprender cada corrección antes de aceptarla.

